

Evaluación Parcial N°3

Explotación y Mitigación de Vulnerabilidades Web Estudiante

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación
OCY1102	Ciberseguridad en Desarrollo	4 horas	35 %

1. Situación evaluativa

	Ejecución práctica	x	Entrega de encargo		Presentación
--	--------------------	---	--------------------	--	--------------

2. Instrucciones generales

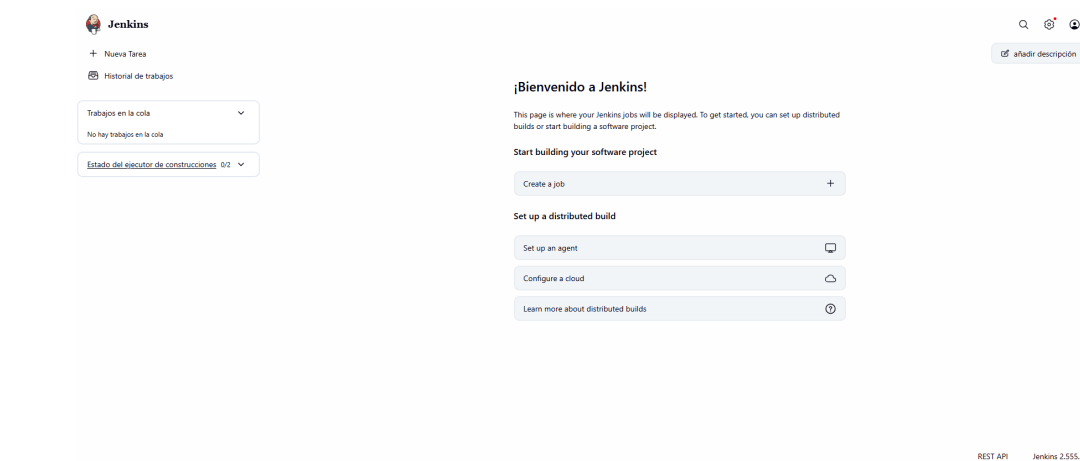
Integrantes: Catrina Corral y Fernanda Manriquez

▪ **Tareas:**

Implementación de Pipeline e Identificación y Corrección de Vulnerabilidades:

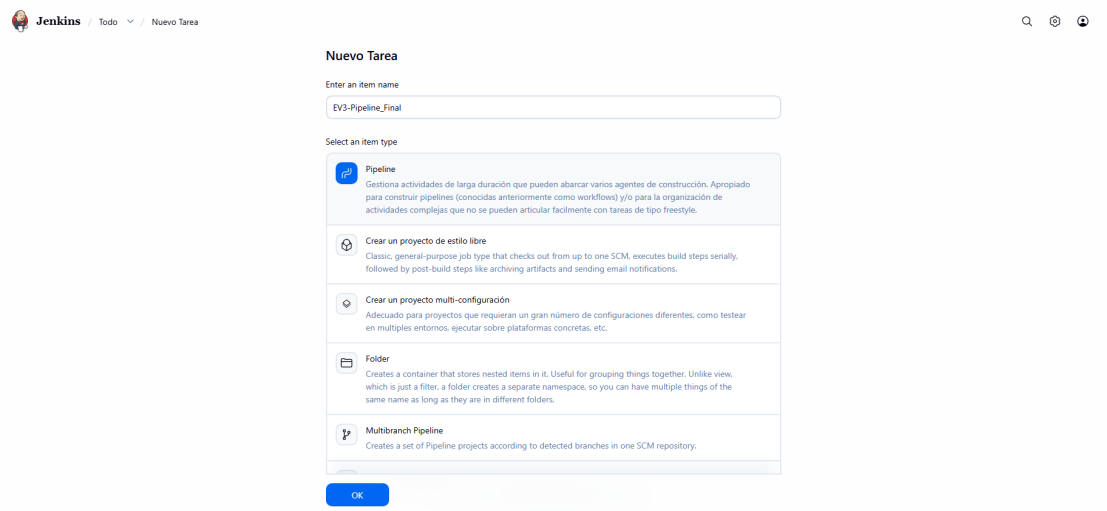
- Crea un pipeline de CI/CD en Jenkins que incluya las siguientes etapas: construcción, pruebas y despliegue. (Pegar Pantallazos)

Captura 1: Acceso al panel principal de Jenkins.



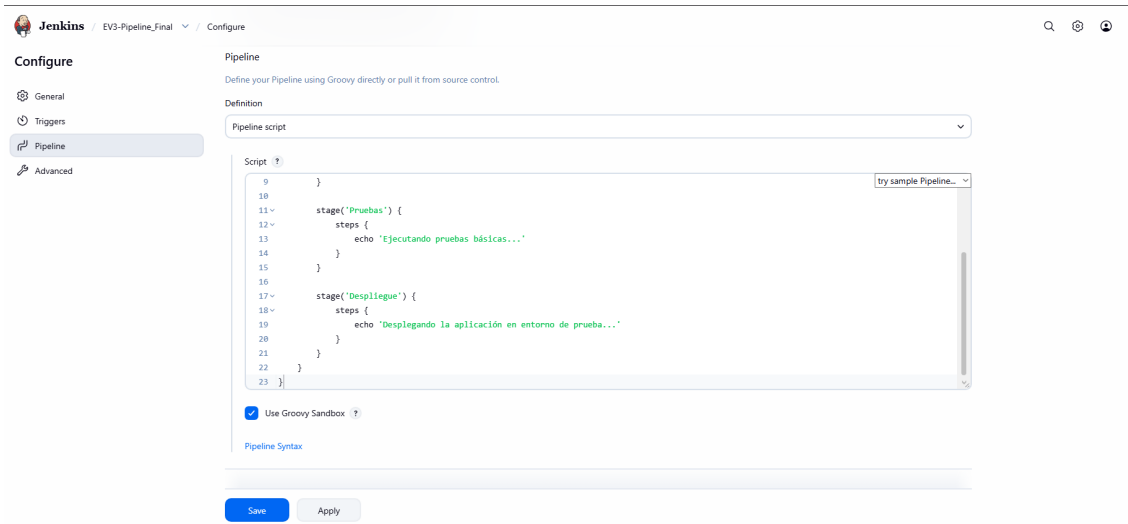
Descripción: Se accedió correctamente al panel principal de Jenkins, verificando que el servidor de integración continua se encuentra operativo. Esta evidencia respalda el inicio de la implementación del entorno CI/CD solicitado en la evaluación, permitiendo la creación y ejecución de tareas automatizadas dentro del proceso DevSecOps.

Captura 2: Creación de nueva tarea de tipo Pipeline en Jenkins.



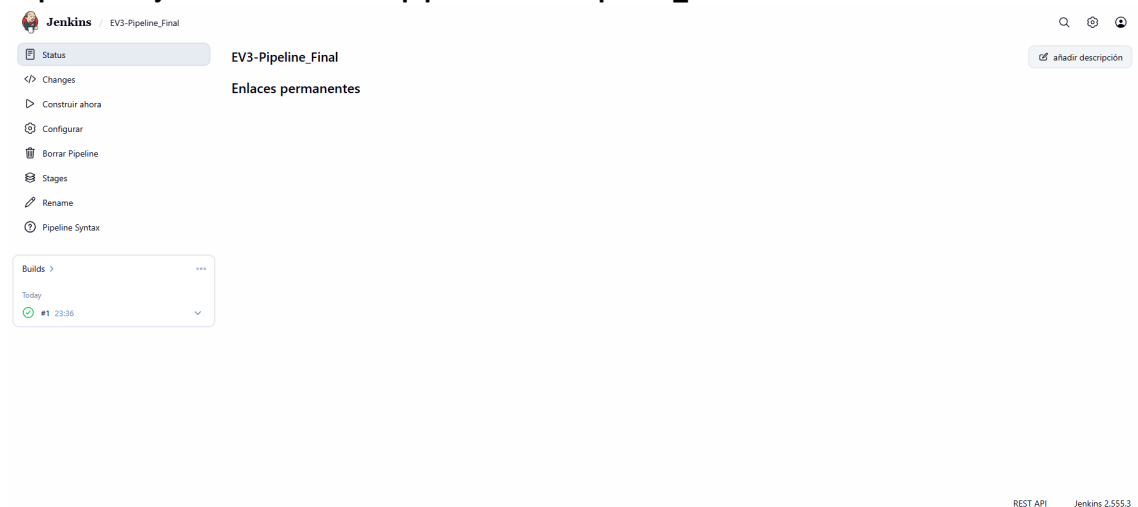
Descripción: Se creó una nueva tarea en Jenkins con el nombre **“EV3-Pipeline_Final”**, seleccionando el tipo **Pipeline**. Esta configuración permite implementar el flujo CI/CD requerido en la rúbrica, preparando el entorno para automatizar las etapas de construcción, pruebas y despliegue de la aplicación.

Captura 3: Configuración del pipeline mediante script declarativo en Jenkins.



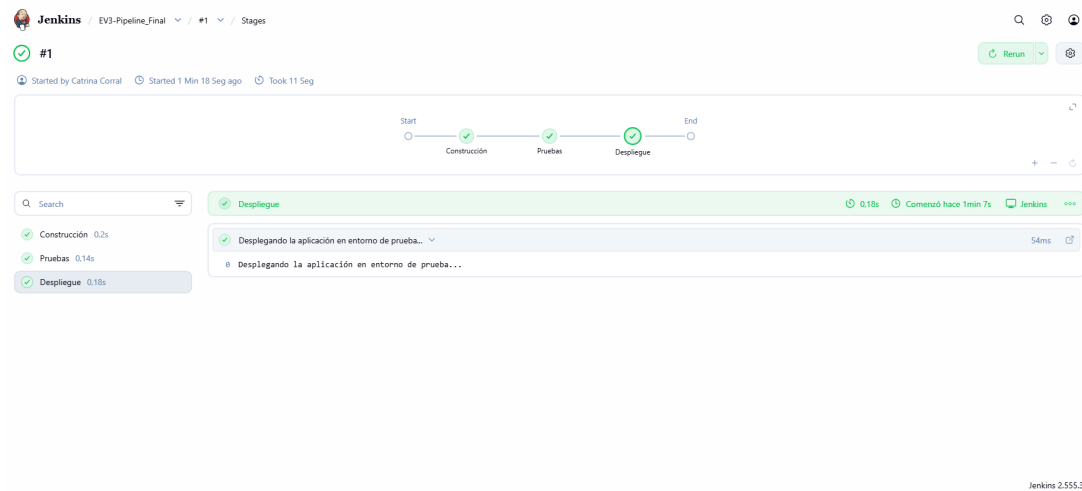
Descripción: Se configuró el pipeline mediante un script declarativo en Jenkins, definiendo las etapas principales del flujo CI/CD: **Construcción, Pruebas y Despliegue**. Esta evidencia cumple con el requerimiento de crear un pipeline que integre las fases básicas del ciclo de vida del desarrollo, permitiendo posteriormente incorporar revisiones de seguridad y pruebas automatizadas.

Captura 4: Ejecución exitosa del pipeline “EV3-Pipeline_Final” en Jenkins.



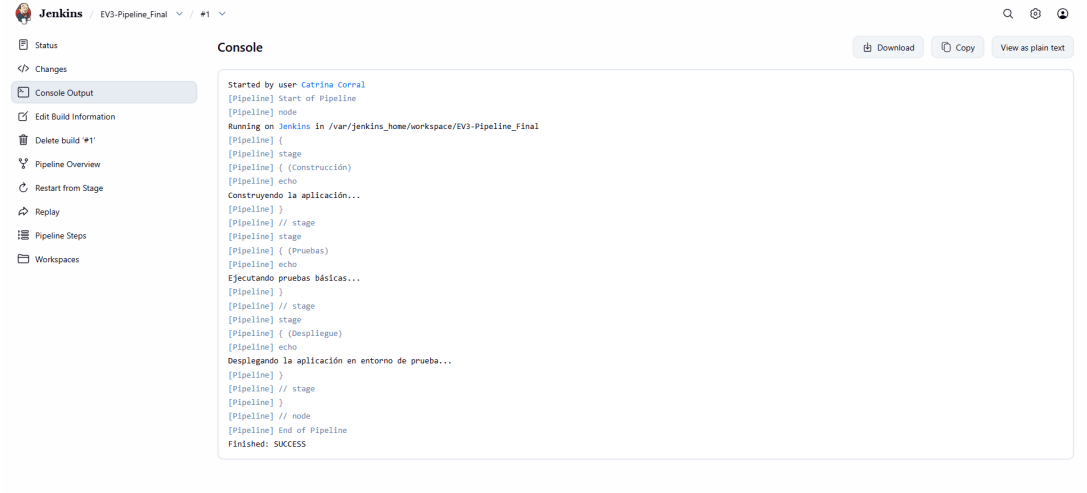
Descripción: Se ejecutó el pipeline “EV3-Pipeline_Final”, generándose correctamente el **build #1** con estado exitoso. Esta evidencia demuestra que el flujo CI/CD fue ejecutado sin errores, validando la correcta configuración inicial del pipeline solicitado en la evaluación.

Captura 5: Vista de etapas del build #1 del pipeline “EV3-Pipeline_Final”.



Descripción: Se visualizan las etapas **Construcción, Pruebas y Despliegue** ejecutadas correctamente dentro del build **#1**. Esta captura evidencia la trazabilidad del proceso automatizado, ya que permite verificar cada fase del pipeline y confirmar que el flujo CI/CD se completó de principio a fin sin errores.

Captura 6: Salida de consola del build #1 del pipeline “EV3-Pipeline_Final”.



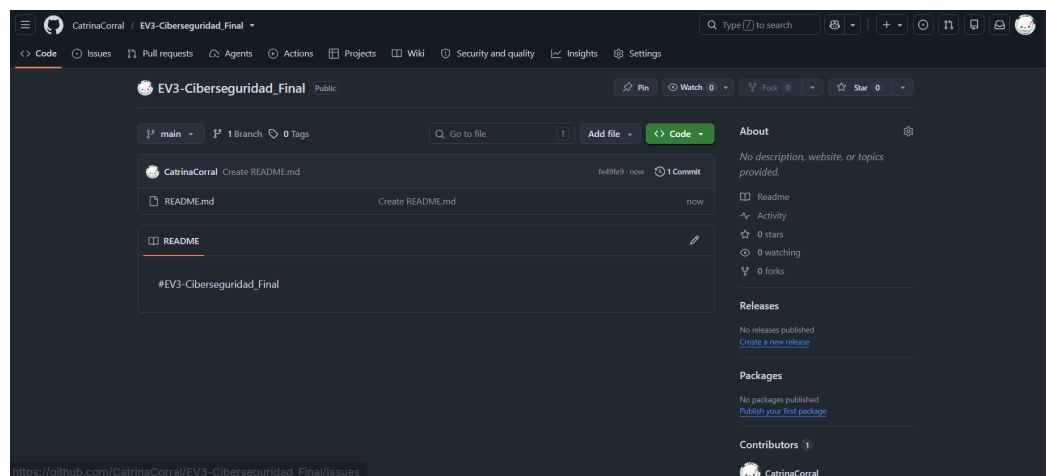
```
started by user Catrina Corral
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/EV3-Pipeline_Final
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Construcción)
[Pipeline] echo
Construyendo la aplicación...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Pruebas)
[Pipeline] echo
Ejecutando pruebas básicas...
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Despliegue)
[Pipeline] echo
Desplegando la aplicación en entorno de prueba...
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Descripción: Se muestra el log completo de ejecución del pipeline, evidenciando el paso por las etapas de **Construcción, Pruebas y Despliegue** mediante sus respectivos mensajes de salida. El proceso finaliza con el estado **“Finished: SUCCESS”**, lo que confirma la correcta ejecución del flujo CI/CD y aporta trazabilidad técnica para el informe, tal como solicita la rúbrica en relación con la documentación de las actividades realizadas en el pipeline.

- Utilizando el código vulnerable proporcionado, realiza revisiones de seguridad continuas en todas las etapas del ciclo de vida

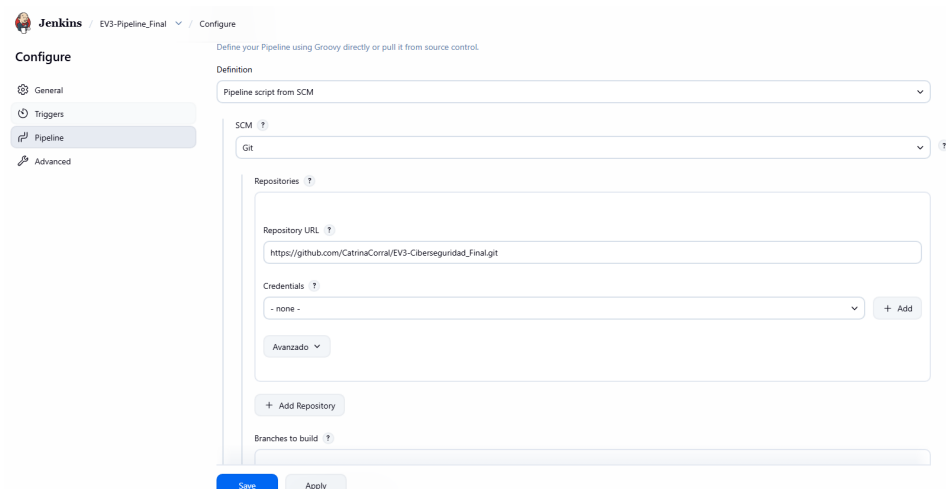
del desarrollo para identificar y mitigar vulnerabilidades. (Pegar Pantallazos)

Captura 7: Repositorio del proyecto vulnerable en GitHub.



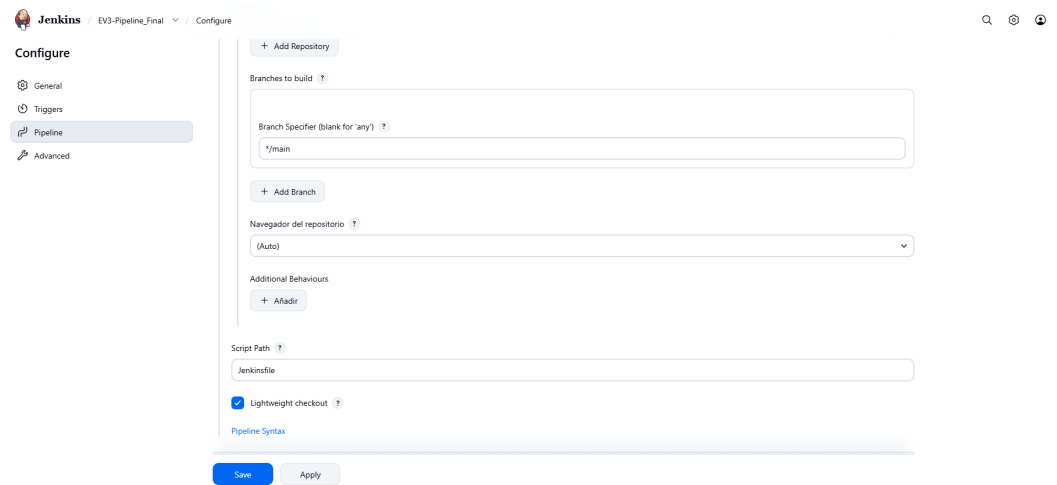
Descripción: Se visualiza el repositorio “**EV3-Ciberseguridad_Final**” creado en GitHub, el cual será utilizado para almacenar el código vulnerable proporcionado y mantener trazabilidad de los cambios realizados durante el desarrollo. Esta evidencia respalda el inicio del proceso de revisión de seguridad dentro del ciclo de vida del software, permitiendo registrar commits, modificaciones y futuras correcciones aplicadas para la identificación y mitigación de vulnerabilidades.

Captura 8: Configuración del pipeline desde repositorio GitHub mediante SCM.



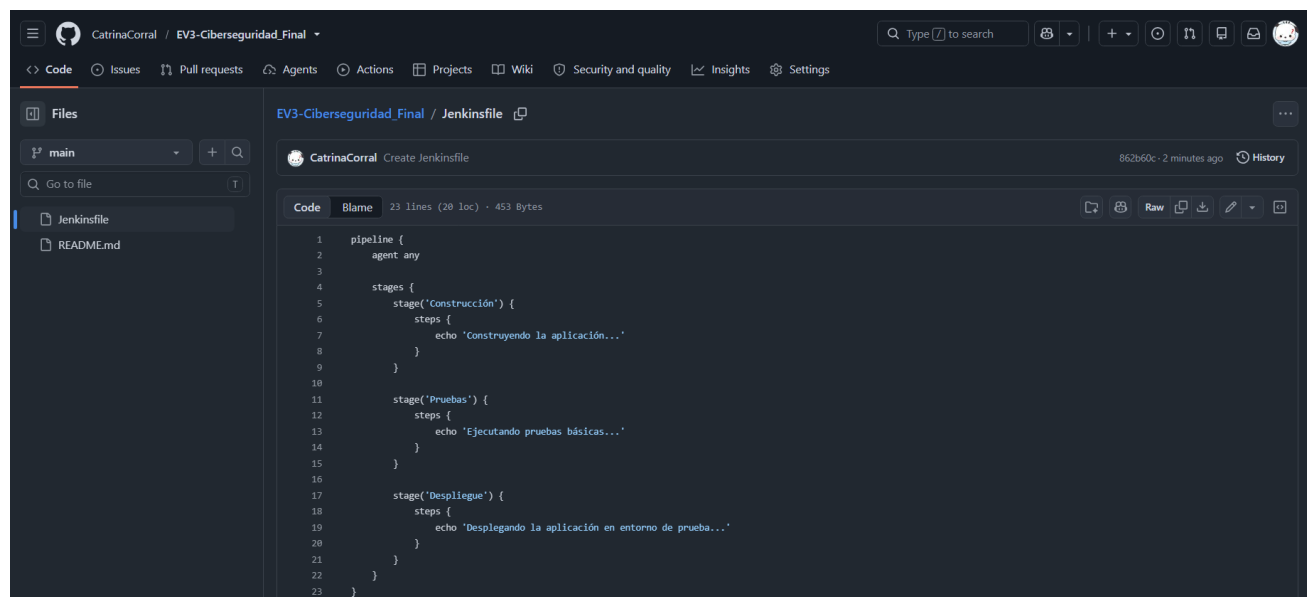
Descripción: Se configuró el pipeline “EV3-Pipeline_Final” utilizando la opción **Pipeline script from SCM**, seleccionando **Git** como sistema de control de versiones e ingresando la URL del repositorio “EV3-Ciberseguridad_Final”. Esta evidencia demuestra la integración entre Jenkins y GitHub, permitiendo que el pipeline obtenga el código fuente directamente desde el repositorio para ejecutar revisiones y procesos automatizados dentro del ciclo de vida del desarrollo.

Captura 9: Configuración de rama y Jenkinsfile para ejecución desde GitHub.



Descripción: Se configuró el pipeline para construir desde la rama **main** del repositorio, indicando como ruta del script el archivo **Jenkinsfile**. Esta configuración permite que Jenkins lea automáticamente las instrucciones del pipeline desde el código fuente almacenado en GitHub, asegurando que las revisiones y ejecuciones se realicen sobre la versión controlada del proyecto.

Captura 10: Jenkinsfile versionado en el repositorio GitHub.



The screenshot shows the GitHub interface for the repository 'EV3-Ciberseguridad_Final'. The 'Jenkinsfile' is selected, showing its code. The code is a Jenkins pipeline definition with three stages: 'Construcción', 'Pruebas', and 'Despliegue'. Each stage contains an 'echo' step to indicate progress.

```

1 pipeline {
2   agent any
3
4   stages {
5     stage('Construcción') {
6       steps {
7         echo 'Construyendo la aplicación...'
8       }
9     }
10
11    stage('Pruebas') {
12      steps {
13        echo 'Ejecutando pruebas básicas...'
14      }
15    }
16
17    stage('Despliegue') {
18      steps {
19        echo 'Desplegando la aplicación en entorno de prueba...'
20      }
21    }
22  }
23 }

```

Descripción: Se visualiza el archivo **Jenkinsfile** almacenado dentro del repositorio “**EV3-Ciberseguridad_Final**”, el cual contiene la definición del pipeline con las etapas de **Construcción, Pruebas y Despliegue**. Esta evidencia demuestra que la configuración del flujo CI/CD se encuentra versionada en GitHub, permitiendo trazabilidad sobre los cambios realizados y facilitando la ejecución automatizada desde Jenkins.

Captura 11: Registro de dependencia Flask en el repositorio del proyecto.

```
MINGW64:/c/Users/danae/Downloads/EV3-Ciberseguridad_Final

danae@LAPTOP-NSV0ER1Q MINGW64 ~
$ cd ~/Downloads

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads
$ git clone https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git
Cloning into 'EV3-Ciberseguridad_Final'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads
$ cd EV3-Ciberseguridad_Final

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ echo "flask>=3.0.0" > requirements.txt

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    requirements.txt

nothing added to commit but untracked files present (use "git add" to track)

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git add requirements.txt
warning: in the working copy of 'requirements.txt', LF will be replaced by CRLF the next time Git touches it

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git commit -m "Agrega archivo requirements"
[main 3540ce1] Agrega archivo requirements
1 file changed, 1 insertion(+)
create mode 100644 requirements.txt

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git push
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 346 bytes | 86.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git
 862b60c..3540ce1  main -> main

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ cat requirements.txt
flask>=3.0.0

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ |
```

Descripción: Se evidencia la clonación del repositorio “EV3-Ciberseguridad_Final” desde GitHub y la creación del archivo **requirements.txt**, donde se declara la dependencia **Flask==3.0.0**. Posteriormente, el archivo es agregado, confirmado mediante commit y enviado al repositorio remoto. Esta captura demuestra trazabilidad en el control de versiones y permite identificar las dependencias utilizadas por la aplicación para futuras revisiones de seguridad.

Captura 12: Actualización del Jenkinsfile para incorporar revisión de seguridad.

```
danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ notepad Jenkinsfile

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   Jenkinsfile

no changes added to commit (use "git add" and/or "git commit -a")

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git add Jenkinsfile

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git commit -m "Actualiza pipeline Security Scan"
[main 4c277d5] Actualiza pipeline Security Scan
 1 file changed, 63 insertions(+), 5 deletions(-)

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 942 bytes | 314.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git
 3540ce1..4c277d5  main -> main

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$
```

Descripción: Se evidencia la modificación del archivo **Jenkinsfile** mediante Git Bash, donde posteriormente se agregan los cambios al control de versiones con git add, se realiza el commit **“Actualiza pipeline Security Scan”** y se envía la actualización al repositorio remoto con git push. Esta captura demuestra que el pipeline fue actualizado para incorporar una etapa relacionada con revisión de seguridad, manteniendo trazabilidad de los cambios realizados en el ciclo de vida del desarrollo.

Captura 13: Incorporación del código vulnerable proporcionado al repositorio.

```
danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        create_db.py
        vulnerable_app.py

nothing added to commit but untracked files present (use "git add" to track)

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git add create_db.py vulnerable_app.py
warning: in the working copy of 'create_db.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'vulnerable_app.py', LF will be replaced by CRLF the next time Git touches it

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git commit -m "Agrega codigo vulnerable proporcionado"
[main 2823048] Agrega codigo vulnerable proporcionado
 2 files changed, 172 insertions(+)
 create mode 100644 create_db.py
 create mode 100644 vulnerable_app.py

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 2.09 KiB | 153.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Catrinacorral/EV3-Ciberseguridad_Final.git
 4c277d5..2823048  main -> main

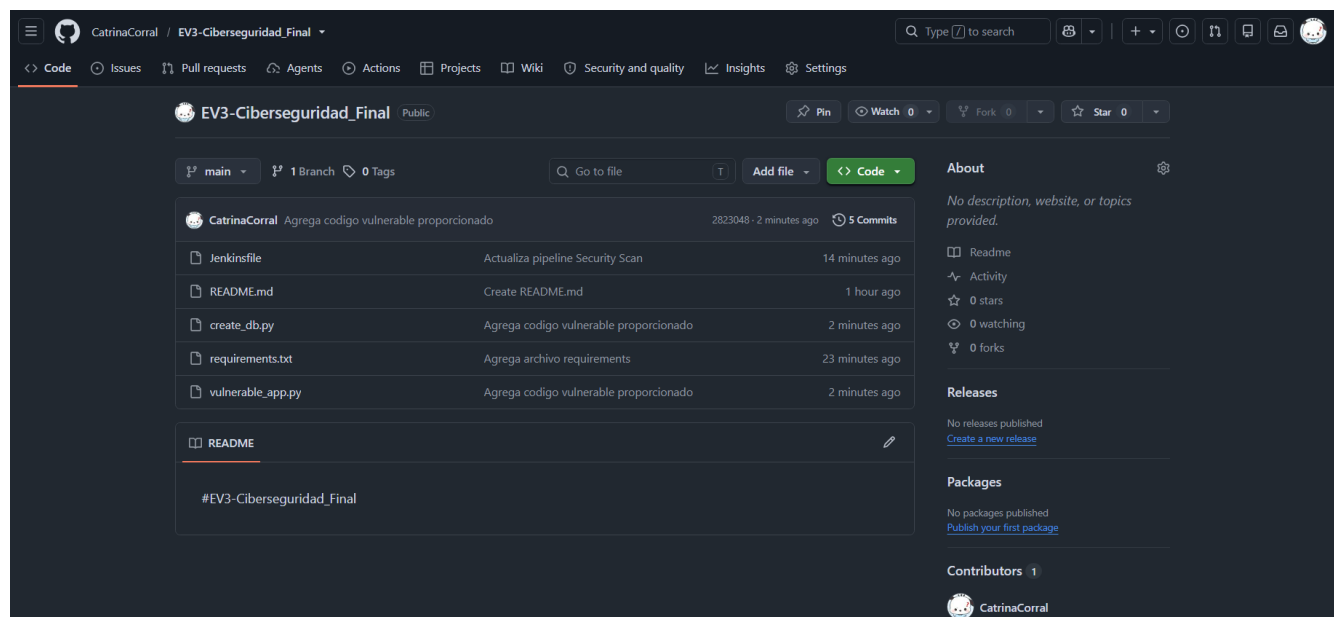
danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ !
```

Descripción: Se evidencia la incorporación de los archivos `create_db.py` y `vulnerable_app.py` al repositorio “EV3-Ciberseguridad_Final” mediante Git Bash. Ambos archivos fueron agregados con `git add`, confirmados mediante el `commit` “**Agrega codigo vulnerable proporcionado**” y enviados al repositorio remoto con `git push origin main`. Esta evidencia demuestra que el código vulnerable fue integrado al proyecto para ser utilizado posteriormente en las revisiones de seguridad.

Captura 14: Verificación del código vulnerable y archivos del proyecto en GitHub.



Descripción: Se visualiza el repositorio “EV3-Ciberseguridad_Final” en GitHub con los archivos principales del proyecto: **Jenkinsfile**, **README.md**, **create_db.py**, **requirements.txt** y **vulnerable_app.py**. Además, se observa el commit “**Agrega codigo vulnerable proporcionado**”, lo que evidencia que el código vulnerable fue cargado correctamente al repositorio y quedó disponible para ser analizado mediante el pipeline.

Captura 15: Ejecución del pipeline actualizado con etapa de Security Scan.

Jenkins / EV3-Pipeline_Final / #4 / Stages

Started by Catrina Corral / Started 22 Seg ago / Build has been executing for 22 Seg

Start → Checkout SCM → Construccion → Python Version → Install Dependencies → Security Scan → Pruebas → Despliegue → Post Actions → End

Search

- ✓ Checkout SCM 2s
- ✓ Construccion 0.58s
- ✓ Python Version 3s
- ✓ Install Dependencies 1min
- ✓ Security Scan 12s
- ✓ Pruebas 2s
- ✓ Despliegue 0.38s
- ✓ Post Actions 0.28s

1min Comenzó hace 1min 2s Jenkins

```
python3 -m venv $[ENV] || true . $[ENV]/bin/activate || true pip install --upgrade pip || true pip install -r requirements.txt || true pip install bandit safety pytest || true
```

```
0 + python3 -m venv venv
1 + . venv/bin/activate
2 + deactivate nondestructive
3 + [ -n ]
4 + [ -n ]
5 + hash -r
6 + [ -n ]
7 + unset VIRTUAL_ENV
8 + unset VIRTUAL_ENV_PROMPT
9 + [ ! nondestructive = nondestructive ]
10 + unname
11 + export VIRTUAL_ENV=/var/jenkins_home/workspace/EV3-Pipeline_Final/venv
12 + _OLD_VIRTUAL_PATH=/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
13 + PATH=/var/jenkins_home/workspace/EV3-Pipeline_Final/venv/bin:/opt/java/openjdk/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
14 + export PATH
```

localhost:8080

Descripción: Se visualiza la ejecución del build #4 del pipeline “EV3-Pipeline_Final”, ahora con un flujo actualizado respecto al Jenkinsfile mostrado inicialmente en la Captura 10. En esta nueva versión se incorporan más etapas al pipeline, tales como **Checkout SCM**, **Python Version**, **Install Dependencies**, **Security Scan**, **Pruebas**, **Despliegue** y **Post Actions**. Esto evidencia que el código del pipeline fue modificado para integrar revisiones de seguridad y validaciones adicionales dentro del proceso automatizado.

Captura 16: Resultados del Security Scan con Bandit en el pipeline actualizado.

```
. $[ENV]/bin/activate || true echo "===== RESULTADOS SECURITY SCAN - BANDIT =====" bandit -r . -x ./venv -f txt || true echo "===== RESULTADOS DEPENDENCY SCAN - SAF... 12s

22 + echo ===== RESULTADOS SECURITY SCAN - BANDIT =====
23 ===== RESULTADOS SECURITY SCAN - BANDIT =====
24 + bandit -r . -x ./venv -f txt
25 [main] INFO profile include tests: None
26 [main] INFO profile exclude tests: None
27 [main] INFO cli include tests: None
28 [main] INFO cli exclude tests: None
29 [main] INFO running on Python 3.13.5
30 Run started:2026-07-02 01:32:29.531760+00:00
31
32 Test results:
33 >> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
34 Severity: Medium Confidence: Low
35 CME: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
36 More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608\_hardcoded\_sql\_expressions.html
37 Location: ./vulnerable_app.py:35:22
38 34         if "" OR "" in password:
39 35             query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
40 36             user = conn.execute(query).fetchone()
41
42 -----
43 >> Issue: [B201:flask_debug_true] A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.
44 Severity: High Confidence: Medium
45 CME: CWE-94 (https://cwe.mitre.org/data/definitions/94.html)
46 More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b201\_flask\_debug\_true.html
47 Location: ./vulnerable_app.py:124:4
```

Descripción: Se visualiza la ejecución de la etapa **Security Scan** dentro del build **#4** del pipeline **“EV3-Pipeline_Final”**. En esta fase se ejecuta la herramienta **Bandit**, la cual analiza el archivo **vulnerable_app.py** e identifica vulnerabilidades en el código, entre ellas una posible **inyección SQL** asociada a la construcción de consultas mediante strings, clasificada con severidad **Medium** y relacionada con **CWE-89**. Además, se detecta la configuración insegura de Flask con **debug=True**, clasificada con severidad **High**, debido a que puede exponer el depurador y permitir riesgos de ejecución no autorizada.

Captura 17: Ejecución de análisis de dependencias con Safety dentro del Security Scan.

Jenkins / EV3-Pipeline_Final / #4 / Stages

Search

- ✓ Checkout SCM 2s
- ✓ Construcción 0.58s
- ✓ Python Version 3s
- ✓ Install Dependencies 1min
- ✓ Security Scan 12s
- ✓ Pruebas 2s
- ✓ Despliegue 0.38s
- ✓ Post Actions 0.28s

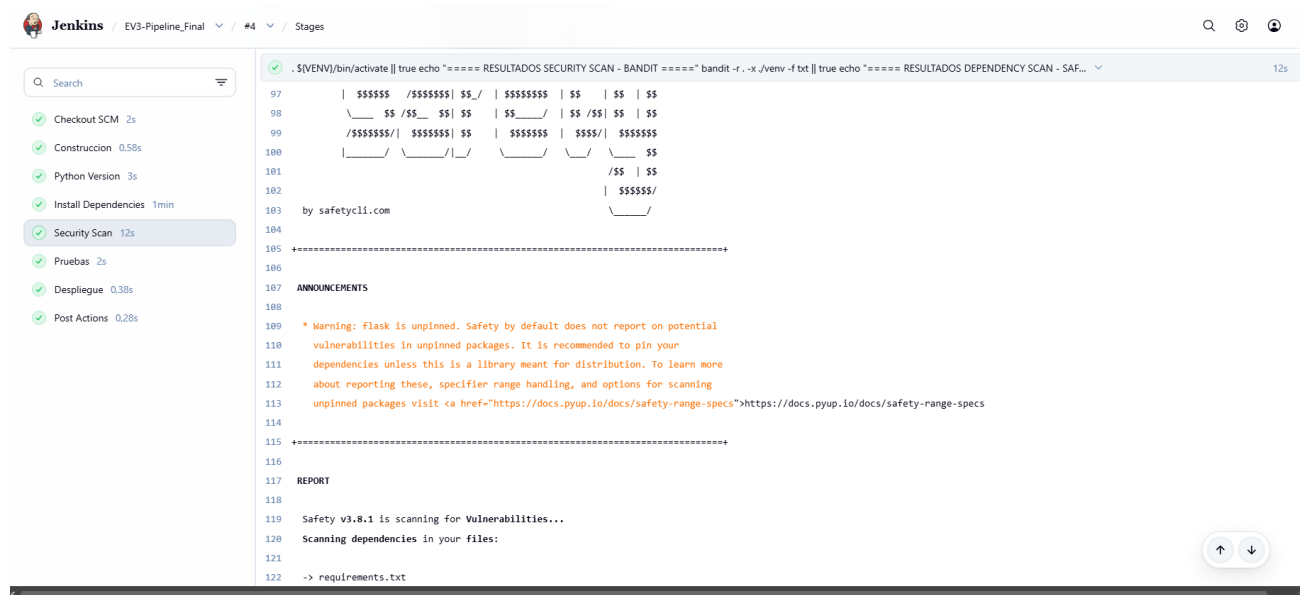
```

1. $[ENV]/bin/activate || true echo "===== RESULTADOS SECURITY SCAN - BANDIT =====" bandit -r -x ./venv -f txt || true echo "===== RESULTADOS DEPENDENCY SCAN - SAF..."
71 + echo ===== RESULTADOS DEPENDENCY SCAN - SAFETY =====
72 ----- RESULTADOS DEPENDENCY SCAN - SAFETY -----
73 + safety check -r requirements.txt
74 /var/jenkins_home/workspace/EV3-Pipeline_Final/venv/lib/python3.13/site-packages/safety/auth/main.py:5: AuthlibDeprecationWarning: authlib.jose module is deprecated, please use josefc
instead.
75 It will be compatible before version 2.0.0.
76   from authlib.jose.errors import ExpiredTokenError
77
78
79 +-----+
80
81
82 DEPRECATED: this command ('check') has been DEPRECATED, and will be unsupported beyond 01 June 2024.
83
84
85 We highly encourage switching to the new 'scan' command which is easier to use, more powerful, and can be set up to mimic the deprecated command if required.
86
87
88 +-----+
89
90
91 +-----+
92
93          /$$$$$$          /$$
94          /$__ $ $         | $
95 /$$$$$$$ /$$$$$$ | $ $ _/ /$$$$$ /$$$$$ /$ /$ /$

```

Descripción: Se visualiza la continuación de la etapa **Security Scan** del build **#4**, donde además del análisis de código con Bandit se ejecuta la revisión de dependencias mediante **Safety** usando el archivo **requirements.txt**. Esta etapa permite verificar las librerías utilizadas por la aplicación e identificar posibles vulnerabilidades asociadas a dependencias instaladas en el proyecto. En la salida también se observa una advertencia de deprecación del comando **safety check**, lo que evidencia la necesidad de mantener actualizadas las herramientas de análisis utilizadas dentro del pipeline.

Captura 18: Advertencia de Safety sobre dependencia Flask en requirements.txt.



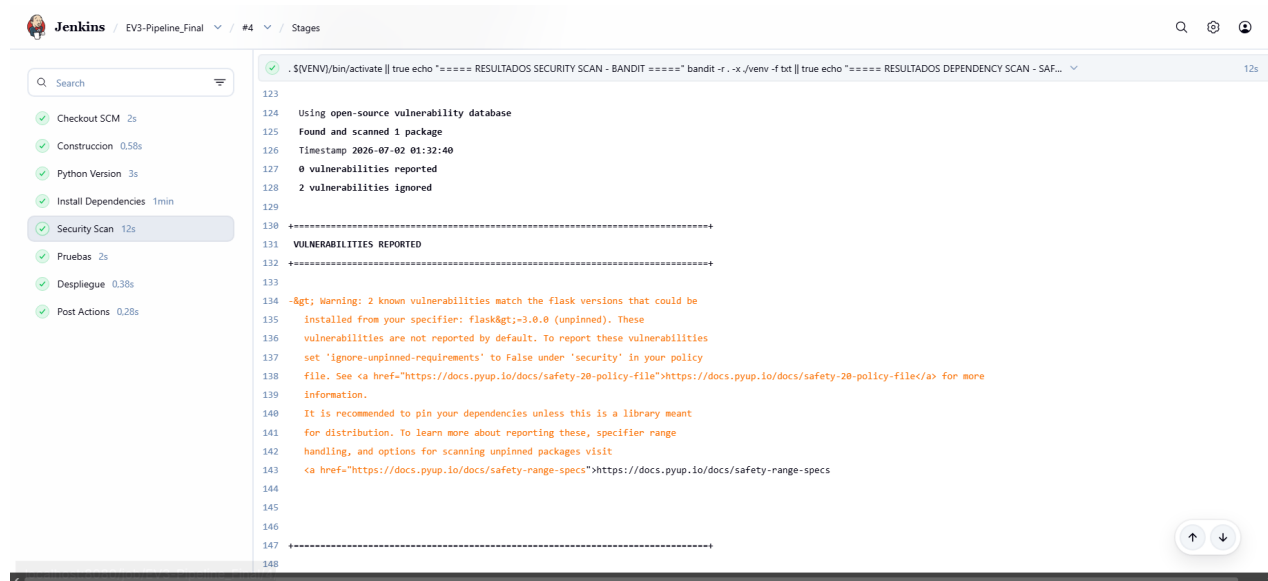
The screenshot shows the Jenkins interface for a pipeline named 'EV3-Pipeline_Final' at stage '#4'. The 'Security Scan' step is highlighted in the left sidebar, showing a duration of 12s. The main console output displays the results of a Safety scan. It begins with a decorative ASCII art header for 'bandit' and 'safetycli11.com'. Below this, a section titled 'ANNOUNCEMENTS' contains a warning: '* Warning: flask is unpinned. Safety by default does not report on potential vulnerabilities in unpinned packages. It is recommended to pin your dependencies unless this is a library meant for distribution. To learn more about reporting these, specifier range handling, and options for scanning unpinned packages visit https://docs.pyup.io/docs/safety-range-specs'. This is followed by a 'REPORT' section indicating that 'Safety v3.8.1 is scanning for Vulnerabilities...' and 'Scanning dependencies in your files:'. The output ends with a line pointing to 'requirements.txt'.

```
. $[ENV]/bin/activate || true echo "===== RESULTADOS SECURITY SCAN - BANDIT =====" bandit -r . -x ./venv -f txt || true echo "===== RESULTADOS DEPENDENCY SCAN - SAF..." 12s

97 | $$$$$$ /$$$$$$ $$_/ | $$$$$$$$ | $$ | $ | $
98 | _ $ $ /$ $ $ $ | $ _ $ | $ $ /$ $ | $ $ | $
99 | /$$$$$$$/ | $$$$$$ $ $ | $$$$$$ | $$$$/ | $$$$$$
100 | _ $ _ $ _ $ _ $ _ $ _ $ _ $ _ $ _ $
101 | _ $ _ $ _ $ _ $ _ $ _ $ _ $ _ $ _ $
102 | _ $ _ $ _ $ _ $ _ $ _ $ _ $ _ $ _ $
103 by safetycli11.com
104
105 +-----+
106
107 ANNOUNCEMENTS
108
109 * Warning: flask is unpinned. Safety by default does not report on potential
110 vulnerabilities in unpinned packages. It is recommended to pin your
111 dependencies unless this is a library meant for distribution. To learn more
112 about reporting these, specifier range handling, and options for scanning
113 unpinned packages visit <a href="https://docs.pyup.io/docs/safety-range-specs">https://docs.pyup.io/docs/safety-range-specs</a>
114
115 +-----+
116
117 REPORT
118
119 Safety v3.8.1 is scanning for Vulnerabilities...
120 Scanning dependencies in your files:
121
122 -> requirements.txt
```

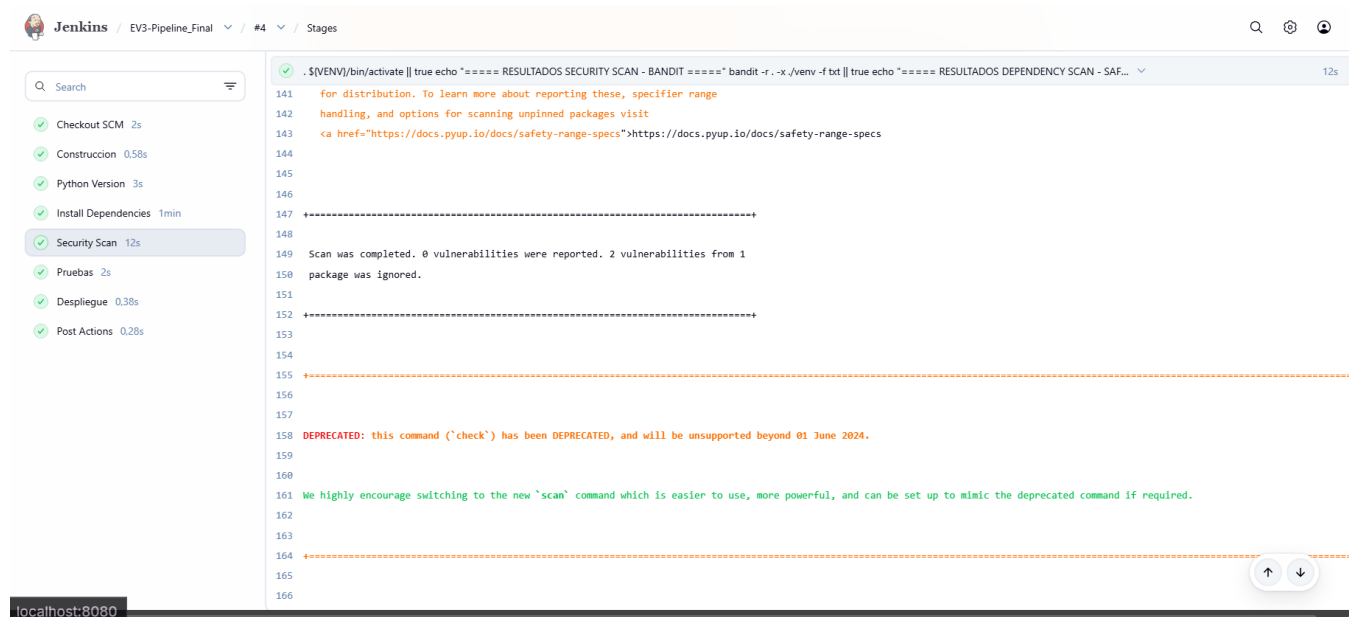
Descripción: Se visualiza el reporte generado por **Safety** dentro de la etapa **Security Scan** del build **#4**. La herramienta analiza el archivo **requirements.txt** y muestra una advertencia indicando que la dependencia **Flask** aparece sin una versión fijada correctamente, recomendando definir versiones específicas para evitar riesgos asociados a paquetes no controlados. Esta revisión permite detectar posibles debilidades en la gestión de dependencias antes de continuar con las siguientes etapas del pipeline.

Captura 19: Resultado del análisis de dependencias con Safety.



Descripción: Se visualiza el resultado del análisis de dependencias ejecutado por **Safety** dentro de la etapa **Security Scan** del build **#4**. La herramienta escanea el archivo **requirements.txt**, detectando un paquete analizado y mostrando una advertencia asociada a la especificación **Flask>=3.0.0**, indicando que existen **2 vulnerabilidades conocidas** que podrían coincidir con versiones instalables de Flask, pero que no son reportadas por defecto al estar la dependencia sin una versión completamente fijada. Esta evidencia permite identificar un riesgo en la gestión de dependencias y refuerza la necesidad de definir versiones seguras y controladas.

Captura 20: Finalización del análisis de dependencias con Safety.



Jenkins / EV3-Pipeline_Final / #4 / Stages

Search

- ✓ Checkout SCM 2s
- ✓ Construcción 0.58s
- ✓ Python Version 3s
- ✓ Install Dependencies 1min
- ✓ Security Scan 12s
- ✓ Pruebas 2s
- ✓ Despliegue 0.38s
- ✓ Post Actions 0.28s

```
141 . ${ENVV}/bin/activate || true echo "==== RESULTADOS SECURITY SCAN - BANDIT ====" bandit -r. -x ./venv -f txt || true echo "==== RESULTADOS DEPENDENCY SCAN - SAF...
142 for distribution. To learn more about reporting these, specifier range
143 handling, and options for scanning unpinned packages visit
144 <a href="https://docs.pyup.io/docs/safety-range-specs">https://docs.pyup.io/docs/safety-range-specs
145
146
147 +-----+
148 Scan was completed. 0 vulnerabilities were reported. 2 vulnerabilities from 1
149 package was ignored.
150
151 +-----+
152
153
154
155 +-----+
156
157
158 DEPRECATED: this command (`check`) has been DEPRECATED, and will be unsupported beyond 01 June 2024.
159
160
161 We highly encourage switching to the new `scan` command which is easier to use, more powerful, and can be set up to mimic the deprecated command if required.
162
163
164 +-----+
165
166
```

localhost:8080

Descripción: Se visualiza la finalización del análisis de dependencias ejecutado por **Safety** dentro de la etapa **Security Scan** del build **#4**. El reporte indica que el escaneo fue completado, mostrando **0 vulnerabilidades reportadas**, pero también advierte que **2 vulnerabilidades de 1 paquete fueron ignoradas**, debido a la forma en que se encuentra definida la dependencia. Además, se observa una advertencia indicando que el comando **safety check** está deprecado, recomendando migrar al comando **safety scan**.

Captura 21: Salida de consola del build **#4** con obtención del Jenkinsfile desde GitHub.

Jenkins / EV3-Pipeline_Final / #4

Status

Changes

Console Output

Edit Build Information

Delete build '#4'

Git Build Data

Pipeline Overview

Restart from Stage

Replay

Pipeline Steps

Workspaces

Previous Build

Download Copy View as plain text

Console

```
Started by user Catrina Corral
Obtained Jenkinsfile from git https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/EV3-Pipeline_Final
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
Cloning the remote repository
Cloning repository https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git
> git init /var/jenkins_home/workspace/EV3-Pipeline_Final # timeout=10
Fetching upstream changes from https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git
> git --version # timeout=10
> git --version # 'git version 2.47.3'
> git fetch --tags --force --progress -- https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git +refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final.git # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 282304801af3df298337f70d1c1163fa8e8a8211 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 282304801af3df298337f70d1c1163fa8e8a8211 # timeout=10
Commit message: "Agrega codigo vulnerable proporcionado"
First time build. Skipping changelog.
[Pipeline] }
```

Descripción: Se visualiza la salida de consola del build #4 del pipeline “EV3-Pipeline_Final”, donde Jenkins obtiene el **Jenkinsfile** directamente desde el repositorio GitHub “EV3-Ciberseguridad_Final”. En el log se observa el proceso de clonación del repositorio, la lectura de la rama **main** y el commit asociado “**Agrega codigo vulnerable proporcionado**”, confirmando que el pipeline se ejecuta sobre el código vulnerable versionado en el repositorio.

Captura 22: Referencia técnica de la vulnerabilidad CWE-89 SQL Injection.


Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities



New to CWE?
[Start here!](#)

Home > CWE List > CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') (4.20)

Home | About | Learn | Access Content | Community | Search

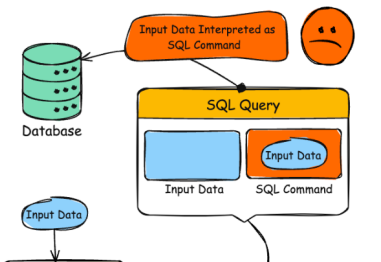
ID Lookup: Go

CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

Weakness ID: 89
Vulnerability Mapping: ALLOWED
 Abstraction: Base

View customized information:

▼ Description



The product constructs all or part of an SQL command using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component. Without sufficient removal or quoting of SQL syntax in user-controllable inputs, the generated SQL query can cause those inputs to be interpreted as SQL instead of ordinary user data.

Descripción: Se visualiza la página oficial de **CWE-89**, correspondiente a la vulnerabilidad **SQL Injection**. Esta evidencia permite respaldar técnicamente la vulnerabilidad detectada por Bandit en **vulnerable_app.py**, donde se identificó una consulta SQL construida con entradas del usuario. CWE indica que esta debilidad ocurre cuando la aplicación no neutraliza correctamente datos externos antes de usarlos en una consulta SQL.

Captura 23: Referencia técnica de la vulnerabilidad CWE-94 Code Injection.


Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities



New to CWE?
[Start here!](#)

[Home](#) > [CWE List](#) > [CWE-94: Improper Control of Generation of Code \('Code Injection'\) \(4.20\)](#)

[Home](#) | [About](#) | [Learn](#) | [Access Content](#) | [Community](#) | [Search](#)

ID Lookup:

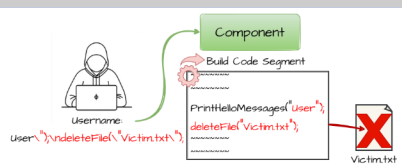
CWE-94: Improper Control of Generation of Code ('Code Injection')

Weakness ID: 94
Vulnerability Mapping: ALLOWED (with careful review of mapping notes)
 Abstraction: Base

View customized information:

Description

The product constructs all or part of a code segment using externally-influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the syntax or behavior of the intended code segment.



Alternate Terms

Code Injection

Common Consequences

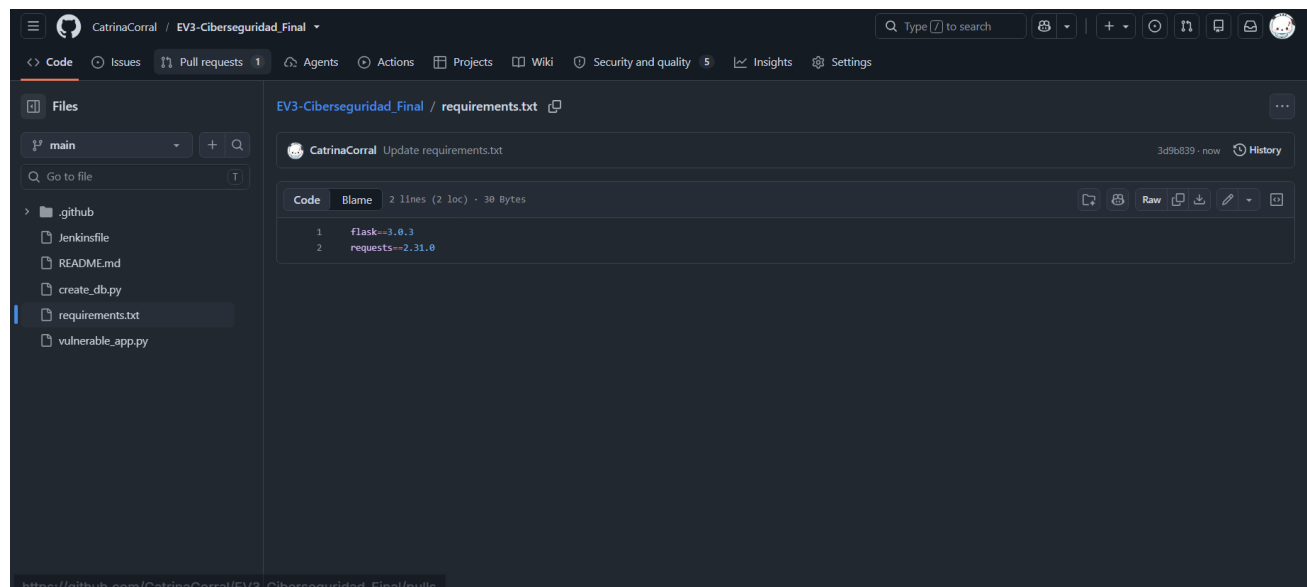
Impact

<https://cwe.mitre.org/index.html>

Scope: Access Control

Descripción: Se visualiza la página oficial de **CWE-94**, correspondiente a **Code Injection**. Esta evidencia respalda la vulnerabilidad detectada por Bandit relacionada con la configuración insegura de Flask en modo **debug=True**, ya que esta debilidad ocurre cuando una aplicación permite que entradas externas modifiquen el comportamiento o ejecución del código.

- Corrige cualquier vulnerabilidad encontrada en el código proporcionado. (Pegar Pantallazos)

Captura 24: Corrección de dependencias en requirements.txt.

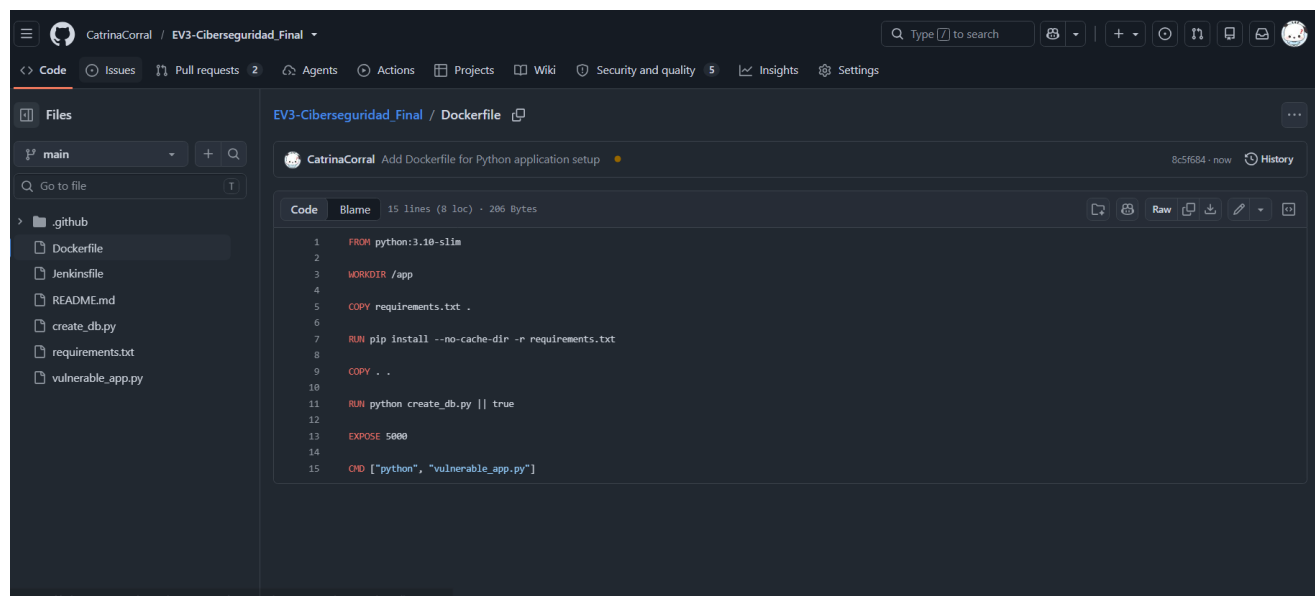
The screenshot shows a GitHub repository named 'EV3-Ciberseguridad_Final' by user 'CatrinaCorral'. The 'Code' tab is active, displaying the file 'requirements.txt'. The file content is as follows:

```
1 flask==3.0.3
2 requests==2.31.0
```

The interface includes a sidebar with the file explorer showing 'main' branch and files like '.github', 'Jenkinsfile', 'README.md', 'create_db.py', 'requirements.txt', and 'vulnerable_app.py'. The top navigation bar shows 'Code', 'Issues', 'Pull requests', 'Agents', 'Actions', 'Projects', 'Wiki', 'Security and quality', 'Insights', and 'Settings'.

Descripción: Se visualiza la actualización del archivo **requirements.txt**, donde las dependencias fueron fijadas con versiones específicas, incluyendo **flask==3.0.3** y **requests==2.31.0**. Esta corrección permite controlar las librerías utilizadas por la aplicación y mitigar el riesgo detectado anteriormente por Safety, relacionado con dependencias no fijadas o mal definidas.

Captura 25: Incorporación de Dockerfile para ejecución controlada de la aplicación.

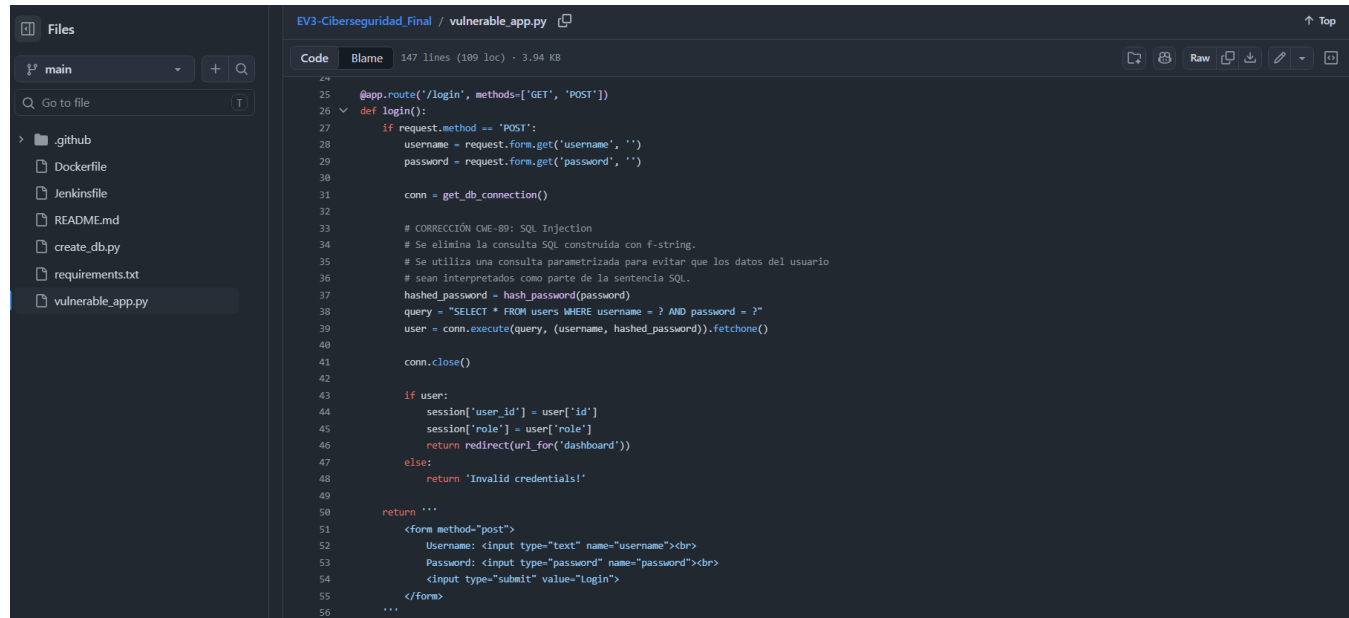


The screenshot shows a GitHub repository named 'EV3-Ciberseguridad_Final' by user 'CatrinaCorral'. The 'Dockerfile' is selected, showing a commit by 'CatrinaCorral' with the message 'Add Dockerfile for Python application setup'. The Dockerfile content is as follows:

```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 RUN python create_db.py || true
12
13 EXPOSE 5000
14
15 CMD ["python", "vulnerable_app.py"]
```

Descripción: Se visualiza la creación del archivo **Dockerfile** en el repositorio, configurando una imagen base de Python, el directorio de trabajo, la instalación de dependencias desde **requirements.txt**, la inicialización de la base de datos y la ejecución de **vulnerable_app.py**. Esta configuración permite ejecutar la aplicación en un contenedor, manteniendo un entorno controlado y reproducible para aplicar y validar las correcciones de seguridad.

Captura 26: Corrección de vulnerabilidad SQL Injection en **vulnerable_app.py**.

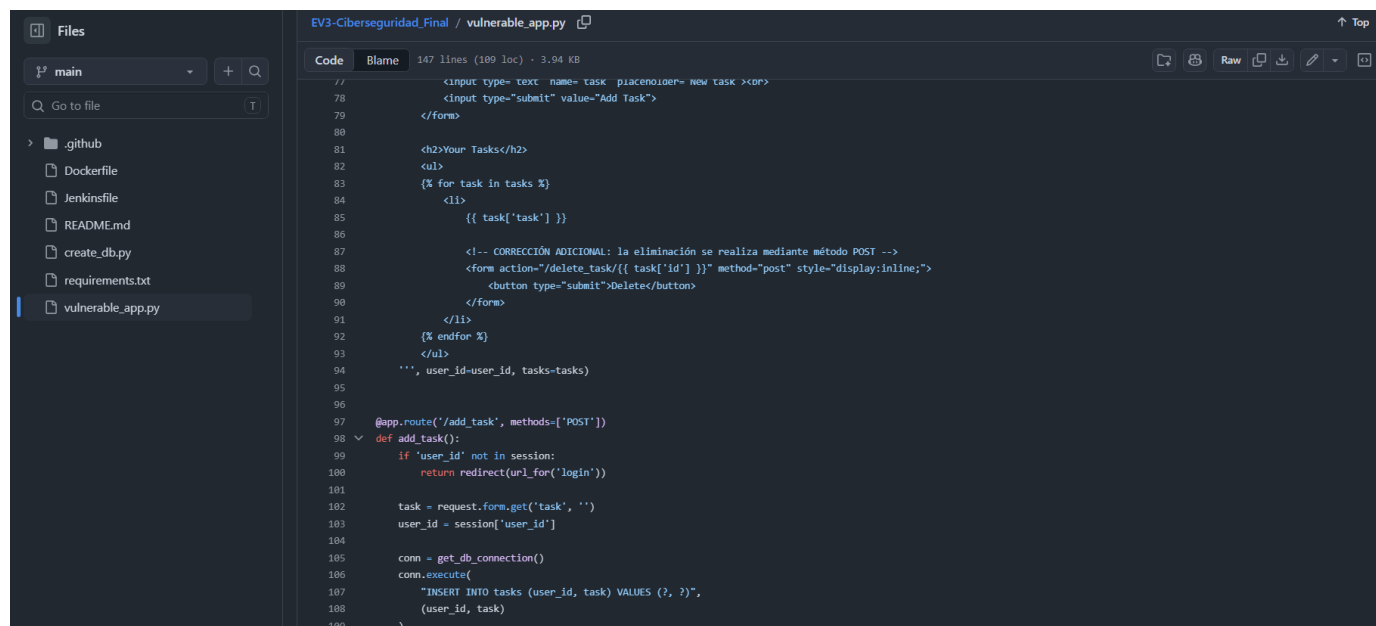


The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'main' with files like 'Dockerfile', 'Jenkinsfile', 'README.md', 'create_db.py', 'requirements.txt', and 'vulnerable_app.py'. The code editor shows the 'vulnerable_app.py' file with the following code:

```
25 @app.route('/login', methods=['GET', 'POST'])
26 def login():
27     if request.method == 'POST':
28         username = request.form.get('username', '')
29         password = request.form.get('password', '')
30
31         conn = get_db_connection()
32
33         # CORRECCIÓN CWE-89: SQL Injection
34         # Se elimina la consulta SQL construida con f-string.
35         # Se utiliza una consulta parametrizada para evitar que los datos del usuario
36         # sean interpretados como parte de la sentencia SQL.
37         hashed_password = hash_password(password)
38         query = "SELECT * FROM users WHERE username = ? AND password = ?"
39         user = conn.execute(query, (username, hashed_password)).fetchone()
40
41         conn.close()
42
43         if user:
44             session['user_id'] = user['id']
45             session['role'] = user['role']
46             return redirect(url_for('dashboard'))
47         else:
48             return 'Invalid credentials!'
49
50     return '''
51     <form method="post">
52         Username: <input type="text" name="username"><br>
53         Password: <input type="password" name="password"><br>
54         <input type="submit" value="Login">
55     </form>
56     '''
```

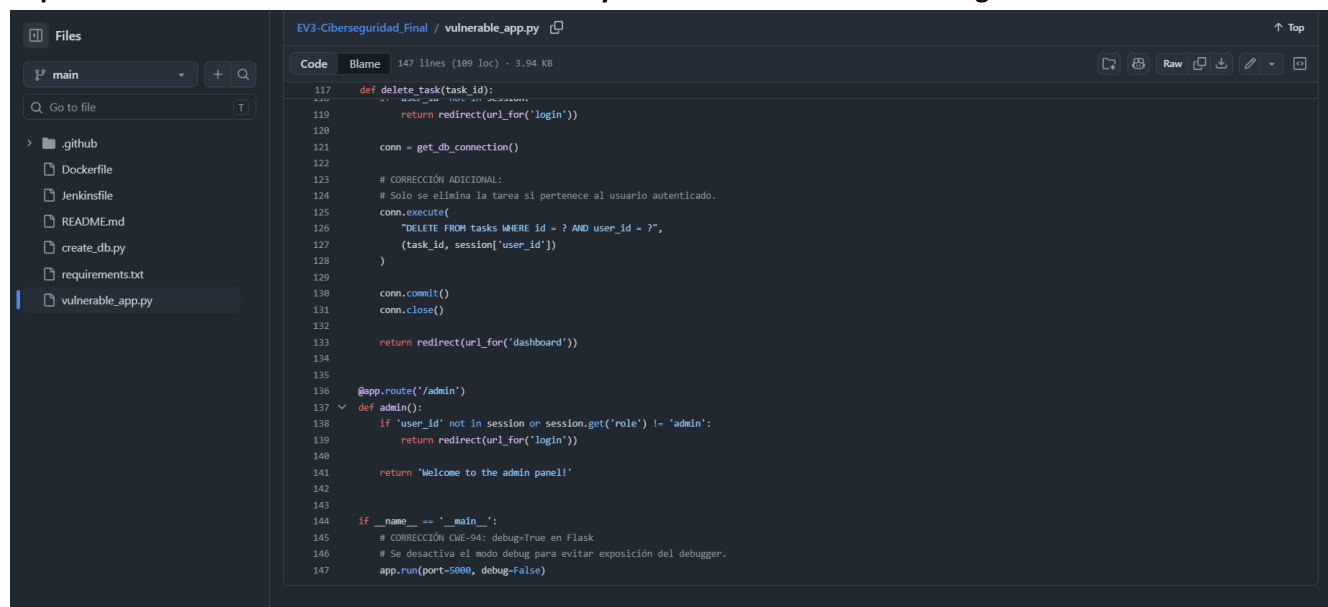
Descripción: Se visualiza la modificación del archivo **vulnerable_app.py**, donde se corrige la vulnerabilidad **CWE-89: SQL Injection** detectada previamente. La consulta SQL ya no se construye mediante concatenación o f-string, sino que se reemplaza por una **consulta parametrizada**, separando los datos ingresados por el usuario de la sentencia SQL. Además, se aplica el uso de contraseña hasheada antes de validar las credenciales, fortaleciendo el proceso de autenticación.

Captura 27: Corrección adicional en eliminación e inserción de tareas.



```
177 <input type="text" name="task" placeholder="new task" ><br>
178 <input type="submit" value="Add Task">
179 </form>
180
181 <h2>Your Tasks</h2>
182 <ul>
183   {% for task in tasks %}
184     <li>
185       {{ task['task'] }}
186     </li>
187     <!-- CORRECCIÓN ADICIONAL: la eliminación se realiza mediante método POST -->
188     <form action="/delete_task/{{ task['id'] }}" method="post" style="display:inline;">
189       <button type="submit">Delete</button>
190     </form>
191   </li>
192   {% endfor %}
193 </ul>
194   {% if user_id == user_id %}
195     {{ user_id }}
196   </if>
197 @app.route('/add_task', methods=['POST'])
198 def add_task():
199     if 'user_id' not in session:
200         return redirect(url_for('login'))
201
202     task = request.form.get('task', '')
203     user_id = session['user_id']
204
205     conn = get_db_connection()
206     conn.execute(
207         "INSERT INTO tasks (user_id, task) VALUES (?, ?)",
208         (user_id, task)
209     )
```

Descripción: Se visualiza una corrección en **vulnerable_app.py**, donde la eliminación de tareas se realiza mediante el método **POST** en lugar de una acción insegura por URL. Además, en la función **add_task()** se utiliza una consulta parametrizada para insertar datos en la base de datos, evitando que las entradas del usuario sean interpretadas como parte de una sentencia SQL.

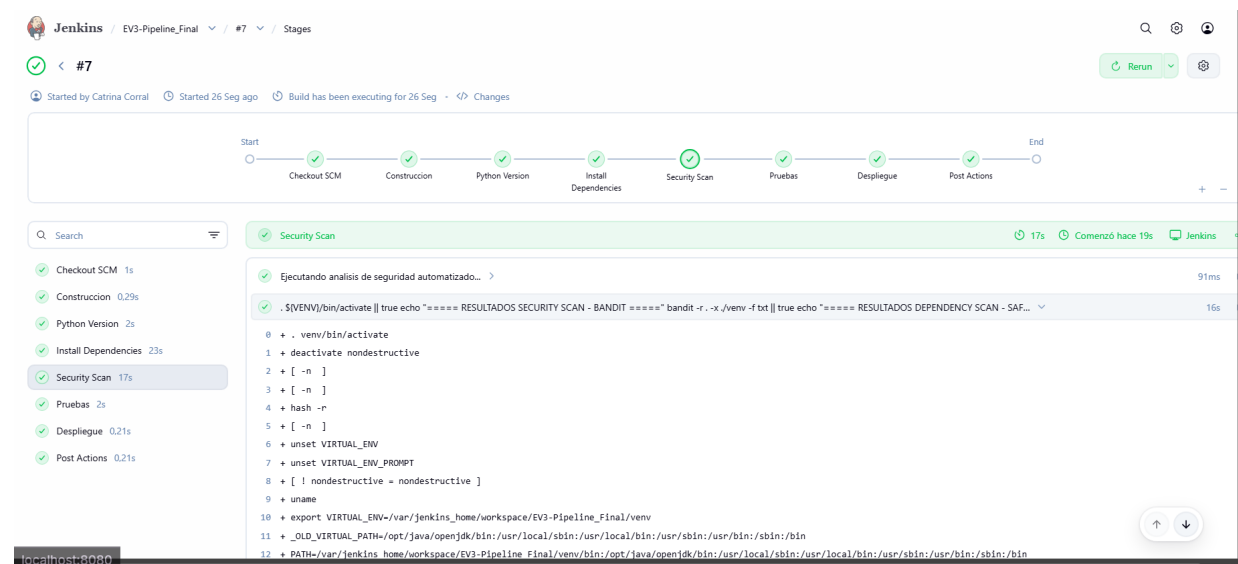
Captura 28: Corrección de eliminación de tareas y desactivación del modo debug.

```
117 def delete_task(task_id):
118     return redirect(url_for('login'))
119
120
121 conn = get_db_connection()
122
123 # CORRECCIÓN ADICIONAL:
124 # Solo se elimina la tarea si pertenece al usuario autenticado.
125 conn.execute(
126     "DELETE FROM tasks WHERE id = ? AND user_id = ?",
127     (task_id, session['user_id'])
128 )
129
130 conn.commit()
131 conn.close()
132
133 return redirect(url_for('dashboard'))
134
135
136 @app.route('/admin')
137 def admin():
138     if 'user_id' not in session or session.get('role') != 'admin':
139         return redirect(url_for('login'))
140
141     return 'Welcome to the admin panel!'
142
143
144 if __name__ == '__main__':
145     # CORRECCIÓN CVE-94: debug=True en Flask
146     # Se desactiva el modo debug para evitar exposición del debugger.
147     app.run(port=5000, debug=False)
```

Descripción: Se visualiza la corrección del archivo **vulnerable_app.py**, donde la eliminación de tareas se realiza mediante una consulta parametrizada y validando que la tarea pertenezca al usuario autenticado. Además, se corrige la configuración insegura de Flask, cambiando **debug=True** por **debug=False**, evitando la exposición del depurador en el entorno de ejecución.

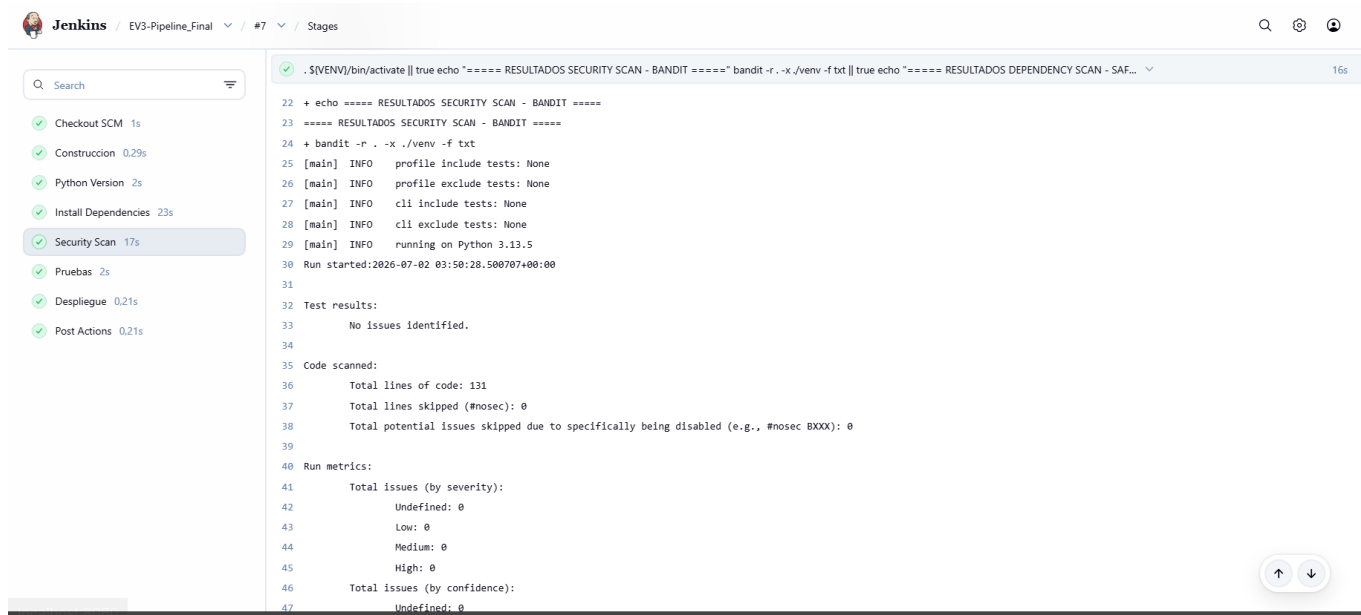
- Documenta todas las revisiones de seguridad realizadas, las vulnerabilidades identificadas y las correcciones aplicadas, incluyendo detalles de cómo fueron mitigadas. (Pegar Pantallazos)

Captura 29: Ejecución del Security Scan posterior a las correcciones aplicadas.



Descripción: Se visualiza el build **#7** del pipeline **“EV3-Pipeline_Final”**, donde se ejecuta nuevamente la etapa **Security Scan** después de aplicar correcciones en el código vulnerable. El pipeline muestra las etapas **Checkout SCM**, **Construcción**, **Python Version**, **Install Dependencies**, **Security Scan**, **Pruebas**, **Despliegue** y **Post Actions** ejecutadas correctamente, evidenciando que las revisiones de seguridad fueron integradas y validadas dentro del flujo CI/CD.

Captura 30: Validación de correcciones mediante Bandit sin vulnerabilidades detectadas.



The screenshot shows the Jenkins web interface for build #7 of the 'EV3-Pipeline_Final' job. The left sidebar lists the build stages: Checkout SCM (1s), Construcción (0.29s), Python Version (2s), Install Dependencies (23s), Security Scan (17s), Pruebas (2s), Despliegue (0.21s), and Post Actions (0.21s). The 'Security Scan' stage is selected and highlighted. The main panel displays the console output for this stage, which includes the execution of the Bandit security scanner. The output shows that no issues were identified, and the metrics for severity (Low, Medium, High) and confidence (Undefined) are all zero. The console output is as follows:

```
. $[ENV]/bin/activate || true echo "==== RESULTADOS SECURITY SCAN - BANDIT =====" bandit -r . -x ./venv -f txt || true echo "==== RESULTADOS DEPENDENCY SCAN - SAF... 16s

22 + echo ===== RESULTADOS SECURITY SCAN - BANDIT =====
23 ===== RESULTADOS SECURITY SCAN - BANDIT =====
24 + bandit -r . -x ./venv -f txt
25 [main] INFO profile include tests: None
26 [main] INFO profile exclude tests: None
27 [main] INFO cli include tests: None
28 [main] INFO cli exclude tests: None
29 [main] INFO running on Python 3.13.5
30 Run started:2026-07-02 03:50:28.500707+00:00
31
32 Test results:
33 No issues identified.
34
35 Code scanned:
36 Total lines of code: 131
37 Total lines skipped (#nosec): 0
38 Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0
39
40 Run metrics:
41 Total issues (by severity):
42 Undefined: 0
43 Low: 0
44 Medium: 0
45 High: 0
46 Total issues (by confidence):
47 Undefined: 0
```

Descripción: Se visualiza el resultado del análisis de seguridad ejecutado con **Bandit** dentro del build **#7** del pipeline. A diferencia del análisis anterior, el reporte muestra **“No issues identified”**, indicando que no se detectaron vulnerabilidades en el código revisado. Además, las métricas muestran **0 issues** en severidad baja, media y alta, lo que evidencia que las correcciones aplicadas fueron validadas correctamente.

Captura 31: Validación final de revisión de seguridad con Bandit y Safety.

The screenshot displays the Jenkins web interface for build #7 of the 'EV3-Pipeline_Final' job. The left sidebar shows a list of stages: Checkout SCM (1s), Construcción (0.29s), Python Version (2s), Install Dependencies (23s), Security Scan (17s), Pruebas (2s), Despliegue (0.21s), and Post Actions (0.21s). The 'Security Scan' stage is selected and highlighted. The main panel shows the console output for this stage, which includes the execution of a Bandit security scan and a Safety dependency scan. The Bandit scan results show 0 vulnerabilities detected across all confidence levels (Medium, High, Undefined, Low, Medium, High). The Safety scan output shows a deprecation warning for the 'authlib.jose' module and a message encouraging the use of the 'scan' command instead of the deprecated 'check' command.

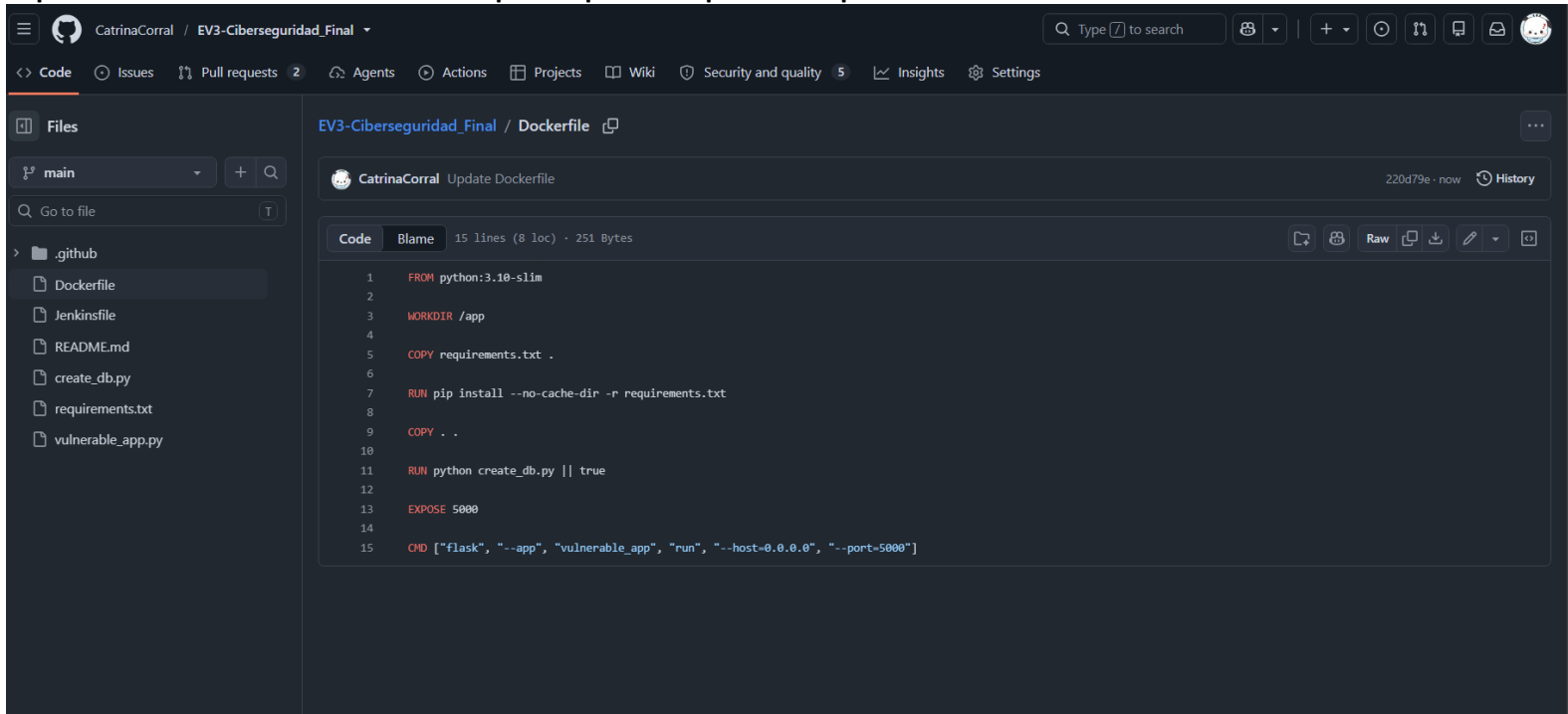
```
44 Medium: 0
45 High: 0
46 Total issues (by confidence):
47 Undefined: 0
48 Low: 0
49 Medium: 0
50 High: 0
51 Files skipped (0):
52 + echo ===== RESULTADOS DEPENDENCY SCAN - SAFETY =====
53 ===== RESULTADOS DEPENDENCY SCAN - SAFETY =====
54 + safety check -r requirements.txt
55 /var/jenkins_home/workspace/EV3-Pipeline_Final/venv/lib/python3.13/site-packages/safety/auth/main.py:5: AuthlibDeprecationWarning: authlib.jose module is deprecated, please use josefc
instead.
56 It will be compatible before version 2.0.0.
57 from authlib.jose.errors import ExpiredTokenError
58
59
60 =====
61
62
63 DEPRECATED: this command ('check') has been DEPRECATED, and will be unsupported beyond 01 June 2024.
64
65
66 We highly encourage switching to the new 'scan' command which is easier to use, more powerful, and can be set up to mimic the deprecated command if required.
67
68
```

Descripción: Se visualiza la etapa **Security Scan** del build **#7**, donde el análisis con **Bandit** mantiene el resultado en **0 vulnerabilidades detectadas** por severidad y confianza. Posteriormente, se ejecuta el análisis de dependencias con **Safety** sobre el archivo **requirements.txt**, dejando evidencia de la revisión de seguridad posterior a las correcciones aplicadas.

Pruebas Automatizadas de Seguridad:

- Integra OWASP ZAP en el pipeline de Jenkins para realizar pruebas de seguridad automatizadas. (Pegar Pantallazos)

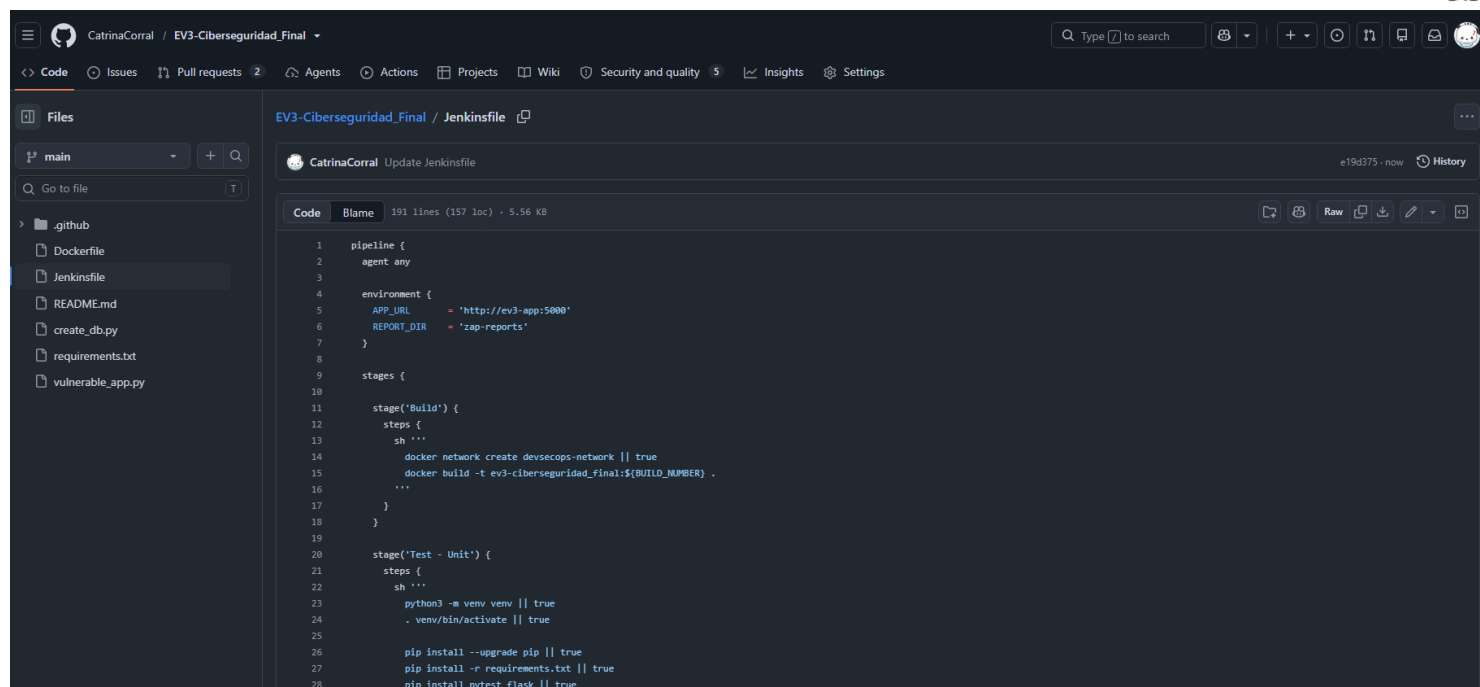
Captura 32: Actualización del Dockerfile para exponer la aplicación a pruebas OWASP ZAP.



```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 COPY . .
10
11 RUN python create_db.py || true
12
13 EXPOSE 5000
14
15 CMD ["flask", "--app", "vulnerable_app", "run", "--host=0.0.0.0", "--port=5000"]
```

Descripción: Se visualiza la actualización del archivo **Dockerfile**, donde la aplicación Flask se configura para ejecutarse en el puerto **5000** y aceptar conexiones desde **0.0.0.0**. Esta configuración permite que la aplicación quede disponible dentro del entorno de ejecución para ser analizada posteriormente por **OWASP ZAP** mediante pruebas automatizadas de seguridad.

Captura 33: Actualización del Jenkinsfile para preparar la integración de OWASP ZAP.



The screenshot shows a GitHub repository named 'EV3-Ciberseguridad_Final' by user 'CatrinaCorral'. The 'Jenkinsfile' is selected in the file explorer on the left. The main area displays the Jenkinsfile code, which is a YAML pipeline script. The script defines a pipeline with an agent 'any', an environment with variables 'APP_URL' and 'REPORT_DIR', and two stages: 'Build' and 'Test - Unit'. The 'Build' stage includes steps to create a Docker network and build the image. The 'Test - Unit' stage includes steps to set up a Python virtual environment and install dependencies.

```
1 pipeline {
2   agent any
3
4   environment {
5     APP_URL = 'http://ev3-app:5000'
6     REPORT_DIR = 'zap-reports'
7   }
8
9   stages {
10
11     stage('Build') {
12       steps {
13         sh '''
14           docker network create devsecops-network || true
15           docker build -t ev3-ciberseguridad_final:${BUILD_NUMBER} .
16         '''
17       }
18     }
19
20     stage('Test - Unit') {
21       steps {
22         sh '''
23           python3 -m venv venv || true
24           . venv/bin/activate || true
25
26           pip install --upgrade pip || true
27           pip install -r requirements.txt || true
28           pip install pytest flask || true
```

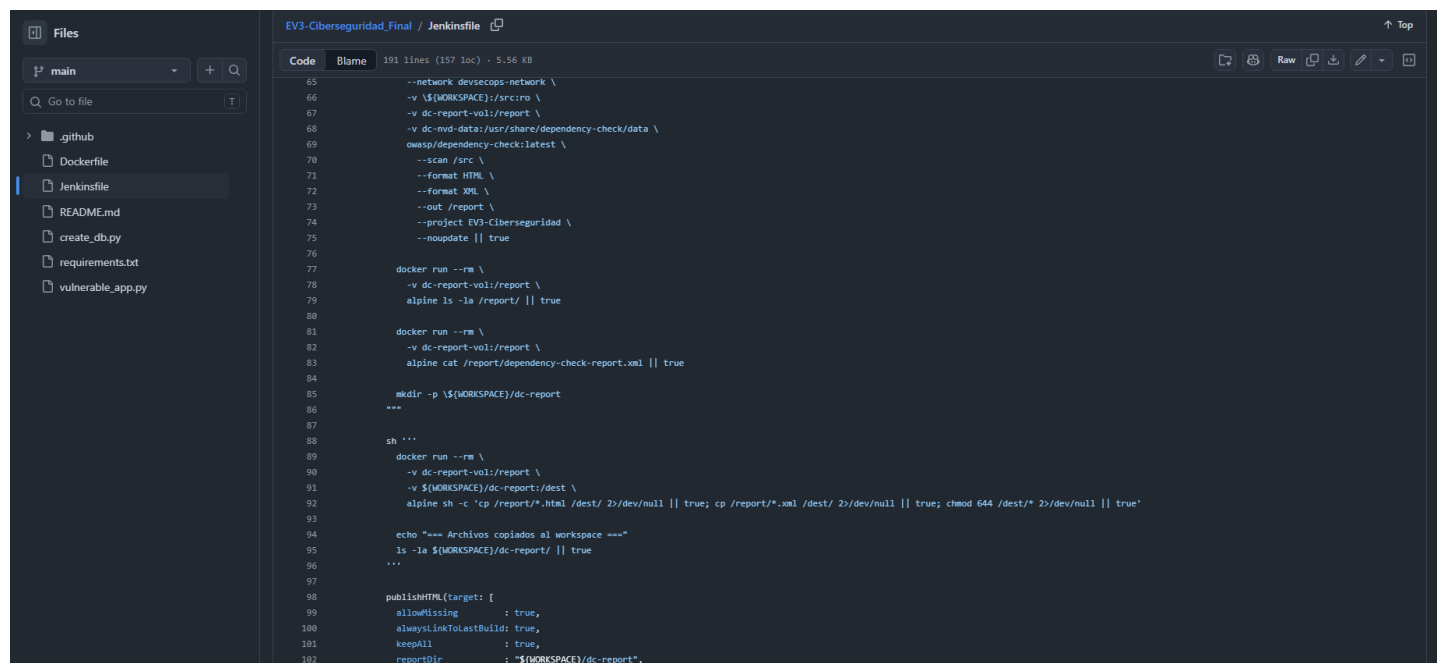
Descripción: Se visualiza el archivo **Jenkinsfile** actualizado en GitHub, incorporando variables de entorno como **APP_URL** y **REPORT_DIR**, además de etapas para construir la imagen Docker de la aplicación y preparar su ejecución dentro del pipeline. Esta configuración deja el entorno listo para que OWASP ZAP pueda realizar pruebas automatizadas sobre la aplicación desplegada.

Captura 34: Despliegue de la aplicación en entorno de staging para pruebas de seguridad.

```
29
30     pytest --junitxml=results.xml || true
31     ...
32     junit allowEmptyResults: true, testResults: 'results.xml'
33 }
34 }
35
36 stage('Deploy - Staging') {
37     steps {
38         sh '''
39             docker rm -f ev3-app || true
40
41             docker run -d \
42                 --name ev3-app \
43                 --network devsecops-network \
44                 -p 5000:5000 \
45                 ev3-ciberseguridad_final:${BUILD_NUMBER}
46
47             sleep 5
48
49             docker run --rm \
50                 --network devsecops-network \
51                 curlimages/curl:latest \
52                 -sf http://ev3-app:5000 || echo "App no responde"
53         '''
54     }
55 }
56
57 stage('Security Test - SCA Dependencies') {
58     steps {
59         sh '''
60             docker volume rm dc-report-vol 2>/dev/null || true
61             docker volume create dc-report-vol
62             docker volume create dc-nvd-data || true
63
64             docker run --rm \
65                 --network devsecops-network \
```

Descripción: Se visualiza el **Jenkinsfile** configurando la etapa **Deploy - Staging**, donde la aplicación se ejecuta en un contenedor Docker dentro de la red **devsecops-network**, exponiendo el puerto **5000** y validando su disponibilidad mediante **curl**. Esta configuración permite que la aplicación quede levantada y accesible para que posteriormente **OWASP ZAP** pueda realizar el escaneo automatizado.

Captura 35: Integración de OWASP Dependency-Check en el Jenkinsfile.



```
65 --network devsecops-network \
66 -v ${WORKSPACE}/src:ro \
67 -v dc-report-vol:/report \
68 -v dc-rvd-data:/usr/share/dependency-check/data \
69 owasp/dependency-check:latest \
70 --scan /src \
71 --format HTML \
72 --format XML \
73 --out /report \
74 --project EV3-Ciberseguridad \
75 --noupdate || true
76
77 docker run --rm \
78 -v dc-report-vol:/report \
79 alpine ls -la /report/ || true
80
81 docker run --rm \
82 -v dc-report-vol:/report \
83 alpine cat /report/dependency-check-report.xml || true
84
85 mkdir -p ${WORKSPACE}/dc-report
86 ""
87
88 sh '''
89 docker run --rm \
90 -v dc-report-vol:/report \
91 -v ${WORKSPACE}/dc-report:/dest \
92 alpine sh -c 'cp /report/*.html /dest/ 2>/dev/null || true; cp /report/*.xml /dest/ 2>/dev/null || true; chmod 644 /dest/* 2>/dev/null || true'
93
94 echo "" Archivos copiados al workspace ""
95 ls -la ${WORKSPACE}/dc-report/ || true
96 '''
97
98 publishHTML(target: [
99   allowMissing      : true,
100   alwaysLinkToLastBuild: true,
101   keepAll           : true,
102   reportDir          : "${WORKSPACE}/dc-report",
```

Descripción: Se visualiza el **Jenkinsfile** con la configuración de una etapa de análisis de dependencias utilizando la imagen **owasp/dependency-check:latest**. Esta etapa ejecuta un escaneo del proyecto, genera reportes en formato **HTML** y **XML**, y copia los resultados al workspace para su posterior revisión. Esta evidencia demuestra la incorporación de una herramienta automatizada de seguridad dentro del pipeline.

Captura 36: Integración de OWASP ZAP en el Jenkinsfile.

```

103     reportFiles      : 'dependency-check-report.html',
104     reportName       : 'Dependency-Check Report'
105   })
106 }
107 }
108
109 stage('Security Test - DAST ZAP') {
110   steps {
111     sh """
112     docker volume rm zap-report-vol 2>/dev/null || true
113     docker volume create zap-report-vol
114
115     docker run --rm \
116       -v zap-report-vol:/zap/wrk \
117       alpine sh -c 'chmod 777 /zap/wrk'
118
119     echo "=== Probando acceso a la aplicación ==="
120     docker run --rm \
121       --network devsecops-network \
122       curlimages/curl:latest \
123       -sf http://ev3-app:5000 || echo "App no responde"
124
125     echo "=== Ejecutando OWASP ZAP Baseline Scan ==="
126     docker run --rm \
127       -u root \
128       --network devsecops-network \
129       -v zap-report-vol:/zap/wrk/rw \
130       ghcr.io/zaproxy/zaproxy:stable \
131         zap-baseline.py \
132         -t http://ev3-app:5000 \
133         -r zap_report.html \
134         -j zap_report.json \
135         --auto || true
136
137     echo "=== Contenido volumen ZAP ==="
138     docker run --rm \
139       -v zap-report-vol:/zap/wrk \

```

Descripción: Se visualiza el **Jenkinsfile** con la etapa **“Security Test - DAST ZAP”**, donde se configura la ejecución de **OWASP ZAP Baseline Scan** mediante Docker. En esta etapa se valida el acceso a la aplicación desplegada y luego se ejecuta ZAP contra la URL **http://ev3-app:5000**, generando reportes en formato **HTML** y **JSON**.

Captura 37: Publicación del reporte OWASP ZAP en Jenkins.

```

140 alpine ls -la /zap/wrk || true
141
142 rm -rf ${WORKSPACE}/zap-reports
143 mkdir -p ${WORKSPACE}/zap-reports
144
145 docker rm -f zap-copy 2>/dev/null || true
146
147 docker create --name zap-copy \
148     -v zap-report-vol:/report \
149     alpine
150
151 docker cp zap-copy:/report/. ${WORKSPACE}/zap-reports/ || true
152 docker rm zap-copy || true
153
154 chmod 644 ${WORKSPACE}/zap-reports/* 2>/dev/null || true
155
156 echo "=== Contenido zap-reports en workspace ==="
157 ls -la ${WORKSPACE}/zap-reports/ || true
158 ***
159
160 publishHTML(target: [
161     allowMissing : true,
162     alwaysLinkToLastBuild: true,
163     keepall : true,
164     reportDir : "${WORKSPACE}/zap-reports",
165     reportFiles : 'zap_report.html',
166     reportName : 'OWASP ZAP Report'
167 ])
168
169 sh "find ${WORKSPACE} -name '*.html' -o -name '*.xml' -o -name '*.json" 2>/dev/null | head -30"
170 }
171 }
172 }
173
174 post {
175     always {
176         sh ***

```

Descripción: Se visualiza el **Jenkinsfile** configurado para copiar los reportes generados por OWASP ZAP al workspace del pipeline y publicarlos mediante **publishHTML**. El reporte queda definido como **“OWASP ZAP Report”**, permitiendo revisar los resultados del escaneo automatizado desde Jenkins en formato **HTML**.

Captura 38: Configuración del despliegue previo a las pruebas automatizadas de seguridad.

```

29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
code
Blame
191 lines (157 loc) · 5.56 KB
pytest --junitxml=results.xml || true
...
junit allomedyResults: true, testResults: 'results.xml'
}
}
stage('Deploy - Staging') {
  steps {
    sh '''
      docker rm -f ev3-app || true
      docker run -d \
        --name ev3-app \
        --network devsecops-network \
        -p 5000:5000 \
        ev3-ciberseguridad_final:${BUILD_NUMBER}
      sleep 5
      docker run --rm \
        --network devsecops-network \
        curlimages/curl:latest \
        -sf http://ev3-app:5000 || echo "App no responde"
    '''
  }
}
stage('Security Test - SCA Dependencies') {
  steps {
    sh '''
      docker volume rm dc-report-vol 2>/dev/null || true
      docker volume create dc-report-vol
      docker volume create dc-md-data || true
      docker run --rm \
        --network devsecops-network \

```

Descripción: Se visualiza el **Jenkinsfile** con la etapa **Deploy - Staging**, donde la aplicación se ejecuta en un contenedor Docker dentro de la red **devsecops-network** y se valida su disponibilidad mediante curl en la URL **http://ev3-app:5000**. Esta etapa permite asegurar que la aplicación esté activa antes de ejecutar las pruebas automatizadas de seguridad.

Captura 39: Archivado de reportes OWASP ZAP y evidencias del pipeline.

```

176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
code
sh '''
cp ${WORKSPACE}/dc-report/*.html ${WORKSPACE}/ 2>/dev/null || true
cp ${WORKSPACE}/dc-report/*.xml ${WORKSPACE}/ 2>/dev/null || true
cp ${WORKSPACE}/zap-reports/*.html ${WORKSPACE}/ 2>/dev/null || true
cp ${WORKSPACE}/zap-reports/*.json ${WORKSPACE}/ 2>/dev/null || true
'''
archiveArtifacts(
  artifacts : 'dependency-check-report.html,dependency-check-report.xml,zap_report.html,zap_report.json,results.xml',
  allowEmptyArchive : true
)
sh 'docker rm -f ev3-app || true'
}
}
}

```

Descripción: Se visualiza la sección final del **Jenkinsfile**, donde se copian y archivan los reportes generados durante el pipeline, incluyendo **zap_report.html**, **zap_report.json**, **dependency-check-report.html**, **dependency-check-report.xml** y **results.xml**. Además, se elimina el contenedor de la aplicación al finalizar la ejecución, manteniendo limpio el entorno de pruebas.

- Ejecuta las pruebas de seguridad sobre el código vulnerable y documenta los resultados, identificando claramente las

vulnerabilidades encontradas y las acciones tomadas para mitigarlas. (Pegar Pantallazos)

Captura 40: Ejecución de prueba automatizada con OWASP ZAP en Jenkins.

The screenshot displays the Jenkins interface for build #10 of the 'EV3-Pipeline_Final'. The build is in progress, and the 'Security Test - DAST ZAP' stage is highlighted. The stage is a 'Shell Script' task that runs a series of commands to set up Docker volumes, run a Docker container with OWASP ZAP, and execute a baseline scan on the application at <http://ev3-app:5000>. The output shows the scan results, including a 'PASS' status and a list of vulnerable JS libraries.

```
0 + docker volume rm zap-report-vol
1 zap-report-vol
2 + docker volume create zap-report-vol
3 zap-report-vol
4 + docker run --rm -v zap-report-vol:/zap/wrk alpine sh -c chmod 777 /zap/wrk
5 + echo === Probando acceso a la aplicación ===
6 === Probando acceso a la aplicación ===
7 + docker run --rm --network devsecops-network curlimages/curl:latest -sf http://ev3-app:5000
8 Welcome to the Task Manager Application!+ echo === Ejecutando OWASP ZAP Baseline Scan ===
9 === Ejecutando OWASP ZAP Baseline Scan ===
10 + docker run --rm -u root --network devsecops-network -v zap-report-vol:/zap/wrk:rw ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://ev3-app:5000 -r zap_report.html -j zap_report.json --auto
11 Using the Automation Framework
12 Total of 3 URLs
13 PASS: Vulnerable JS Library (Powered by Retire.js) [10003]
```

Descripción: Se visualiza el build #10 del pipeline “EV3-Pipeline_Final”, donde se ejecuta correctamente la etapa **Security Test - DAST ZAP**. En la salida se observa que Jenkins valida el acceso a la aplicación en <http://ev3-app:5000> y luego ejecuta **OWASP ZAP Baseline Scan** mediante Docker, generando reportes como **zap_report.html** y **zap_report.json**.

Captura 41: Reporte general de escaneo OWASP ZAP.

ZAP Scanning Report

Site: <http://ev3-app:5000>

Generated on Thu, 2 Jul 2026 04:52:16

ZAP Version: 2.17.0

ZAP by [Checkmarx](#)

Summary of Alerts

Risk Level	Number of Alerts
High	0
Medium	2
Low	6
Informational	1
False Positives:	0

Summary of Sequences

For each step: result (Pass/Fail) - risk (of highest alert(s) for the step, if any).

Alerts

Name	Risk Level	Number of Instances
Content Security Policy (CSP) Header Not Set	Medium	2
Missing Anti-clickjacking Header	Medium	1
Cross-Origin-Embedder-Policy Header Missing or Invalid	Low	1
Cross-Origin-Opener-Policy Header Missing or Invalid	Low	1
Cross-Origin-Resource-Policy Header Missing or Invalid	Low	1

Descripción: Se visualiza el reporte generado por **OWASP ZAP** sobre la aplicación desplegada en **<http://ev3-app:5000>**. El resumen muestra **0 alertas altas, 2 alertas medias, 6 alertas bajas, 1 informativa y 0 falsos positivos**. Entre las vulnerabilidades identificadas se observan cabeceras de seguridad ausentes o mal configuradas, como **Content Security Policy (CSP) Header Not Set**, **Missing Anti-clickjacking Header** y políticas Cross-Origin faltantes.

Captura 42: Detalle de alerta OWASP ZAP - Content Security Policy no configurada.

Permissions Policy Header Not Set	Low	3
Server Leaks Version Information via "Server" HTTP Response Header Field	Low	3
X-Content-Type-Options Header Missing	Low	1
Storable and Cacheable Content	Informational	3

Alert Detail

Medium	Content Security Policy (CSP) Header Not Set
Description	Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks. These attacks are used for everything from data theft to site defacement or distribution of malware. CSP provides a set of standard HTTP headers that allow website owners to declare approved sources of content that browsers should be allowed to load on that page — covered types are JavaScript, CSS, HTML frames, fonts, images and embeddable objects such as Java applets, ActiveX, audio and video files.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://ev3-app:5000/robots.txt
Node Name	http://ev3-app:5000/robots.txt
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
Instances	2
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Content-Security-Policy header. https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html https://www.w3.org/TR/CSP/ https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp
Reference	https://w3c.github.io/webappsec-csp/ https://web.dev/articles/csp

Descripción: Se visualiza el detalle de la alerta **Content Security Policy (CSP) Header Not Set**, clasificada con riesgo **Medium** y detectada en **2 instancias** dentro de la aplicación. OWASP ZAP indica que la ausencia de esta cabecera puede aumentar el riesgo frente a ataques como **XSS** o inyección de contenido.

Captura 43: Detalle de alertas OWASP ZAP - Anti-clickjacking y Cross-Origin-Embedder-Policy.

CWE Id	https://web.dev/articles/csp
WASC Id	https://caniuse.com/#feat=contentsecuritypolicy
Plugin Id	https://content-security-policy.com/693
	Medium
Description	Missing Anti-clickjacking Header The response does not protect against 'ClickJacking' attacks. It should include either Content-Security-Policy with 'frame-ancestors' directive or X-Frame-Options.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	x-frame-options
Attack	
Evidence	
Other Info	
Instances	1 Modern Web browsers support the Content-Security-Policy and X-Frame-Options HTTP headers. Ensure one of them is set on all web pages returned by your site/app.
Solution	If you expect the page to be framed only by pages on your server (e.g. it's part of a FRAMESET) then you'll want to use SAMEORIGIN, otherwise if you never expect the page to be framed, you should use DENY. Alternatively consider implementing Content Security Policy's "frame-ancestors" directive.
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options
CWE Id	1021
WASC Id	15
Plugin Id	10020
	Low
Description	Cross-Origin-Embedder-Policy Header Missing or Invalid Cross-Origin-Embedder-Policy header is a response header that prevents a document from loading any cross-origin resources that don't explicitly grant the document permission (using CORP or CORS).
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	Cross-Origin-Embedder-Policy
Attack	

Descripción: Se visualizan alertas detectadas por **OWASP ZAP** en la aplicación, destacando **Missing Anti-clickjacking Header** con riesgo **Medium** y **Cross-Origin-Embedder-Policy Header Missing or Invalid** con riesgo **Low**. La primera indica que la aplicación no protege adecuadamente contra ataques de **clickjacking**, mientras que la segunda evidencia una política Cross-Origin ausente o mal configurada.

Captura 44: Detalle de alertas OWASP ZAP - Cross-Origin-Embedder-Policy y Cross-Origin-Opener-Policy.

Attack	
Evidence	
Other Info	
Instances	1
	Ensure that the application/web server sets the Cross-Origin-Embedder-Policy header appropriately, and that it sets the Cross-Origin-Embedder-Policy header to 'require-corp' for documents.
Solution	If possible, ensure that the end user uses a standards-compliant and modern web browser that supports the Cross-Origin-Embedder-Policy header (https://caniuse.com/mdn-http_headers_cross-origin-embedder-policy).
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Embedder-Policy
CWE Id	693
WASC Id	14
Plugin Id	90004
Low	Cross-Origin-Opener-Policy Header Missing or Invalid
Description	Cross-Origin-Opener-Policy header is a response header that allows a site to control if others included documents share the same browsing context. Sharing the same browsing context with untrusted documents might lead to data leak.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	Cross-Origin-Opener-Policy
Attack	
Evidence	
Other Info	
Instances	1
	Ensure that the application/web server sets the Cross-Origin-Opener-Policy header appropriately, and that it sets the Cross-Origin-Opener-Policy header to 'same-origin' for documents.
Solution	'same-origin-allow-popups' is considered as less secured and should be avoided.
	If possible, ensure that the end user uses a standards-compliant and modern web browser that supports the Cross-Origin-Opener-Policy header (https://caniuse.com/mdn-http_headers_cross-origin-opener-policy).
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Opener-Policy
CWE Id	693
WASC Id	14

Descripción: Se visualizan alertas de riesgo **Low** detectadas por **OWASP ZAP**, relacionadas con cabeceras Cross-Origin ausentes o mal configuradas. En particular, se identifican problemas en **Cross-Origin-Embedder-Policy** y **Cross-Origin-Opener-Policy**, lo que puede aumentar el riesgo de exposición de información o interacción insegura con recursos de otros orígenes.

Captura 45: Detalle de alertas OWASP ZAP - Cross-Origin-Resource-Policy y Permissions Policy.

Plugin Id	90004
Low	Cross-Origin-Resource-Policy Header Missing or Invalid
Description	Cross-Origin-Resource-Policy header is an opt-in header designed to counter side-channels attacks like Spectre. Resource should be specifically set as shareable amongst different origins.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	Cross-Origin-Resource-Policy
Attack	
Evidence	
Other Info	
Instances	1 Ensure that the application/web server sets the Cross-Origin-Resource-Policy header appropriately, and that it sets the Cross-Origin-Resource-Policy header to 'same-origin' for all web pages.
Solution	'same-site' is considered as less secured and should be avoided. If resources must be shared, set the header to 'cross-origin'. If possible, ensure that the end user uses a standards-compliant and modern web browser that supports the Cross-Origin-Resource-Policy header (https://caniuse.com/mdn-http_headers_cross-origin-resource-policy).
Reference	https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Cross-Origin-Embedder-Policy
CWE Id	693
WASC Id	14
Plugin Id	90004
Low	Permissions Policy Header Not Set
Description	Permissions Policy Header is an added layer of security that helps to restrict from unauthorized access or usage of browser/client features by web resources. This policy ensures the user privacy by limiting or specifying the features of the browsers can be used by the web resources. Permissions Policy provides a set of standard HTTP headers that allow website owners to limit which features of browsers can be used by the page such as camera, microphone, location, full screen etc.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	
Attack	

Descripción: Se visualizan alertas de riesgo **Low** detectadas por **OWASP ZAP**, relacionadas con cabeceras de seguridad ausentes o mal configuradas. Se identifican las alertas **Cross-Origin-Resource-Policy Header Missing or Invalid** y **Permissions Policy Header Not Set**, las cuales pueden afectar el control de recursos compartidos entre orígenes y el uso de funciones del navegador.

Captura 46: Detalle de alertas OWASP ZAP - Permissions Policy y exposición de versión del servidor.

Attack	
Evidence	
Other Info	
URL	http://ev3-app:5000/robots.txt
Node Name	http://ev3-app:5000/robots.txt
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
URL	http://ev3-app:5000/sitemap.xml
Node Name	http://ev3-app:5000/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	
Other Info	
Instances	3
Solution	Ensure that your web server, application server, load balancer, etc. is configured to set the Permissions-Policy header. https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Permissions-Policy https://developer.chrome.com/blog/feature-policy/ https://scotthelme.co.uk/a-new-security-header-feature-policy/ https://w3c.github.io/webappsec-feature-policy/ https://www.smashingmagazine.com/2018/12/feature-policy/
Reference	
CWE Id	693
WASC Id	15
Plugin Id	10063
Low	Server Leaks Version Information via "Server" HTTP Response Header Field
Description	The web/application server is leaking version information via the "Server" HTTP response header. Access to such information may facilitate attackers identifying other vulnerabilities your web/application server is subject to.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000

Descripción: Se visualizan alertas detectadas por **OWASP ZAP** asociadas a cabeceras de seguridad. Primero, se muestra la alerta **Permissions Policy Header Not Set**, con **3 instancias**, indicando que la aplicación no limita funciones del navegador mediante una política de permisos. Además, se identifica la alerta **Server Leaks Version Information via "Server" HTTP Response Header Field**, la cual indica que el servidor expone información de versión en las respuestas HTTP.

Captura 47: Detalle de alertas OWASP ZAP - exposición de versión del servidor y X-Content-Type-Options ausente.

Node Name	http://ev3-app:5000
Method	GET
Parameter	
Attack	
Evidence	Werkzeug/3.1.8 Python/3.10.20
Other Info	
URL	http://ev3-app:5000/robots.txt
Node Name	http://ev3-app:5000/robots.txt
Method	GET
Parameter	
Attack	
Evidence	Werkzeug/3.1.8 Python/3.10.20
Other Info	
URL	http://ev3-app:5000/sitemap.xml
Node Name	http://ev3-app:5000/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	Werkzeug/3.1.8 Python/3.10.20
Other Info	
Instances	3
Solution	Ensure that your web server, application server, load balancer, etc. is configured to suppress the "Server" header or provide generic details. https://httpd.apache.org/docs/current/mod/core.html#servertokens https://learn.microsoft.com/en-us/previous-versions/msp-n-p/ff648552(v=pandp.10) https://www.troyhunt.com/shhh-dont-let-your-response-headers/
Reference	
CWE Id	497
WASC Id	13
Plugin Id	10036
Low	X-Content-Type-Options Header Missing
Description	The Anti-MIME-Sniffing header X-Content-Type-Options was not set to 'nosniff'. This allows older versions of Internet Explorer and Chrome to perform MIME-sniffing on the response body, potentially causing the response body to be interpreted and displayed as a content type other than the declared content type. Current (early 2014) and legacy versions of Firefox will use the declared content type (if one is set), rather than performing MIME-sniffing.
TIDT	http://ev3-app:5000

Descripción: Se visualiza el detalle de alertas detectadas por **OWASP ZAP**. La primera corresponde a **Server Leaks Version Information**, donde el servidor expone información técnica como **Werkzeug/3.1.8 Python/3.10.20** en la cabecera HTTP. También se observa la alerta **X-Content-Type-Options Header Missing**, indicando que falta una cabecera que ayuda a prevenir interpretaciones incorrectas del tipo de contenido por parte del navegador.

Captura 48: Detalle de alertas OWASP ZAP - X-Content-Type-Options y contenido cacheable.

URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	x-content-type-options
Attack	
Evidence	
Other Info	This issue still applies to error type pages (401, 403, 500, etc.) as those pages are often still affected by injection issues, in which case there is still concern for browsers sniffing pages away from their actual content type.
Instances	At "High" threshold this scan rule will not alert on client or server error responses. 1 Ensure that the application/web server sets the Content-Type header appropriately, and that it sets the X-Content-Type-Options header to 'nosniff' for all web pages.
Solution	If possible, ensure that the end user uses a standards-compliant and modern web browser that does not perform MIME-sniffing at all, or that can be directed by the web application/web server to not perform MIME-sniffing.
Reference	https://learn.microsoft.com/en-us/previous-versions/windows/internet-explorer/ie-developer/compatibility/gg622941(v=vs.85) https://owasp.org/www-community/Security_Headers
CWE Id	693
WASC Id	15
Plugin Id	10021
	Informational
	Storable and Cacheable Content
Description	The response contents are storable by caching components such as proxy servers, and may be retrieved directly from the cache, rather than from the origin server by the caching servers, in response to similar requests from other users. If the response data is sensitive, personal or user-specific, this may result in sensitive information being leaked. In some cases, this may even result in a user gaining complete control of the session of another user, depending on the configuration of the caching components in use in their environment. This is primarily an issue where "shared" caching servers such as "proxy" caches are configured on the local network. This configuration is typically found in corporate or educational environments, for instance.
URL	http://ev3-app:5000
Node Name	http://ev3-app:5000
Method	GET
Parameter	
Attack	
Evidence	
Other Info	In the absence of an explicitly specified caching lifetime directive in the response, a liberal lifetime heuristic of 1 year was assumed. This is permitted by rfc7234.
URL	http://ev3-app:5000/robots.txt

Descripción: Se visualizan alertas detectadas por **OWASP ZAP** relacionadas con cabeceras de seguridad y manejo de caché. La alerta **X-Content-Type-Options Header Missing** indica que no se encuentra configurada la cabecera **X-Content-Type-Options: nosniff**, lo que podría permitir que el navegador interprete incorrectamente el tipo de contenido. Además, se observa la alerta informativa **Storable and Cacheable Content**, asociada a contenido que podría ser almacenado en caché.

Captura 49: Detalle de alerta OWASP ZAP - Storable and Cacheable Content.

Other Info	In the absence of an explicitly specified caching lifetime directive in the response, a liberal lifetime heuristic of 1 year was assumed. This is permitted by RFC7234.
URL	http://ev3-app:5000/robots.txt
Node Name	http://ev3-app:5000/robots.txt
Method	GET
Parameter	
Attack	
Evidence	
Other Info	In the absence of an explicitly specified caching lifetime directive in the response, a liberal lifetime heuristic of 1 year was assumed. This is permitted by RFC7234.
URL	http://ev3-app:5000/sitemap.xml
Node Name	http://ev3-app:5000/sitemap.xml
Method	GET
Parameter	
Attack	
Evidence	
Other Info	In the absence of an explicitly specified caching lifetime directive in the response, a liberal lifetime heuristic of 1 year was assumed. This is permitted by RFC7234.
Instances	3 Validate that the response does not contain sensitive, personal or user-specific information. If it does, consider the use of the following HTTP response headers, to limit, or prevent the content being stored and retrieved from the cache by another user: Cache-Control: no-cache, no-store, must-revalidate, private Pragma: no-cache Expires: 0 This configuration directs both HTTP 1.0 and HTTP 1.1 compliant caching servers to not store the response, and to not retrieve the response (without validation) from the cache, in response to a similar request. https://datatracker.ietf.org/doc/html/rfc7234 https://datatracker.ietf.org/doc/html/rfc7231 https://www.w3.org/Protocols/rfc2616/rfc2616-sec13.html
Reference	
CWE Id	524
WASC Id	13
Plugin Id	10049

Descripción: Se visualiza el detalle de la alerta informativa **Storable and Cacheable Content**, detectada por **OWASP ZAP** en rutas como **robots.txt** y **sitemap.xml**. El reporte indica que, al no existir una directiva explícita de caché, el contenido podría ser almacenado por componentes intermedios.

Captura 50: Cierre del reporte OWASP ZAP y referencias técnicas del hallazgo.

Reference	https://datatracker.ietf.org/doc/html/rfc7234 https://datatracker.ietf.org/doc/html/rfc7231 https://www.w3.org/Protocols/rfc2616/rfc2616-sec13.html
CWE Id	524
WASC Id	13
Plugin Id	10049

Sequence Details

With the associated active scan results.

Descripción: Se visualiza la sección final del reporte de **OWASP ZAP**, donde se muestran referencias técnicas asociadas a la alerta **Storable and Cacheable Content**, incluyendo su identificación **CWE-524** y el **Plugin ID 10049**. Esta evidencia permite respaldar documentalmente el hallazgo detectado durante el escaneo automatizado.

Captura 51: Aplicación de mitigaciones en vulnerable_app.py para hallazgos de OWASP ZAP.

The screenshot shows a GitHub interface for a repository named 'EV3-Ciberseguridad_Final'. A pull request titled 'Update vulnerable_app.py' by user 'CatrinaCorral' is displayed. The pull request is 138491 lines old and was updated 2 minutes ago. The file 'vulnerable_app.py' is selected, showing 219 lines of code. The code is a Python Flask application with the following content:

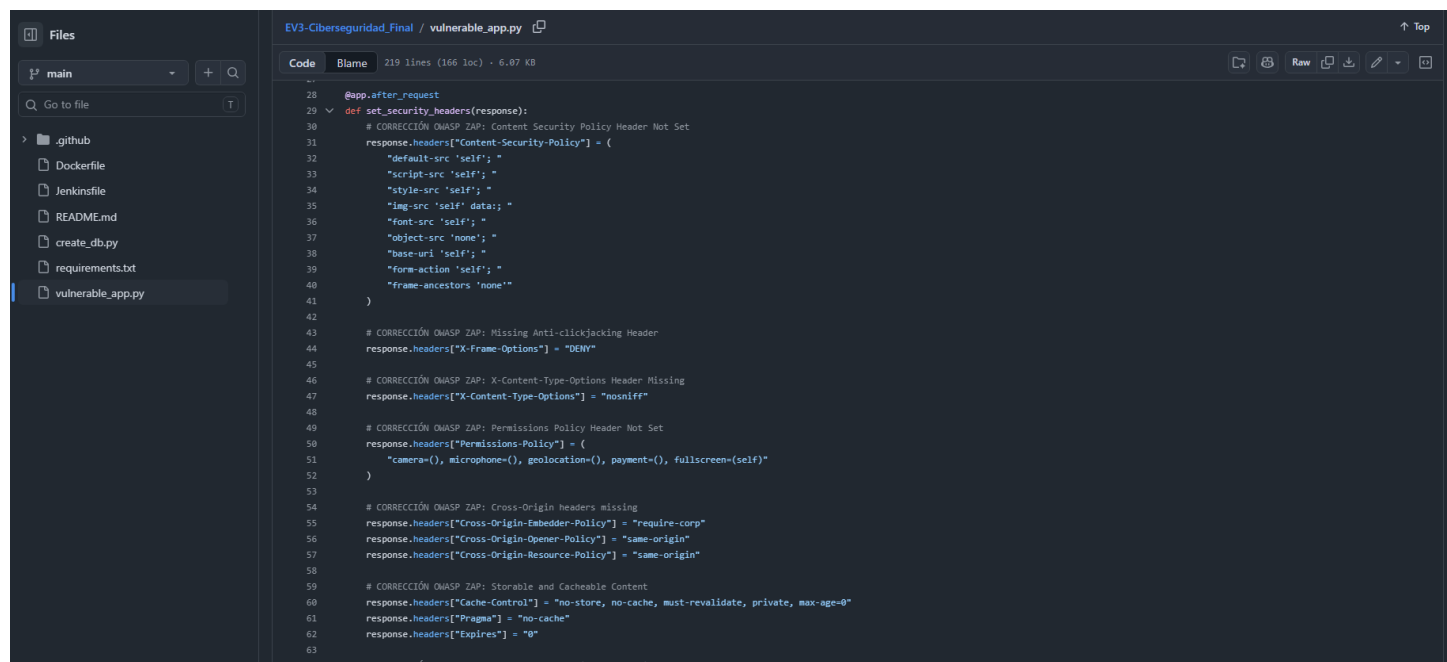
```

1 from flask import Flask, request, render_template_string, session, redirect, url_for, Response
2 import sqlite3
3 import os
4 import hashlib
5
6 try:
7     from werkzeug.serving import WSGIRequestHandler
8     WSGIRequestHandler.server_version = "WebServer"
9     WSGIRequestHandler.sys_version = ""
10 except Exception:
11     pass
12
13
14 app = Flask(__name__)
15 app.secret_key = os.environ.get("SECRET_KEY", os.urandom(24))
16
17
18 def get_db_connection():
19     conn = sqlite3.connect('database.db')
20     conn.row_factory = sqlite3.Row
21     return conn
22
23
24 def hash_password(password):
25     return hashlib.sha256(password.encode()).hexdigest()
26
27
28 @app.after_request

```

Descripción: Se visualiza la actualización del archivo **vulnerable_app.py**, donde se agregan configuraciones de seguridad para mitigar hallazgos identificados por OWASP ZAP. Entre ellas, se elimina la exposición de información del servidor modificando los valores de **Werkzeug**, se configura una **SECRET_KEY** desde variable de entorno y se prepara el uso de respuestas HTTP seguras mediante **after_request**.

Captura 52: Corrección de cabeceras HTTP de seguridad detectadas por OWASP ZAP.



```
28 @app.after_request
29 def set_security_headers(response):
30     # CORRECCIÓN OWASP ZAP: Content Security Policy Header Not Set
31     response.headers["Content-Security-Policy"] = (
32         "default-src 'self'; "
33         "script-src 'self'; "
34         "style-src 'self'; "
35         "img-src 'self' data:; "
36         "font-src 'self'; "
37         "object-src 'none'; "
38         "base-uri 'self'; "
39         "form-action 'self'; "
40         "frame-ancestors 'none'"
41     )
42
43     # CORRECCIÓN OWASP ZAP: Missing Anti-clickjacking Header
44     response.headers["X-Frame-Options"] = "DENY"
45
46     # CORRECCIÓN OWASP ZAP: X-Content-Type-Options Header Missing
47     response.headers["X-Content-Type-Options"] = "nosniff"
48
49     # CORRECCIÓN OWASP ZAP: Permissions Policy Header Not Set
50     response.headers["Permissions-Policy"] = (
51         "camera=(), microphone=(), geolocation=(), payment=(), fullscreen=(self)"
52     )
53
54     # CORRECCIÓN OWASP ZAP: Cross-Origin headers missing
55     response.headers["Cross-Origin-Embedder-Policy"] = "require-corp"
56     response.headers["Cross-Origin-Opener-Policy"] = "same-origin"
57     response.headers["Cross-Origin-Resource-Policy"] = "same-origin"
58
59     # CORRECCIÓN OWASP ZAP: Storable and Cacheable Content
60     response.headers["Cache-Control"] = "no-store, no-cache, must-revalidate, private, max-age=0"
61     response.headers["Pragma"] = "no-cache"
62     response.headers["Expires"] = "0"
```

Descripción: Se visualiza la función `set_security_headers()` en `vulnerable_app.py`, donde se agregan cabeceras HTTP para mitigar los hallazgos detectados por OWASP ZAP. Entre las correcciones aplicadas se incluyen **Content-Security-Policy**, **X-Frame-Options: DENY**, **X-Content-Type-Options: nosniff**, **Permissions-Policy**, cabeceras **Cross-Origin** y políticas de caché como **Cache-Control**, **Pragma** y **Expires**.

Vulnerabilidad identificada	Herramienta	Riesgo	Mitigación aplicada
CSP Header Not Set	OWASP ZAP	Medio	Se agregó cabecera Content-Security-Policy
Missing Anti-clickjacking Header	OWASP ZAP	Medio	Se agregó X-Frame-Options: DENY
X-Content-Type-Options Missing	OWASP ZAP	Bajo	Se agregó X-Content-Type-Options: nosniff
Cross-Origin headers ausentes	OWASP ZAP	Bajo	Se agregaron COEP, COOP y CORP
Contenido cacheable	OWASP ZAP	Informativo	Se agregaron Cache-Control, Pragma y Expires
SQL Injection	Bandit / CWE-89	Medio	Se reemplazó consulta insegura por consulta parametrizada
Flask debug=True	Bandit / CWE-94	Alto	Se cambió a debug=False

Monitorización del Entorno de Producción:

- Configura Grafana para la monitorización del entorno de producción. (Pegar Pantallazos)

Captura 53: Registro de configuración de Prometheus y Grafana en el repositorio.

```
danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        docker-compose-grafana.yml
        prometheus.yml

nothing added to commit but untracked files present (use "git add" to track)

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git add docker-compose-grafana.yml prometheus.yml

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git commit -m "Agrega configuracion de Prometheus y Grafana"
[main d2e61f0] Agrega configuracion de Prometheus y Grafana
 2 files changed, 37 insertions(+)
 create mode 100644 docker-compose-grafana.yml
 create mode 100644 prometheus.yml

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ ls
Jenkinsfile  create_db.py      prometheus.yml    vulnerable_app.py
README.md    docker-compose-grafana.yml  requirements.txt

danae@LAPTOP-NSV0ER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ |
```

Descripción: Se evidencia la creación y versionamiento de los archivos **docker-compose-grafana.yml** y **prometheus.yml**, los cuales serán utilizados para levantar el entorno de monitorización con **Grafana** y **Prometheus**. Además, se realiza el commit **“Agrega configuracion de Prometheus y Grafana”**, dejando trazabilidad de la configuración incorporada al proyecto.

Captura 54: Confirmación de subida de configuración de Grafana y Prometheus al repositorio.

```
danae@LAPTOP-NSVOER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git push origin main
Everything up-to-date

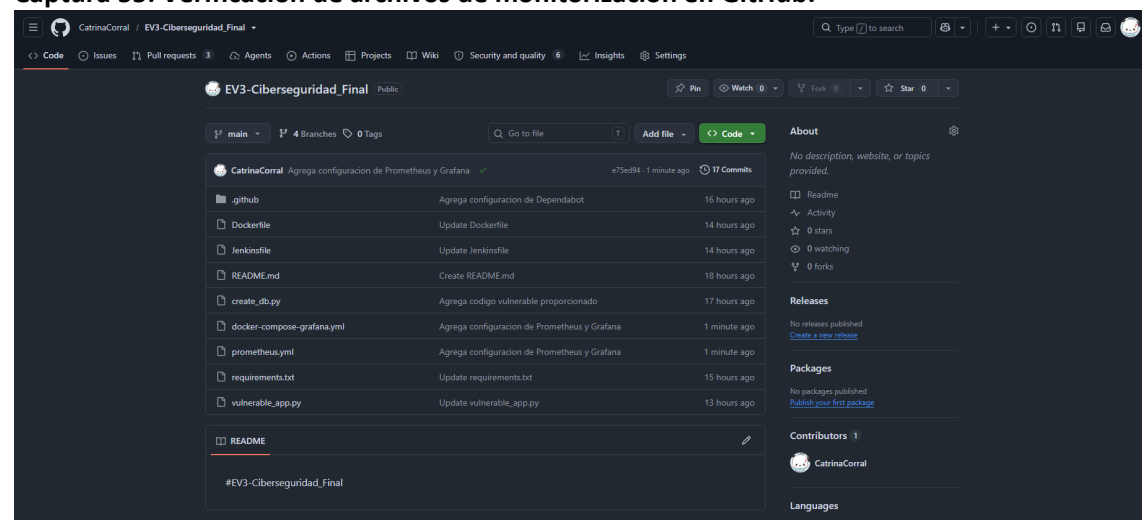
danae@LAPTOP-NSVOER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

danae@LAPTOP-NSVOER1Q MINGW64 ~/Downloads/EV3-Ciberseguridad_Final (main)
$
```

Descripción: Se evidencia la ejecución de `git push origin main`, confirmando que los archivos de configuración de **Prometheus** y **Grafana** fueron enviados correctamente al repositorio remoto. Además, el comando `git status` muestra que la rama **main** está actualizada con **origin/main** y que el árbol de trabajo se encuentra limpio, sin cambios pendientes.

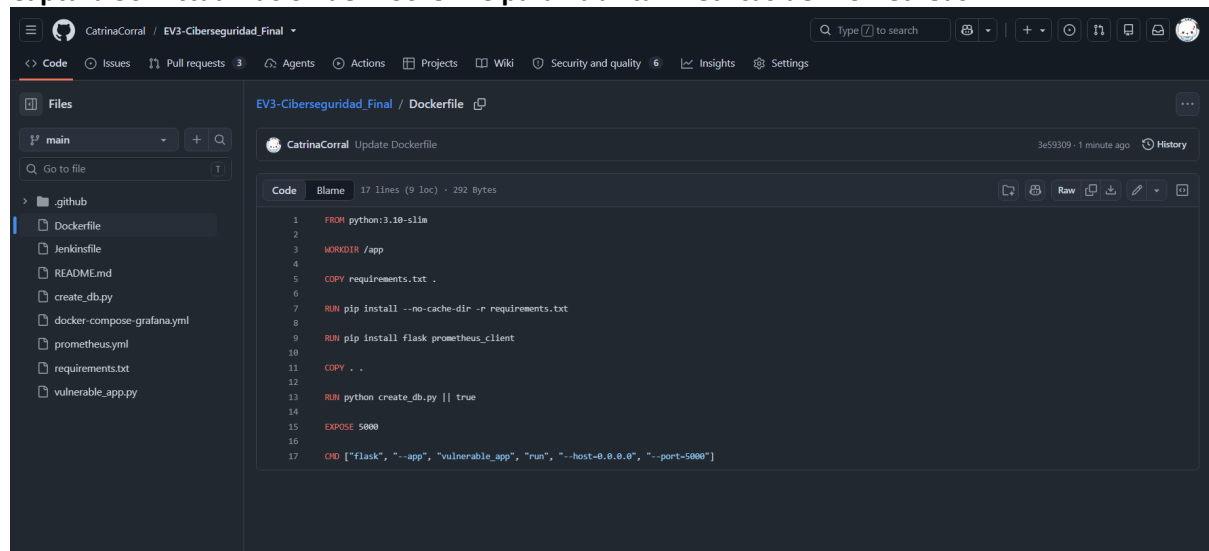
Captura 55: Verificación de archivos de monitorización en GitHub.



Descripción: Se visualiza el repositorio “**EV3-Ciberseguridad_Final**” en GitHub, donde se encuentran los archivos **docker-compose-grafana.yml** y

prometheus.yml ya versionados. Esto confirma que la configuración necesaria para levantar el entorno de monitorización con **Grafana** y **Prometheus** fue incorporada correctamente al proyecto.

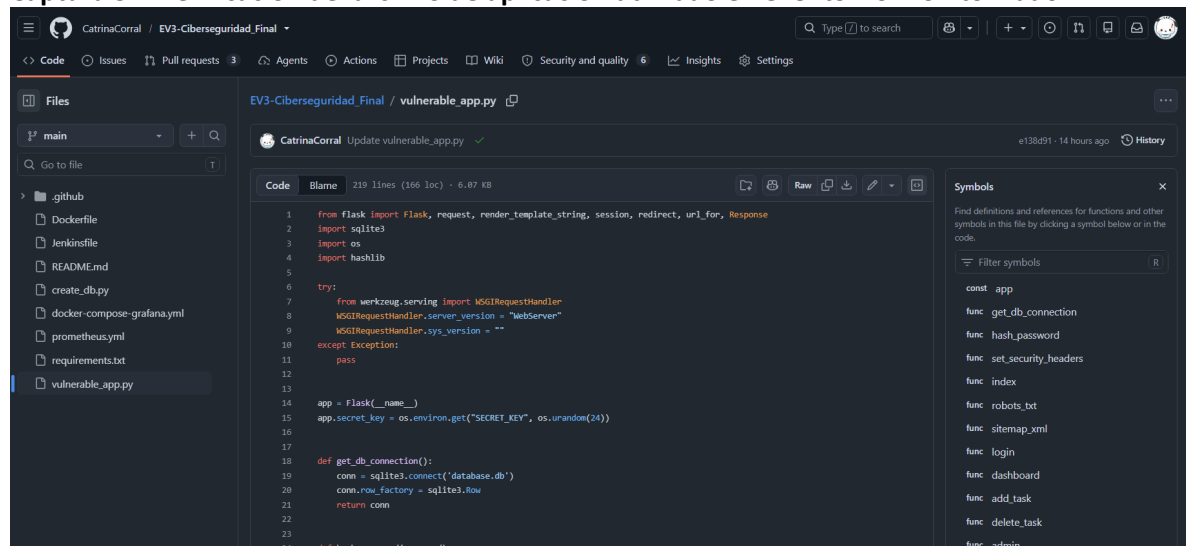
Captura 56: Actualización del Dockerfile para habilitar métricas de Prometheus.



```
1 FROM python:3.10-slim
2
3 WORKDIR /app
4
5 COPY requirements.txt .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 RUN pip install flask prometheus_client
10
11 COPY . .
12
13 RUN python create_db.py || true
14
15 EXPOSE 5000
16
17 CMD ["flask", "--app", "vulnerable_app", "run", "--host=0.0.0.0", "--port=5000"]
```

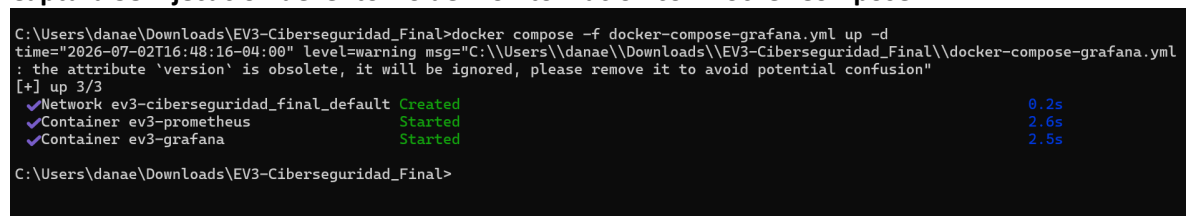
Descripción: Se visualiza la actualización del **Dockerfile**, donde se instala la librería **prometheus_client** junto con las dependencias de la aplicación Flask. Esta configuración permite preparar la aplicación para exponer métricas que posteriormente podrán ser recolectadas por **Prometheus** y visualizadas en **Grafana**.

Captura 57: Verificación del archivo de aplicación utilizado en el entorno monitorizado.

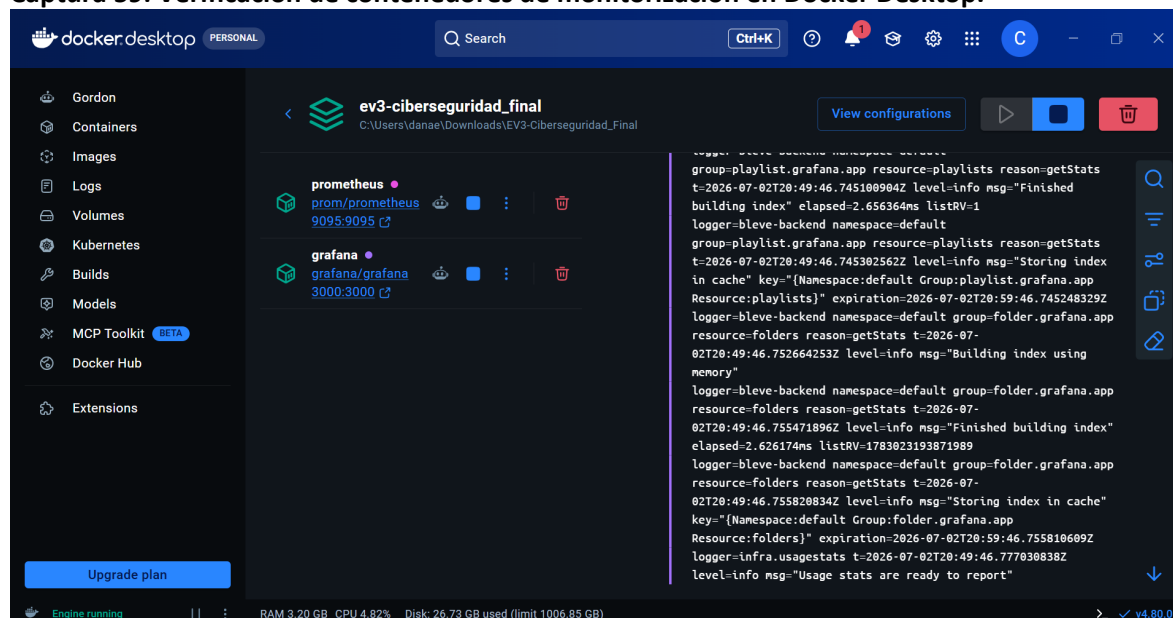


Descripción: Se visualiza el archivo **vulnerable_app.py** actualizado dentro del repositorio, junto con las funciones principales de la aplicación. Esta evidencia permite confirmar que el código de la aplicación se encuentra versionado y disponible para ser ejecutado en el entorno configurado con Docker, Prometheus y Grafana.

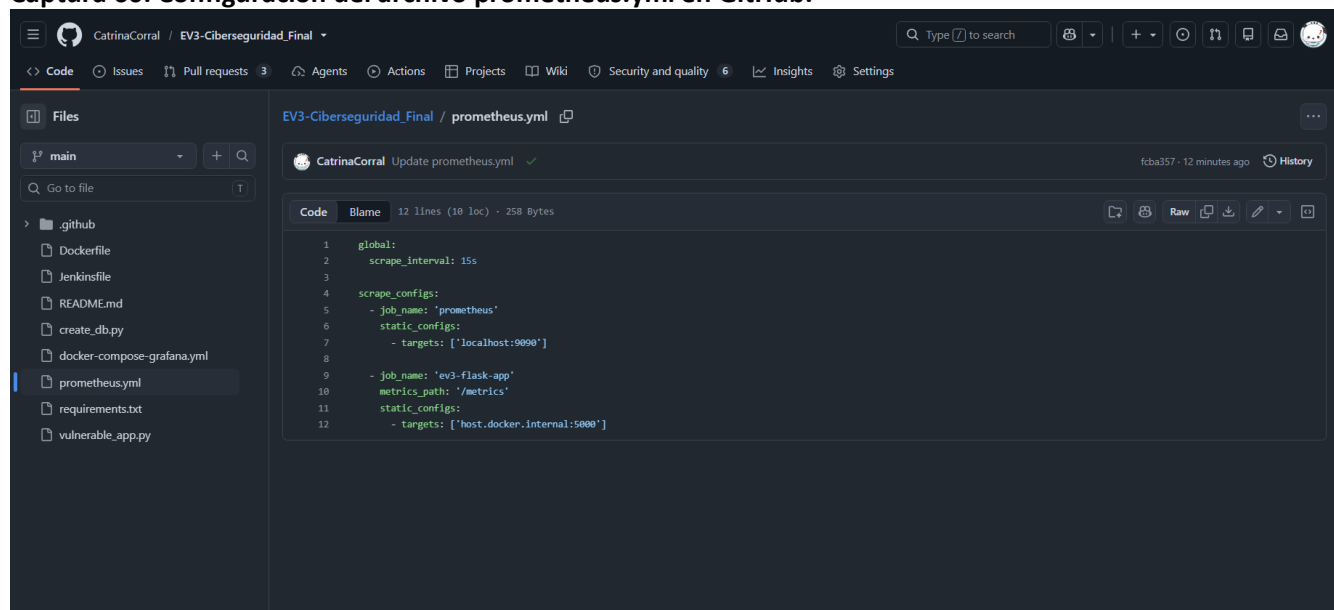
Captura 58: Ejecución del entorno de monitorización con Docker Compose.



Descripción: Se ejecuta el comando **docker compose -f docker-compose-grafana.yml up -d**, creando la red del proyecto e iniciando correctamente los contenedores **ev3-prometheus** y **ev3-grafana**. Esto evidencia que el entorno de monitorización fue desplegado mediante Docker Compose y quedó operativo para recolectar y visualizar métricas del sistema.

Captura 59: Verificación de contenedores de monitorización en Docker Desktop.

Descripción: Se visualiza Docker Desktop con el proyecto **ev3-ciberseguridad_final** ejecutándose correctamente. En esta captura se observa que los contenedores **prometheus** y **grafana** se encuentran activos, junto con sus respectivos puertos publicados: **Prometheus** mediante el puerto 9095:9095 y **Grafana** mediante el puerto 3000:3000. Además, en el panel derecho se muestran los logs generados por Grafana, lo que confirma que el servicio inició correctamente y que se encuentra procesando información interna del sistema. Esta evidencia permite comprobar que el entorno de monitorización fue levantado exitosamente mediante Docker y que las herramientas necesarias para observar métricas y visualizar datos quedaron disponibles para su uso dentro del proyecto.

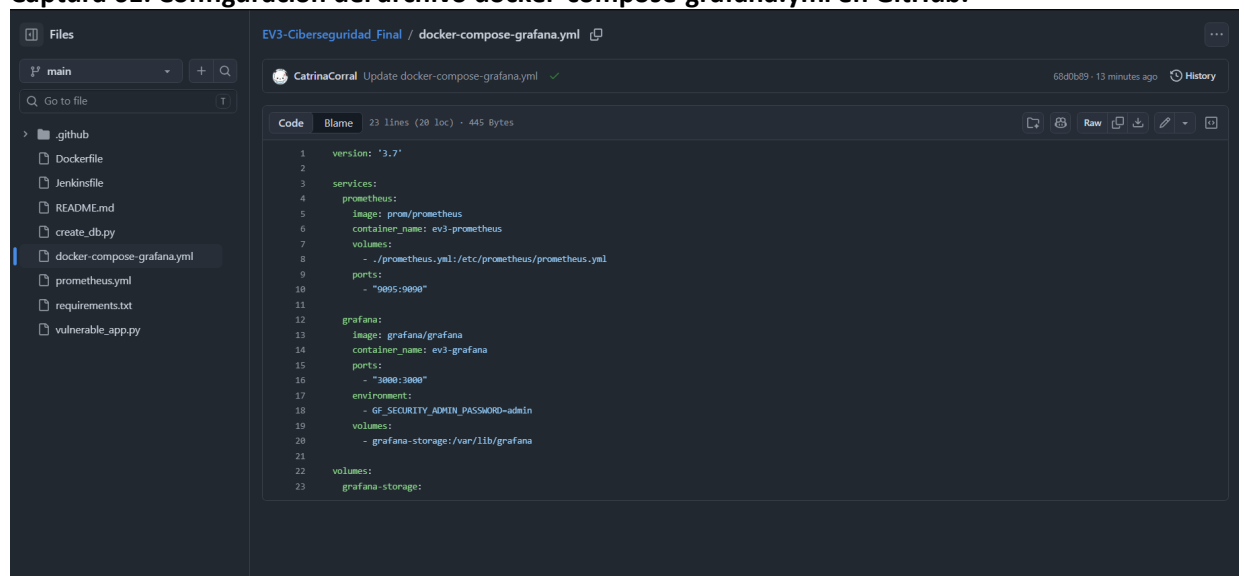
Captura 60: Configuración del archivo prometheus.yml en GitHub.

The screenshot shows the GitHub interface for the repository **EV3-Ciberseguridad_Final** by user **CatrinaCorral**. The file **prometheus.yml** is selected, showing a commit titled "Update prometheus.yml" by **fcba357** from 12 minutes ago. The file content is as follows:

```
1  global:
2    scrape_interval: 15s
3
4  scrape_configs:
5    - job_name: 'prometheus'
6      static_configs:
7        - targets: ['localhost:9090']
8
9    - job_name: 'ev3-flask-app'
10      metrics_path: '/metrics'
11      static_configs:
12        - targets: ['host.docker.internal:5000']
```

Descripción: Se visualiza el archivo **prometheus.yml** dentro del repositorio **EV3-Ciberseguridad_Final** en GitHub. En esta captura se evidencia la configuración principal de **Prometheus** para la recolección automática de métricas del entorno. El archivo define un intervalo global de scraping de **15 segundos**, lo que indica que Prometheus consultará periódicamente los objetivos configurados para obtener información actualizada. Además, se observan dos bloques de configuración: uno correspondiente al propio servicio **prometheus**, apuntando al target **localhost:9090**, y otro correspondiente a la aplicación **ev3-flask-app**, utilizando la ruta **/metrics** y el target **host.docker.internal:5000**. Esto permite que Prometheus recopile métricas tanto de su propio servicio como de la aplicación Flask ejecutada en el entorno del proyecto. Esta evidencia confirma que la monitorización fue configurada correctamente y versionada en GitHub, formando parte del proceso de observabilidad solicitado para el pipeline y el entorno de producción/prueba.

Captura 61: Configuración del archivo docker-compose-grafana.yml en GitHub.

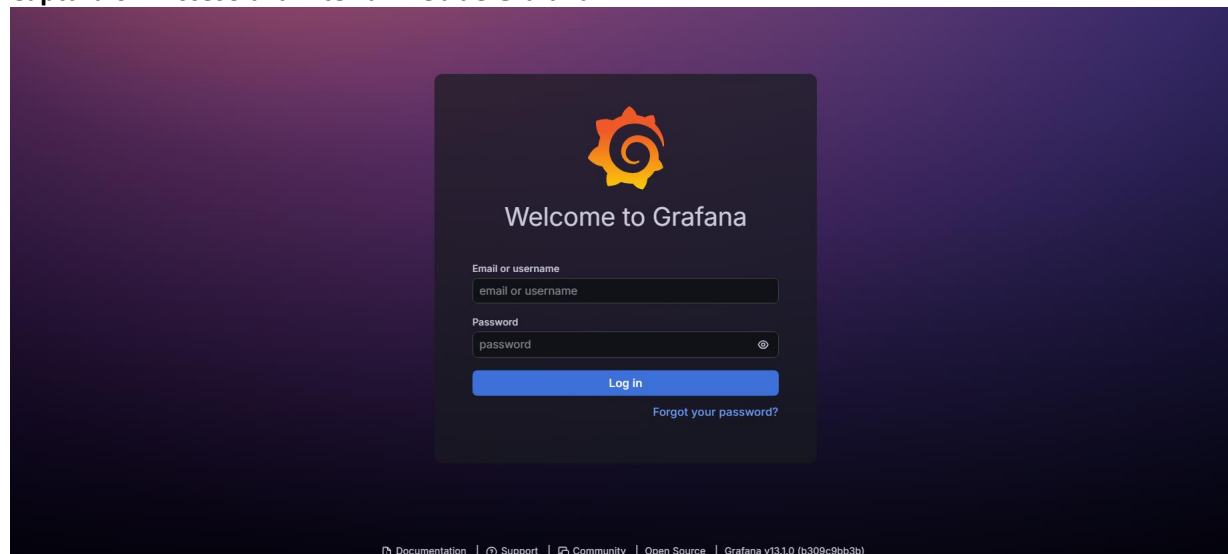


The screenshot shows a GitHub repository interface. On the left, a file explorer lists the repository's contents: .github, Dockerfile, Jenkinsfile, README.md, create_db.py, docker-compose-grafana.yml (selected), prometheus.yml, requirements.txt, and vulnerable_app.py. The main area displays the content of 'docker-compose-grafana.yml' with the following YAML configuration:

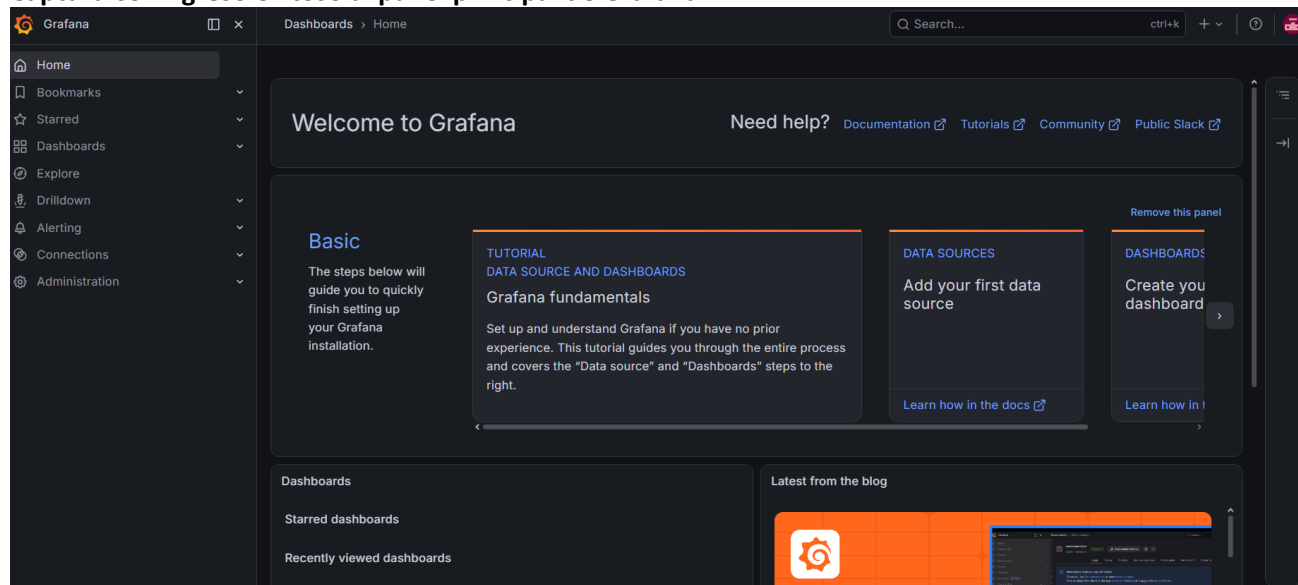
```
1 version: '3.7'
2
3 services:
4   prometheus:
5     image: prom/prometheus
6     container_name: ev3-prometheus
7     volumes:
8       - ./prometheus.yml:/etc/prometheus/prometheus.yml
9     ports:
10      - "9095:9090"
11
12   grafana:
13     image: grafana/grafana
14     container_name: ev3-grafana
15     ports:
16       - "3000:3000"
17     environment:
18       - GF_SECURITY_ADMIN_PASSWORD=admin
19     volumes:
20       - grafana-storage:/var/lib/grafana
21
22 volumes:
23   grafana-storage:
```

Descripción: Se muestra el archivo docker-compose-grafana.yml versionado en el repositorio **EV3-Ciberseguridad_Final**. En esta configuración se definen los servicios **Prometheus** y **Grafana**, indicando sus imágenes, nombres de contenedor y puertos de acceso. Prometheus utiliza la imagen prom/prometheus, se ejecuta como ev3-prometheus y publica el puerto 9095:9090. Grafana utiliza la imagen grafana/grafana, se ejecuta como ev3-grafana y publica el puerto 3000:3000. Además, se monta el archivo prometheus.yml para que Prometheus utilice la configuración de monitoreo definida en el proyecto. Esta captura evidencia que el entorno de monitorización fue preparado mediante Docker Compose.

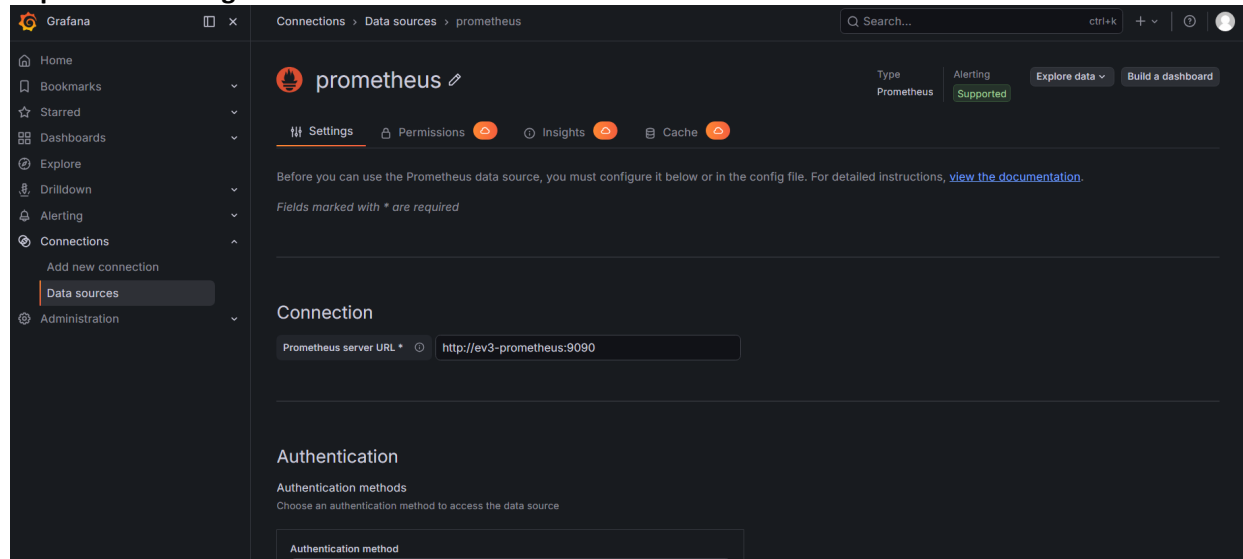
Captura 62: Acceso a la interfaz web de Grafana.



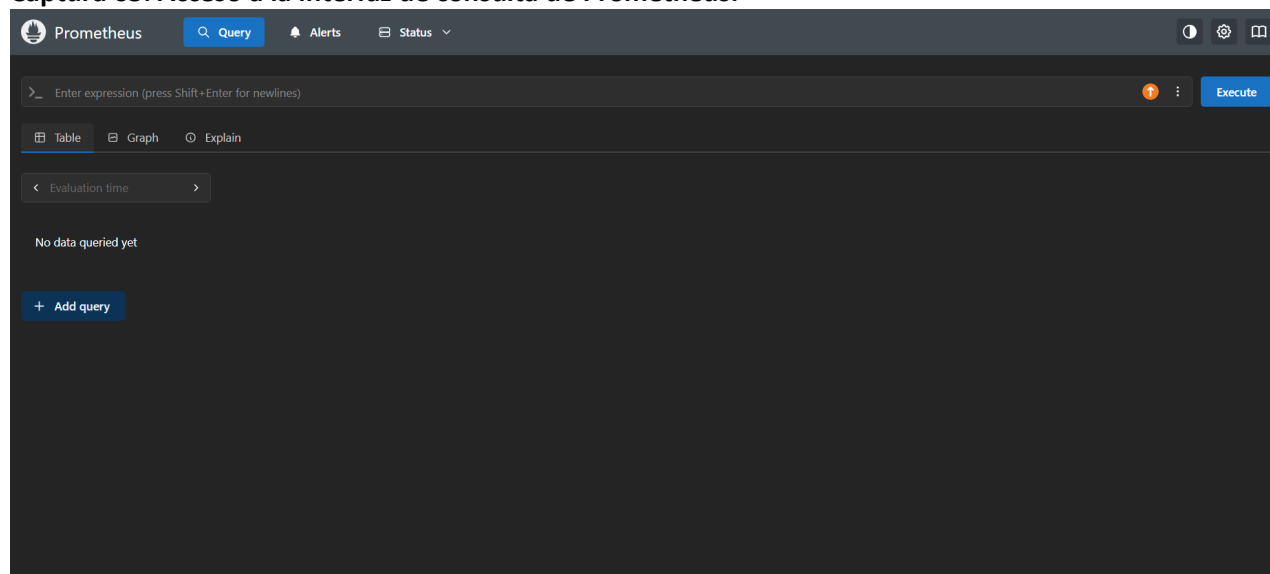
Descripción: Se muestra la pantalla de inicio de sesión de **Grafana**, lo que confirma que el servicio fue levantado correctamente y se encuentra disponible desde el navegador. Esta evidencia válida que el contenedor de Grafana está activo y que la plataforma de visualización ya puede ser utilizada para ingresar, configurar fuentes de datos y crear paneles de monitoreo asociados al proyecto.

Captura 63: Ingreso exitoso al panel principal de Grafana.

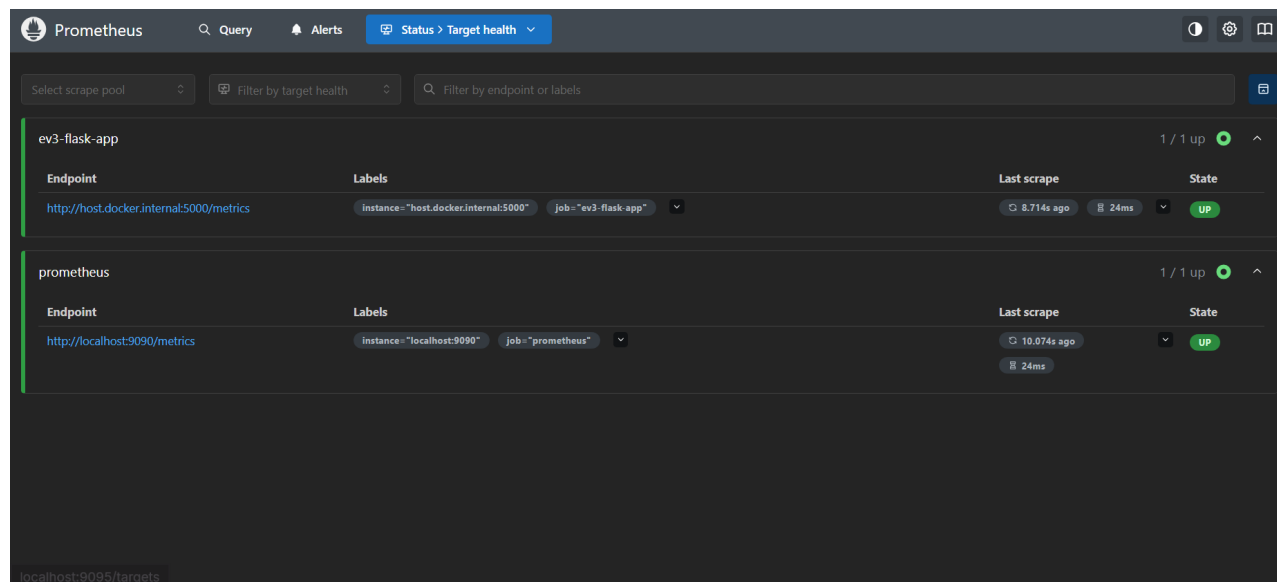
Descripción: Se visualiza la página principal de **Grafana** luego de iniciar sesión correctamente. Esta captura confirma que el servicio no solo fue levantado desde Docker, sino que también permite el acceso a su interfaz web. Desde este panel se pueden agregar fuentes de datos, crear dashboards y revisar opciones de administración para configurar la monitorización del proyecto.

Captura 64: Configuración de Prometheus como fuente de datos en Grafana.

Descripción: Se muestra la sección **Data sources** de Grafana, donde se está configurando **Prometheus** como fuente de datos para el monitoreo del proyecto. En el campo **Prometheus server URL** se ingresa la dirección `http://ev3-prometheus:9090`, que corresponde al nombre del contenedor de Prometheus definido en Docker Compose y al puerto interno del servicio. Esta configuración permite que Grafana se comuniquen directamente con Prometheus dentro de la red creada por Docker, sin depender de una conexión externa. Además, en la parte superior se identifica el tipo de fuente como **Prometheus** y se indica que el soporte de alertas está habilitado. Esta evidencia demuestra que Grafana fue preparado para consultar las métricas recolectadas por Prometheus y utilizarlas posteriormente en dashboards de observabilidad.

Captura 65: Acceso a la interfaz de consulta de Prometheus.

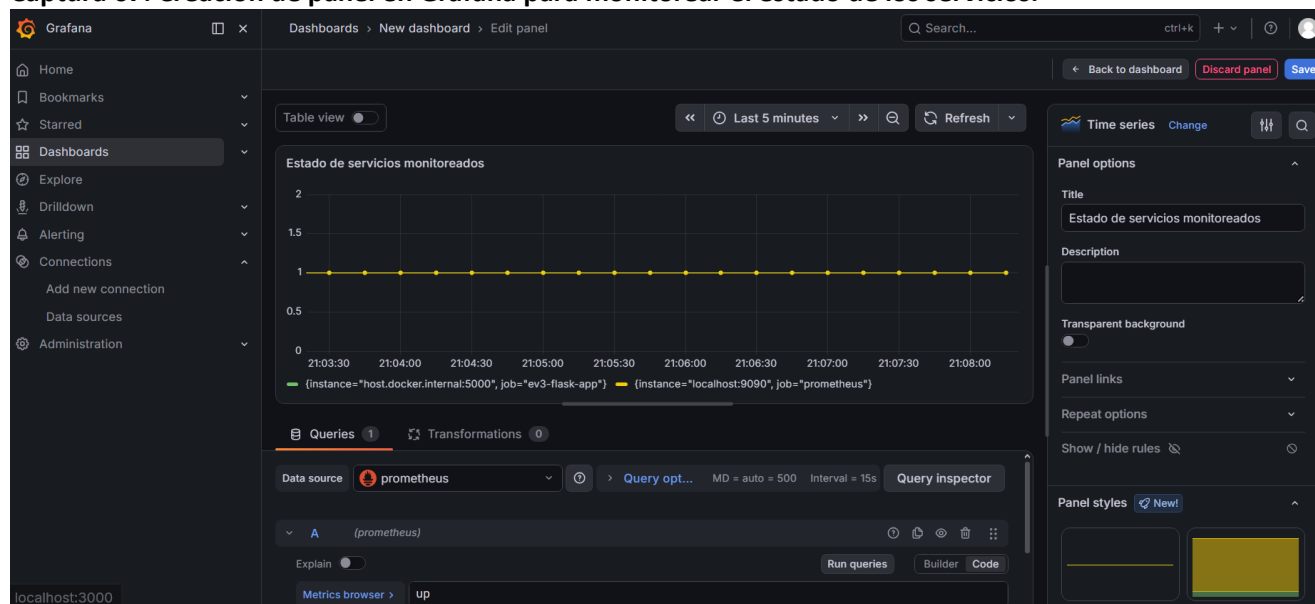
Descripción: Se visualiza la interfaz web de **Prometheus**, específicamente la sección **Query**, donde se pueden ejecutar consultas para revisar las métricas recolectadas. Esta captura confirma que Prometheus se encuentra activo y accesible desde el navegador. Desde esta pantalla es posible ingresar expresiones, ejecutar consultas y visualizar los resultados en formato de tabla o gráfico, permitiendo validar que el sistema de monitorización está funcionando correctamente.

Captura 66: Verificación del estado de los targets en Prometheus.

Descripción: Se muestra la sección **Status > Target health** de Prometheus, donde se verifica que los objetivos configurados para la recolección de métricas se encuentran activos. En la captura aparecen dos targets en estado **UP**: ev3-flask-app, correspondiente a la aplicación Flask expuesta en <http://host.docker.internal:5000/metrics>, y prometheus, correspondiente al propio servicio Prometheus en <http://localhost:9090/metrics>. Esta evidencia confirma que Prometheus está recopilando métricas correctamente desde la aplicación y desde su propio servicio, validando el funcionamiento de la configuración definida en prometheus.yml.

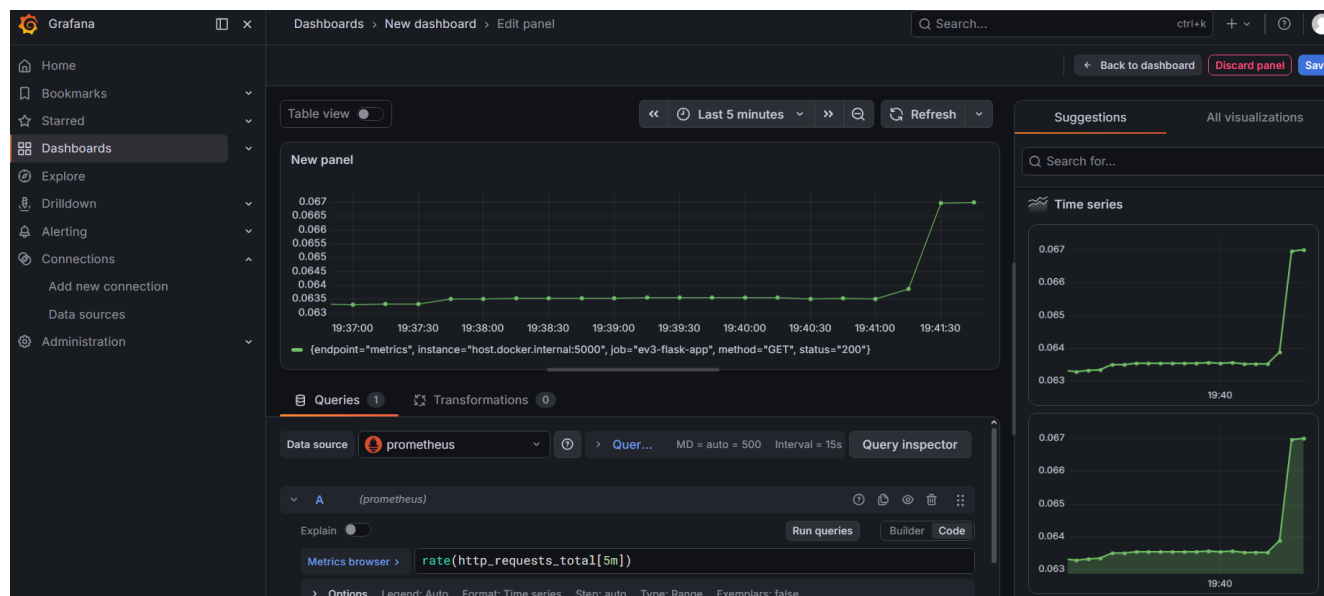
- Monitorea el entorno de producción en tiempo real para detectar posibles incidentes de seguridad y documenta los resultados de la monitorización. (Pegar Pantallazos)

Captura 67: Creación de panel en Grafana para monitorear el estado de los servicios.



Descripción: Se muestra la creación de un panel en **Grafana** utilizando la fuente de datos **Prometheus**. En el gráfico de tipo **Time series** se consulta la métrica **up**, la cual permite verificar si los servicios monitoreados se encuentran activos. El panel fue titulado **“Estado de servicios monitoreados”** y muestra datos de los últimos 5 minutos. La línea del gráfico se mantiene en valor 1, lo que indica que los targets monitoreados, como la aplicación **ev3-flask-app** y el servicio **prometheus**, se encuentran disponibles y respondiendo correctamente. Esta evidencia permite monitorear en tiempo real la disponibilidad de los servicios y detectar rápidamente una posible caída o incidente.

Captura 68: Monitoreo de solicitudes HTTP de la aplicación en Grafana.



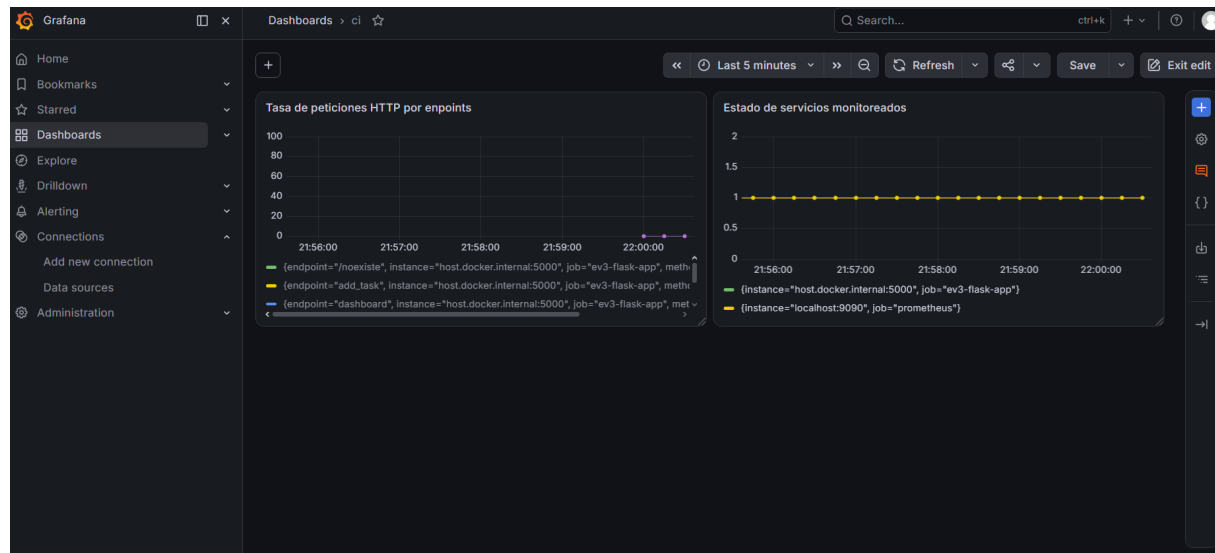
Descripción: Se muestra la creación de un panel en **Grafana** utilizando la fuente de datos **Prometheus**. En la consulta se utiliza la métrica `rate(http_requests_total[5m])`, la cual permite observar la tasa de solicitudes HTTP recibidas por la aplicación durante los últimos 5 minutos. En el gráfico se visualiza el endpoint `/metrics` de la aplicación `ev3-flask-app`, con estado HTTP 200, lo que indica que las peticiones están siendo respondidas correctamente. Esta evidencia permite monitorear el tráfico de la aplicación y detectar posibles aumentos inusuales de solicitudes que podrían estar relacionados con errores, sobrecarga o intentos de acceso anómalos.

Captura 69: Panel de tasa de peticiones HTTP por endpoint en Grafana.



Descripción: Se visualiza la configuración del panel "**Tasa de peticiones HTTP por endpoint**" en Grafana, tipo **Time series**, consultando datos de los últimos **5 minutos**. El gráfico desagrega la métrica por endpoint, permitiendo identificar el tráfico recibido en rutas como **/delete_task/1** (status 405), **/noexiste** (status 404), **add_task** (status 302), **admin** (status 200), **dashboard** (status 200) e **index** (status 200). Esta evidencia permite monitorear en tiempo real el comportamiento de la aplicación **ev3-flask-app**, detectando patrones anómalos como solicitudes a rutas inexistentes (**status 404** en /noexiste) o métodos no permitidos (**status 405** en /delete_task/1), los cuales podrían corresponder a intentos de acceso indebido o incidentes de seguridad dentro del entorno de producción.

- Documenta todas las actividades de monitorización realizadas y los incidentes de seguridad detectados, incluyendo las acciones tomadas para responder a dichos incidentes. (Pegar Pantallazos)

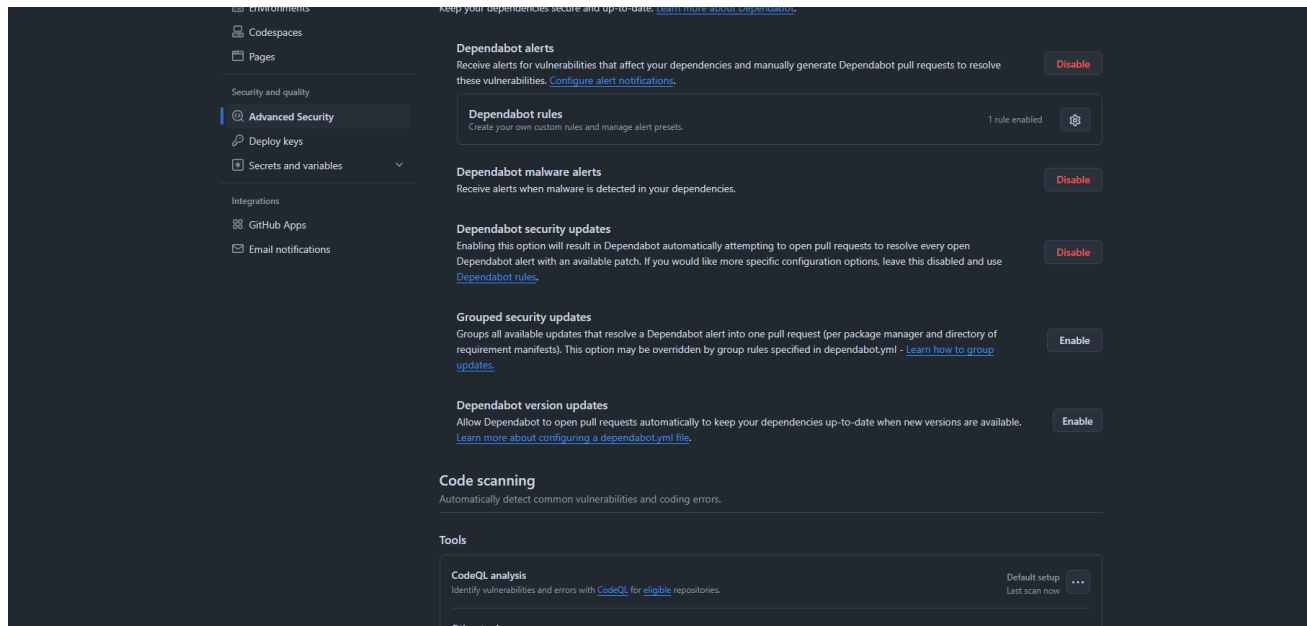
Captura 70: Dashboard consolidado de monitorización en Grafana con paneles de tráfico y disponibilidad de servicios.

Descripción: Se visualiza el dashboard "ci" en Grafana, que integra en una sola vista los dos paneles de monitorización configurados durante la actividad. El panel izquierdo, "Tasa de peticiones HTTP por endpoints", muestra el tráfico recibido en distintos endpoints de la aplicación, incluyendo /noexiste, add_task y dashboard, correspondientes al job ev3-flask-app. El panel derecho, "Estado de servicios monitoreados", muestra la métrica up con valor constante en 1 para ambos targets (ev3-flask-app y prometheus), confirmando que los servicios se mantuvieron activos y disponibles durante el periodo observado (últimos 5 minutos). Esta evidencia permite validar que la monitorización del entorno de producción se realizó en tiempo real, no detectándose incidentes de caída de servicio durante la ventana analizada, mientras que el tráfico por endpoint permite identificar patrones de acceso, incluyendo solicitudes a rutas inexistentes que podrían representar intentos de exploración o acceso indebido.

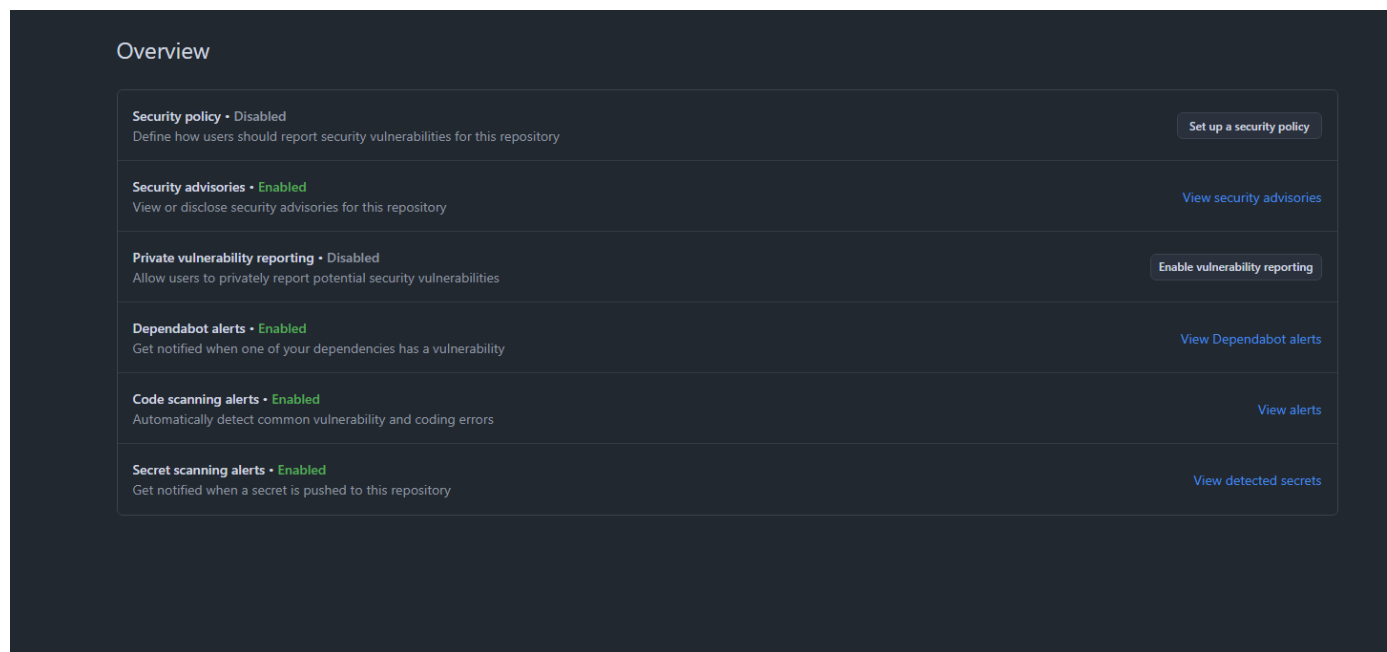
Gestion de dependencias:

- Implementa la gestión de dependencias en el pipeline utilizando Dependabot.

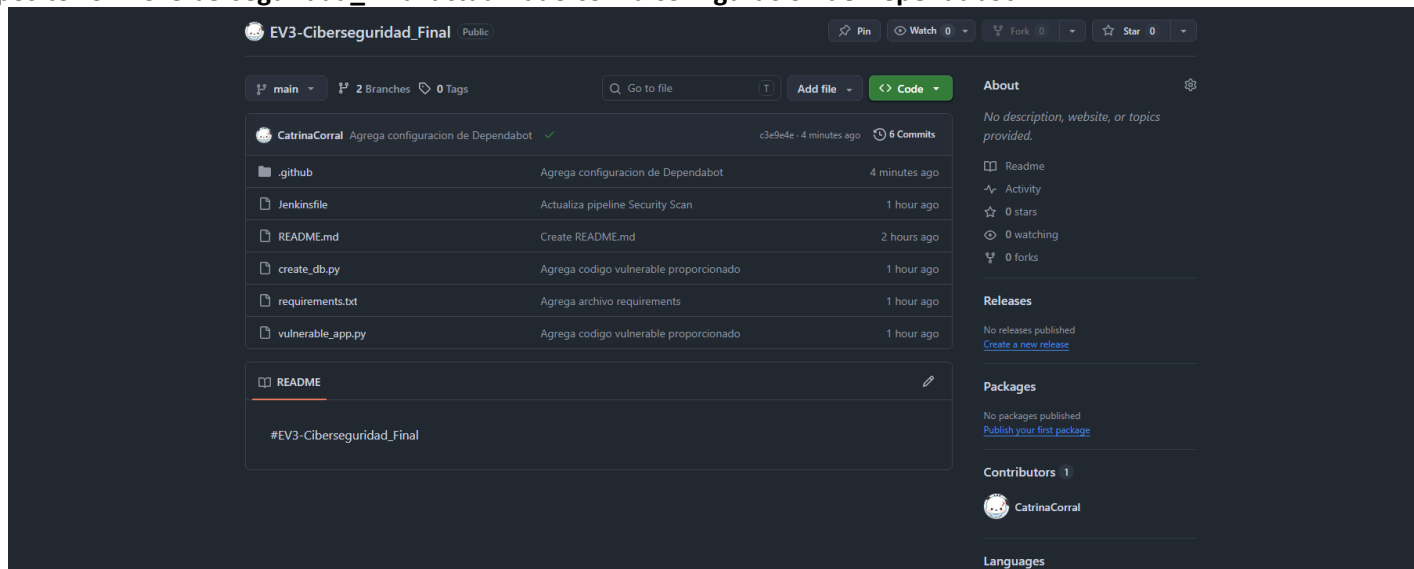
Captura 71: Configuración de Advanced Security y Dependabot en el repositorio GitHub.



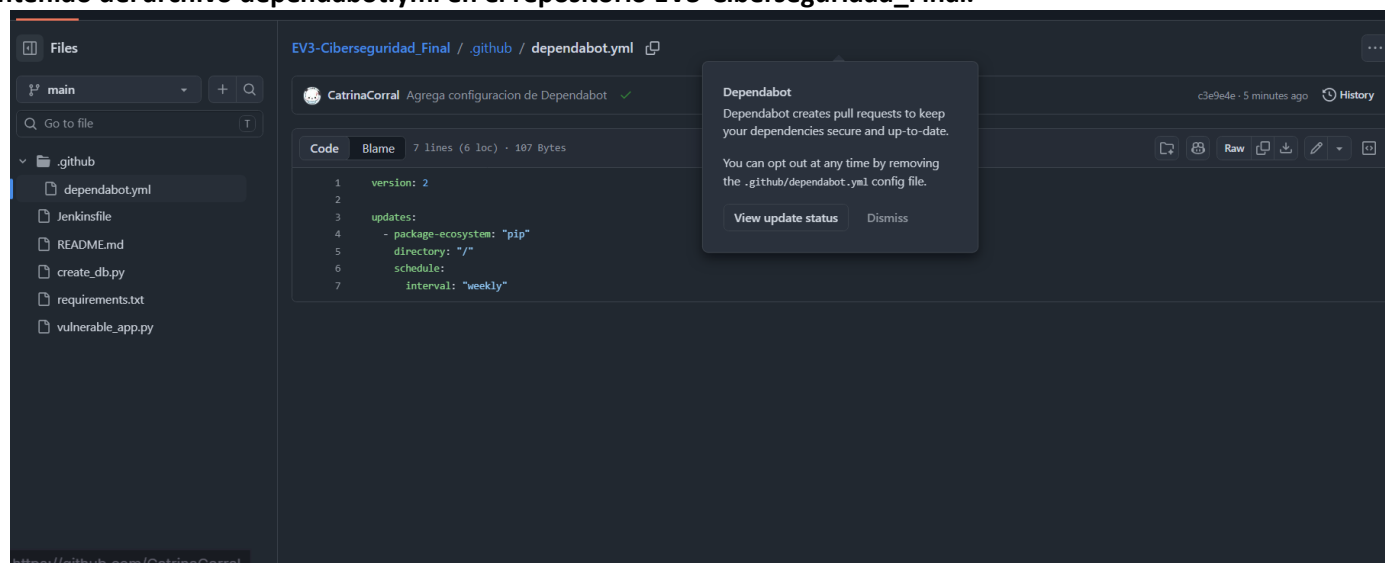
Captura 72: Vista general (Overview) de la configuración de seguridad del repositorio en GitHub.



Descripción: Se visualiza el panel "Overview" de seguridad del repositorio "EV3-Ciberseguridad_Final" en GitHub, donde se resumen los estados de las distintas funcionalidades de seguridad disponibles. Se observa que las Security advisories, Dependabot alerts, Code scanning alerts y Secret scanning alerts se encuentran en estado Enabled, lo que confirma que el repositorio cuenta con detección activa de vulnerabilidades en dependencias, errores de codificación y exposición accidental de secretos. Por su parte, la Security policy y el Private vulnerability reporting permanecen en estado Disabled, al no ser funcionalidades requeridas para esta etapa del proyecto. Esta evidencia consolida la implementación de la gestión de dependencias mediante Dependabot dentro del repositorio, demostrando que el pipeline cuenta con mecanismos automatizados para identificar y alertar sobre vulnerabilidades en las librerías utilizadas por la aplicación.

Captura 73: Repositorio EV3-Ciberseguridad_Final actualizado con la configuración de Dependabot.

Descripción: Se visualiza el repositorio "EV3-Ciberseguridad_Final" en GitHub, donde se observa la incorporación de la carpeta .github mediante el commit "Agrega configuracion de Dependabot", realizado por CatrinaCorral hace 4 minutos. Esta carpeta corresponde al directorio donde se aloja el archivo de configuración de Dependabot (dependabot.yml), encargado de definir el ecosistema de paquetes a monitorear y la frecuencia de revisión de actualizaciones. Además, se mantiene visible el historial de commits del proyecto, incluyendo la actualización del Jenkinsfile con el pipeline de Security Scan y la incorporación previa del código vulnerable (create_db.py, vulnerable_app.py) y el archivo requirements.txt. Esta evidencia confirma que la configuración de Dependabot fue versionada correctamente dentro del repositorio, quedando disponible para automatizar la gestión de dependencias y sus actualizaciones de seguridad.

Captura 74: Contenido del archivo dependabot.yml en el repositorio EV3-Ciberseguridad_Final.

Descripción: Se visualiza el contenido del archivo dependabot.yml ubicado dentro de la carpeta .github del repositorio "EV3-Ciberseguridad_Final". El archivo define la version 2 de la configuración e incluye un bloque de updates para el ecosistema de paquetes pip, indicando como directory la raíz del proyecto ("/") y un schedule con interval semanal ("weekly"). GitHub reconoce automáticamente este archivo de configuración y muestra el mensaje informativo "Dependabot creates pull requests to keep your dependencies secure and up-to-date", confirmando que la herramienta quedó activa para el proyecto. Esta evidencia demuestra la implementación de la gestión de dependencias en el pipeline mediante Dependabot, configurando la revisión automática y periódica de las librerías Python definidas en requirements.txt.

Captura 75: Ejecución exitosa del build #14 del pipeline "EV3-Pipeline_Final" con etapa de análisis de dependencias.

Jenkins / EV3-Pipeline_Final / #14 / Stages

✓ < #14

Started by Catrina Corral Started 8.7 Seg ago Build has been executing for 8.7 Seg

Start End

Checkout SCM Build Test - Unit Deploy - Staging Security Test - SCA Dependencies Security Test - DAST ZAP Post Actions

Search

✓ Build 1min 46s

✓ Checkout SCM 2s

✓ Test - Unit 28s

✓ Deploy - Staging 11s

✓ Security Test - SCA Dependencies 5

✓ Security Test - DAST ZAP 2min 50s

✓ Post Actions 2s

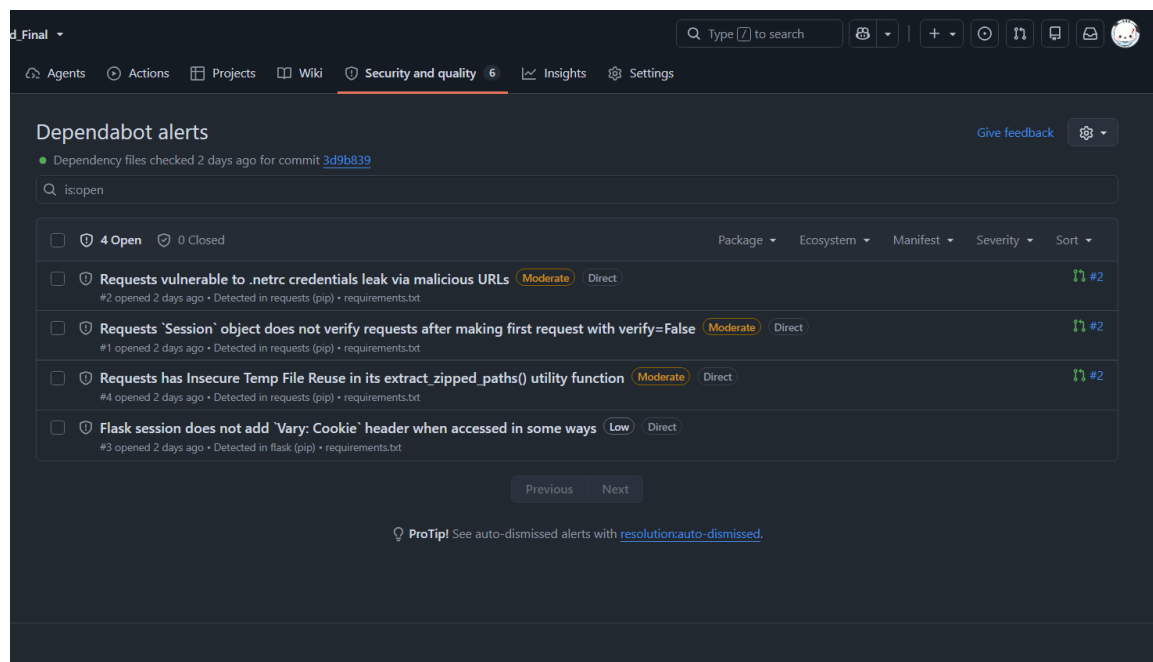
✓ Build 1min 46s Comenzó hace 7min Jenkins

✓ docker network create devsecops-network || true docker build -t ev3-ciberseguridad_final:\${BUILD_NUMBER} . 1min 46s

```
0 + docker network create devsecops-network
1 04abacadcd83fe3021a6860d23c55f36257f72b2d75e99571c88a56978b18a19
2 + docker build -t ev3-ciberseguridad_final:14 .
3 #0 building with "default" instance using docker driver
4
5 #1 [internal] load build definition from Dockerfile
6 #1 transferring dockerfile:
7 #1 transferring dockerfile: 331B 0.1s done
8 #1 DONE 0.3s
9
```

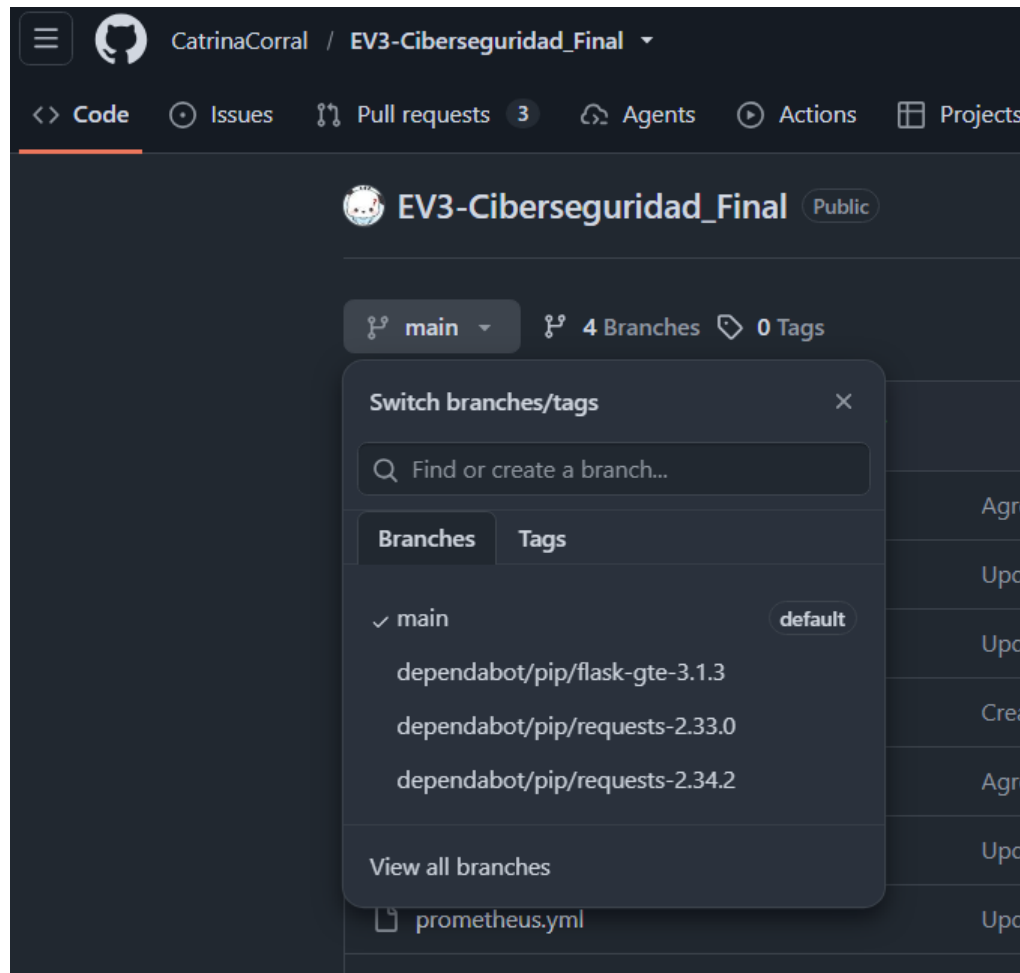
Descripción: Se visualiza la ejecución del build #14 del pipeline "EV3-Pipeline_Final", iniciado por Catrina Corral, completando correctamente todas sus etapas: Checkout SCM, Build, Test - Unit, Deploy - Staging, Security Test - SCA Dependencies, Security Test - DAST ZAP y Post Actions. Se observa el detalle de la etapa Build, donde se ejecuta el comando `docker network create devsecops-network` y posteriormente `docker build -t ev3-ciberseguridad_final:14`, confirmando la construcción exitosa de la imagen Docker de la aplicación. La incorporación de la etapa "Security Test - SCA Dependencies" dentro del flujo evidencia que el pipeline integra la revisión automatizada de dependencias como parte del ciclo de vida del desarrollo, complementando la gestión de actualizaciones seguras realizada mediante Dependabot.

Captura 76: Alertas de Dependabot detectadas en el repositorio EV3-Ciberseguridad_Final.



Descripción: Se visualiza la sección "Dependabot alerts" del repositorio, donde se identifican 4 alertas abiertas asociadas a las dependencias declaradas en requirements.txt. Se detectan tres vulnerabilidades de severidad Moderate en el paquete requests (pip), relacionadas con la fuga de credenciales .netrc mediante URLs maliciosas, el objeto Session que no verifica solicitudes tras un primer request con verify=False, y la reutilización insegura de archivos temporales en la función extract_zipped_paths(). Además, se identifica una alerta de severidad Low en el paquete flask, relacionada con la ausencia de la cabecera Vary: Cookie en ciertos accesos a la sesión. Esta evidencia confirma que Dependabot está monitoreando activamente las dependencias del proyecto y generando alertas detalladas sobre vulnerabilidades conocidas en librerías directas (Direct) utilizadas por la aplicación.

Captura 77: Ramas generadas automáticamente por Dependabot para actualizar dependencias.

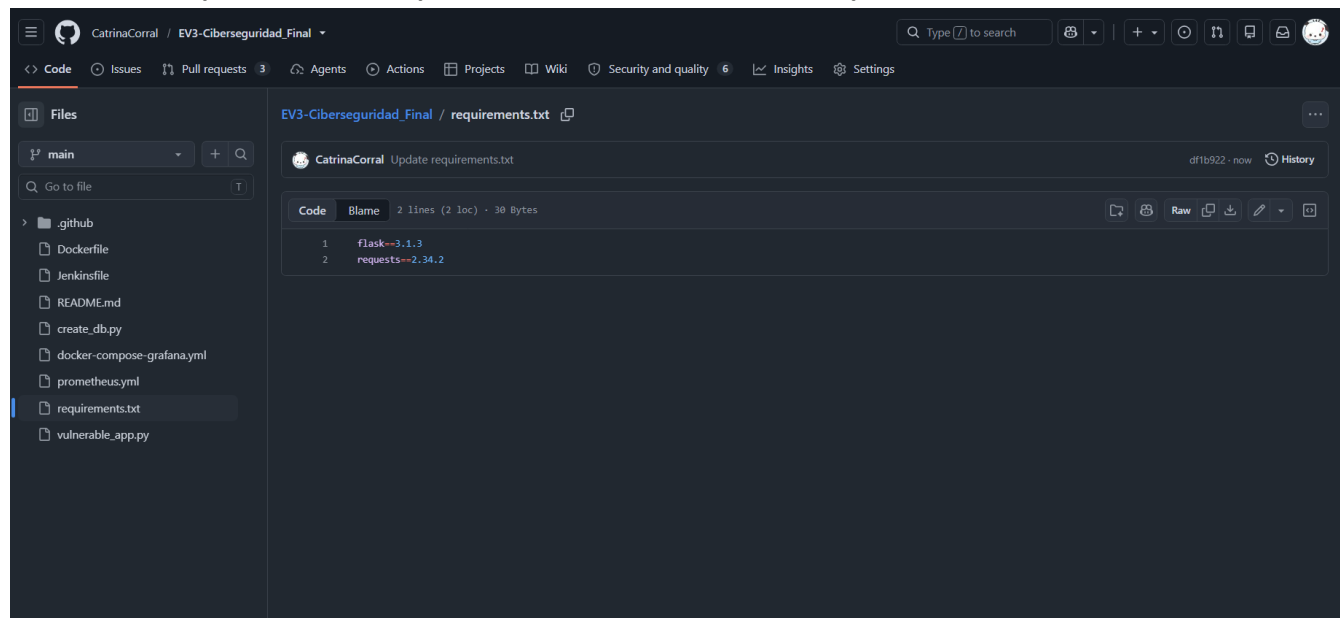


Descripción: Se visualiza el listado de ramas del repositorio "EV3-Ciberseguridad_Final", donde además de la rama main se observan tres ramas creadas automáticamente por Dependabot: dependabot/pip/flask-gte-3.1.3, dependabot/pip/requests-2.33.0 y dependabot/pip/requests-2.34.2. Estas ramas corresponden a propuestas de actualización de versión generadas para resolver las vulnerabilidades detectadas en Flask y en las distintas versiones sugeridas para requests. Esta evidencia demuestra que Dependabot no solo identifica vulnerabilidades, sino que también automatiza la generación de actualizaciones mediante ramas y pull requests, facilitando la gestión de dependencias y su respectiva corrección dentro del ciclo de vida del proyecto.

- Asegúrate de actualizar todas las dependencias del código vulnerable proporcionado y verifica que las actualizaciones no

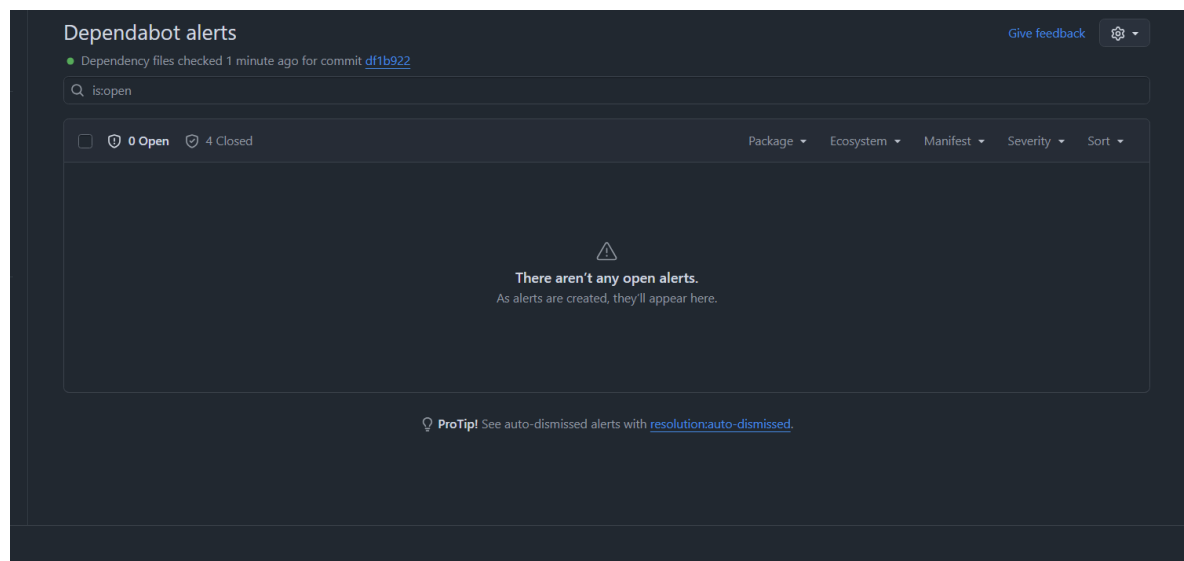
introduzcan nuevas vulnerabilidades.

Captura 78: Actualización de dependencias en requirements.txt tras las alertas de Dependabot.



Descripción: Se visualiza el archivo requirements.txt del repositorio "EV3-Ciberseguridad_Final", actualizado mediante el commit "Update requirements.txt" realizado por CatrinaCorral. Las dependencias fueron fijadas con las versiones parcheadas correspondientes a las alertas detectadas previamente: flask==3.1.3 y requests==2.34.2. Esta actualización corresponde a la fusión de las ramas generadas automáticamente por Dependabot (dependabot/pip/flask-gte-3.1.3 y dependabot/pip/requests-2.34.2), asegurando que se aplicaron las versiones más recientes que resuelven las vulnerabilidades identificadas de tipo Moderate y Low. Esta evidencia confirma que las dependencias del código vulnerable proporcionado fueron actualizadas correctamente, quedando pendiente la verificación posterior mediante el pipeline para constatar que dichas actualizaciones no introdujeron nuevas vulnerabilidades.

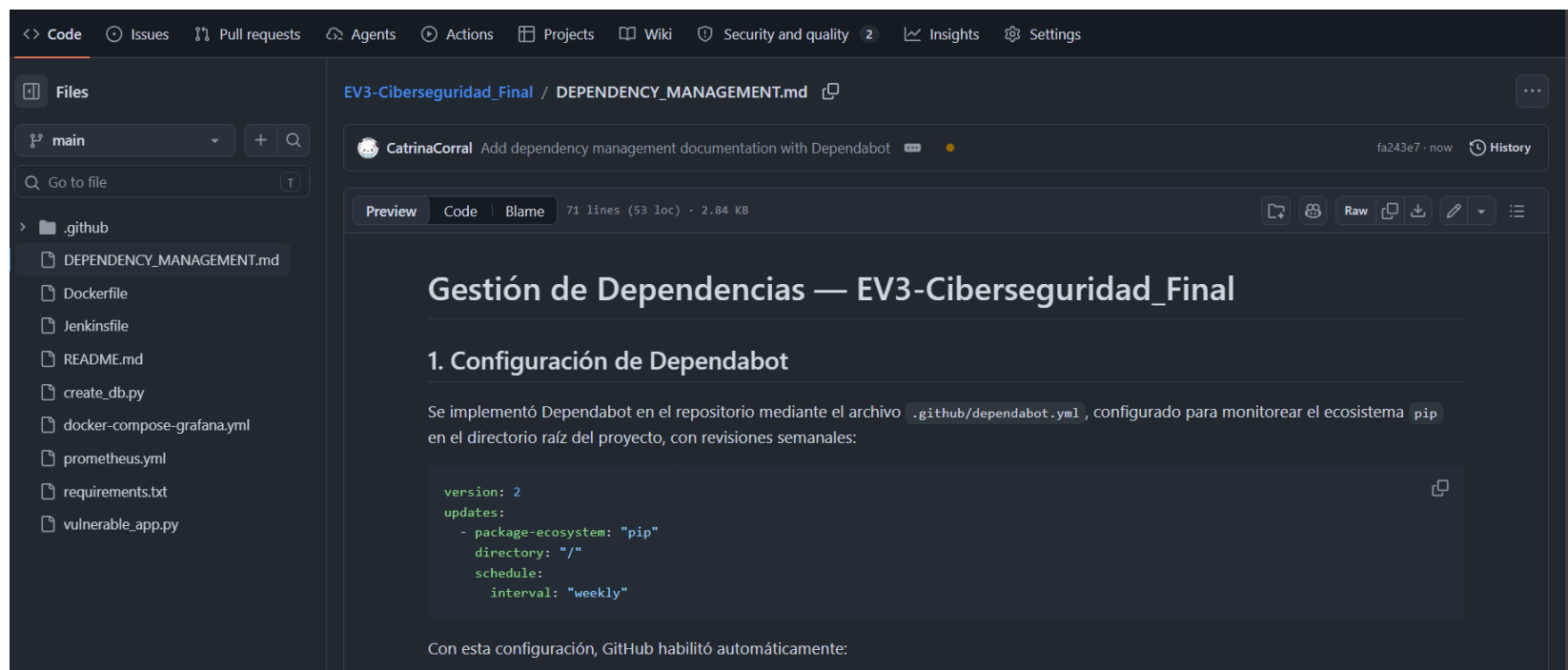
Captura 79: Verificación de resolución de alertas de Dependabot tras la actualización de dependencias.



Descripción: Se visualiza la sección "Dependabot alerts" del repositorio "EV3-Ciberseguridad_Final", donde se confirma que las 4 alertas previamente detectadas se encuentran ahora en estado Closed, indicando "0 Open" y "4 Closed". El mensaje "There aren't any open alerts" confirma que no existen vulnerabilidades activas asociadas a las dependencias del proyecto tras la actualización del commit df1b922. Esta evidencia demuestra que la actualización de flask a la versión 3.1.3 y de requests a la versión 2.34.2 resolvió correctamente las vulnerabilidades identificadas, validando que las nuevas versiones instaladas no introdujeron vulnerabilidades adicionales dentro del ecosistema de dependencias del proyecto.

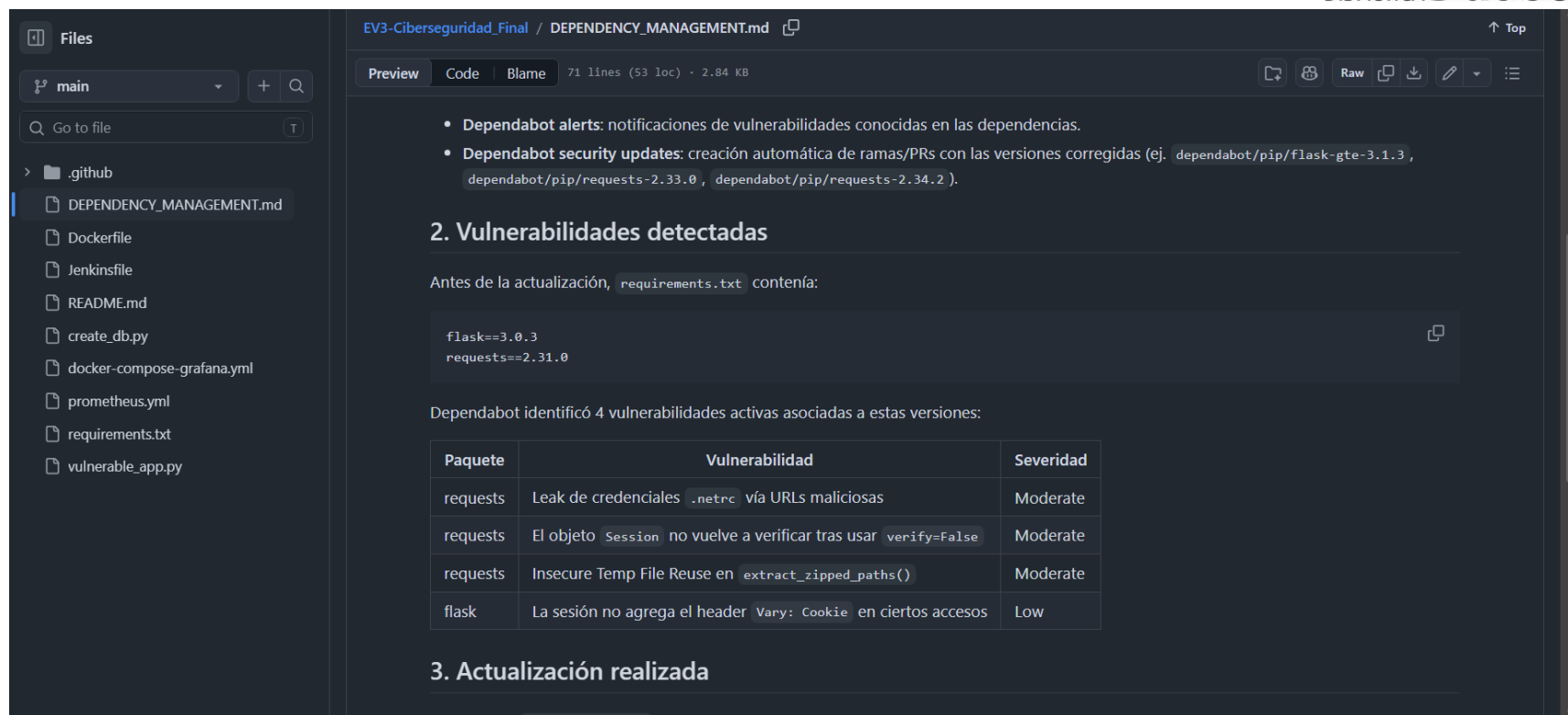
- Documenta el proceso de gestión de dependencias, incluyendo las actualizaciones realizadas y cómo se aseguraron de no introducir nuevas vulnerabilidades.

Captura 80: Documento DEPENDENCY_MANAGEMENT.md con el proceso de gestión de dependencias.



Descripción: Se visualiza el archivo DEPENDENCY_MANAGEMENT.md creado en el repositorio "EV3-Ciberseguridad_Final" mediante el commit "Add dependency management documentation with Dependabot". El documento detalla en su sección "1. Configuración de Dependabot" que la herramienta fue implementada mediante el archivo .github/dependabot.yml, configurado para monitorear el ecosistema pip en el directorio raíz del proyecto con revisiones semanales (interval: weekly), mostrando el contenido completo del archivo de configuración. Esta evidencia demuestra que el proceso de gestión de dependencias fue documentado formalmente dentro del repositorio, dejando registro de la configuración aplicada para automatizar la detección y actualización de dependencias vulnerables.

Captura 81: Detalle de vulnerabilidades detectadas antes de la actualización de dependencias.



Files

main

Go to file

.github

DEPENDENCY_MANAGEMENT.md

Dockerfile

Jenkinsfile

README.md

create_db.py

docker-compose-grafana.yml

prometheus.yml

requirements.txt

vulnerable_app.py

EV3-Ciberseguridad_Final / DEPENDENCY_MANAGEMENT.md

Preview Code Blame 71 lines (53 loc) · 2.84 KB

Raw Copy Download Edit

- **Dependabot alerts:** notificaciones de vulnerabilidades conocidas en las dependencias.
- **Dependabot security updates:** creación automática de ramas/PRs con las versiones corregidas (ej. `dependabot/pip/flask-gte-3.1.3`, `dependabot/pip/requests-2.33.0`, `dependabot/pip/requests-2.34.2`).

2. Vulnerabilidades detectadas

Antes de la actualización, `requirements.txt` contenía:

```
flask==3.0.3
requests==2.31.0
```

Dependabot identificó 4 vulnerabilidades activas asociadas a estas versiones:

Paquete	Vulnerabilidad	Severidad
requests	Leak de credenciales <code>.netrc</code> vía URLs maliciosas	Moderate
requests	El objeto <code>Session</code> no vuelve a verificar tras usar <code>verify=False</code>	Moderate
requests	Insecure Temp File Reuse en <code>extract_zipped_paths()</code>	Moderate
flask	La sesión no agrega el header <code>Vary: Cookie</code> en ciertos accesos	Low

3. Actualización realizada

Descripción: Se visualiza la continuación del documento `DEPENDENCY_MANAGEMENT.md`, específicamente las secciones "Dependabot alerts" y "Dependabot security updates", donde se detalla que la creación automática de ramas/PRs con versiones corregidas incluyó `dependabot/pip/flask-gte-3.1.3`, `dependabot/pip/requests-2.33.0` y `dependabot/pip/requests-2.34.2`. En la sección "2. Vulnerabilidades detectadas" se muestra que antes de la actualización el archivo `requirements.txt` contenía `flask==3.0.3` y `requests==2.31.0`, versiones sobre las cuales Dependabot identificó 4 vulnerabilidades activas: leak de credenciales `.netrc` vía URLs maliciosas, el objeto `Session` que no vuelve a verificar tras usar `verify=False` y reutilización insegura de archivos temporales en `extract_zipped_paths()`, todas de severidad Moderate en el paquete `requests`, además de una vulnerabilidad Low en `flask` relacionada con la ausencia del header `Vary: Cookie` en ciertos accesos a la sesión.

Captura 82: Actualización, verificación y conclusión del proceso de gestión de dependencias.

Files

main

Go to file

.github

- DEPENDENCY_MANAGEMENT.md
- Dockerfile
- Jenkinsfile
- README.md
- create_db.py
- docker-compose-grafana.yml
- prometheus.yml
- requirements.txt
- vulnerable_app.py

EV3-Ciberseguridad_Final / DEPENDENCY_MANAGEMENT.md

Preview Code Blame 71 lines (53 loc) · 2.84 KB

3. Actualización realizada

Se actualizó `requirements.txt` a las versiones mínimas seguras indicadas por Dependabot:

```
flask==3.1.3
requests==2.34.2
```

Cambio aplicado en el commit `df1b922` ("Update requirements.txt").

4. Verificación de que no se introdujeron nuevas vulnerabilidades

Tras el commit, GitHub volvió a analizar automáticamente las dependencias ("Dependency files checked... for commit df1b922"), confirmando:

- **0 alertas abiertas** (0 Open / 4 Closed) en Dependabot → Vulnerabilities.
- No se generaron nuevas alertas de Dependabot tras la actualización.
- Se re-ejecutó el pipeline de Jenkins (`EV3-Pipeline_Final`) con las nuevas versiones, validando que la etapa `Security Scan` (Bandit + Safety) se ejecutara correctamente y sin errores de compatibilidad, confirmando que la actualización no rompió la aplicación ni introdujo hallazgos nuevos.

5. Conclusión

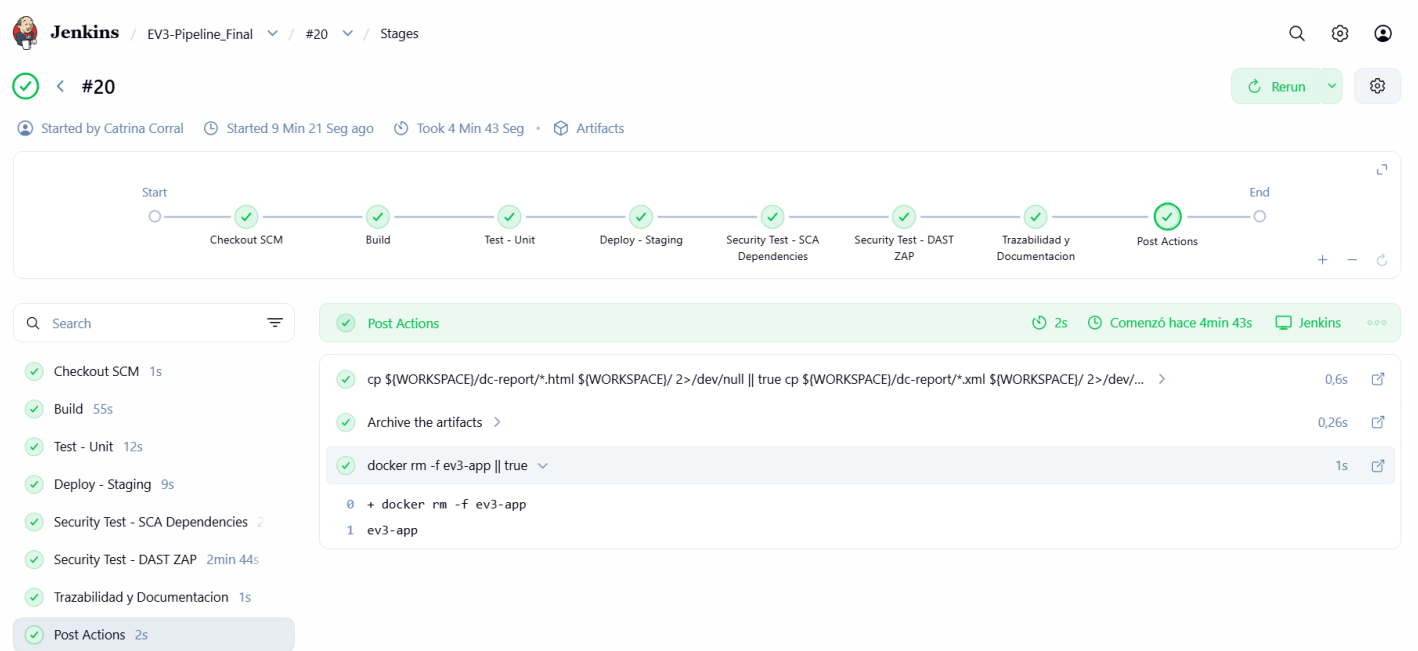
El proceso de gestión de dependencias se realizó de forma trazable: detección automática (Dependabot) → actualización manual dirigida por las recomendaciones del propio Dependabot → verificación posterior (cierre de alertas + re-ejecución del pipeline). Esto asegura que las dependencias del proyecto se mantienen libres de vulnerabilidades conocidas sin afectar la funcionalidad de la aplicación.

Descripción: Se visualiza el cierre del documento `DEPENDENCY_MANAGEMENT.md`. En la sección "3. Actualización realizada" se detalla que `requirements.txt` fue actualizado a las versiones mínimas seguras indicadas por Dependabot (`flask==3.1.3` y `requests==2.34.2`), cambio aplicado mediante el commit `df1b922`. La sección "4. Verificación de que no se introdujeron nuevas vulnerabilidades" confirma que, tras el commit, GitHub volvió a analizar automáticamente las dependencias, arrojando 0 alertas abiertas (0 Open / 4 Closed) y sin generación de nuevas alertas. Además, se documenta que se re-ejecutó el pipeline de Jenkins (`EV3-Pipeline_Final`), validando que la etapa `Security Scan` (Bandit + Safety) se ejecutó correctamente y sin errores de compatibilidad. Finalmente, la sección "5. Conclusión" resume que el proceso se realizó de forma trazable: detección automática mediante Dependabot, actualización manual dirigida por sus recomendaciones y verificación posterior mediante cierre de alertas y re-ejecución del pipeline, asegurando que las dependencias del proyecto se mantienen libres de vulnerabilidades conocidas sin afectar la funcionalidad de la aplicación.

Documentación y Trazabilidad en SDLC:

- Prepara la documentación detallada y trazabilidad de todas las actividades realizadas en el pipeline, incluyendo revisiones de seguridad, pruebas automatizadas y gestión de dependencias.

Captura 83: Ejecución completa del pipeline con nueva etapa de Trazabilidad y Documentación.



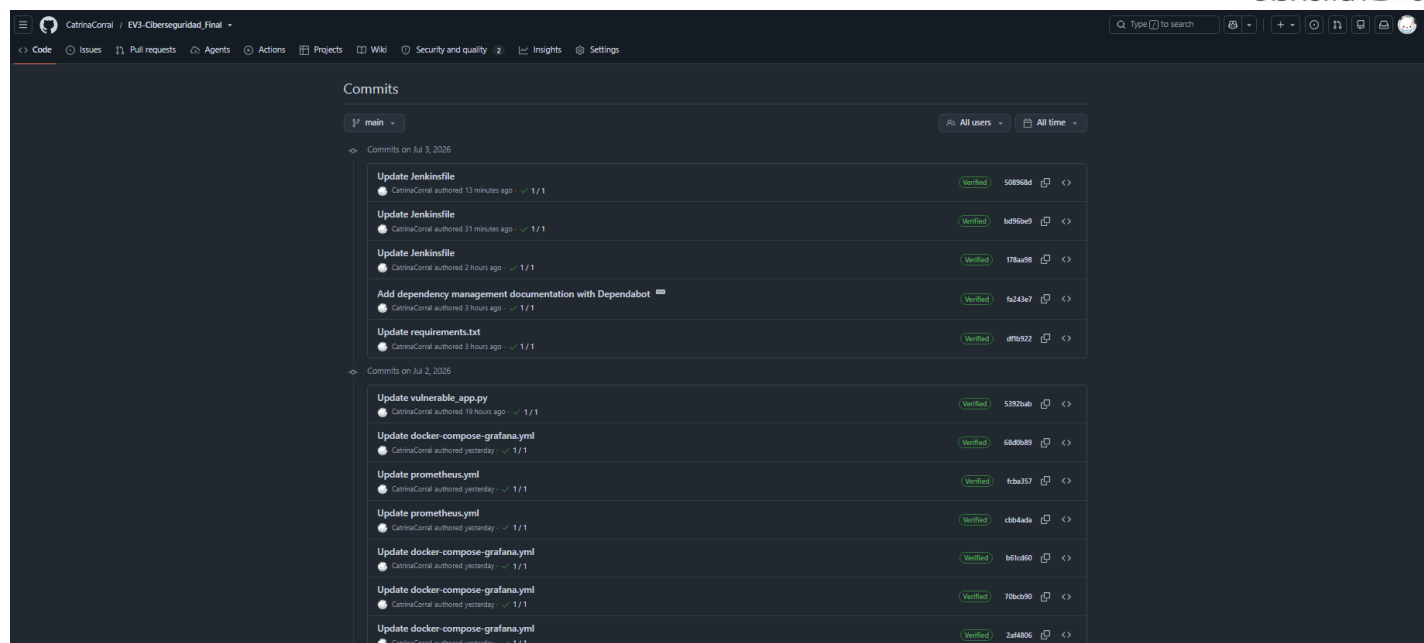
Descripción: Se visualiza el build #20 del pipeline "EV3-Pipeline_Final", iniciado por Catrina Corral, con una duración total de 4 minutos 43 segundos. El flujo incorpora una nueva etapa denominada "Trazabilidad y Documentación", ejecutada exitosamente entre "Security Test - DAST ZAP" y "Post Actions". En el detalle de la etapa Post Actions se observa la copia de los reportes generados (dc-report en formato HTML y XML), el archivado de artefactos (Archive the artifacts) y la eliminación del contenedor ev3-app mediante `docker rm -f ev3-app`, confirmando la limpieza del entorno al finalizar la ejecución. Esta evidencia demuestra que el pipeline fue ampliado para generar automáticamente documentación y trazabilidad de cada build ejecutado.

Captura 84: Registro de trazabilidad generado automáticamente por el build #20.

```
trazabilidad_build_20.md X
C:\Users\danae\Downloads > trazabilidad_build_20.md > ## # Registro de Trazabilidad - Build #20 > ## Monitorizacion
1  # Registro de Trazabilidad - Build #20
2
3  ## Informacion del Build
4  - **Numero de Build:** 20
5  - **Fecha de ejecucion:** 2026-07-03 22:48:08
6  - **Job:** EV3-Pipeline_Final
7
8  ## Control de Versiones (Git)
9  - **Commit:** 548968d83990d3af4c7772aed0cfca6822d6eeea
10 - **Repositorio:** https://github.com/CatrinaCorral/EV3-Ciberseguridad_Final
11
12 ## Etapas Ejecutadas
13 1. Build: construccion de imagen Docker ev3-ciberseguridad_final:20 - EXITOSA
14 2. Test - Unit: ejecucion de pruebas unitarias (pytest) - EXITOSA
15 3. Deploy - Staging: despliegue del contenedor en devsecops-network - EXITOSA
16 4. Security Test - SCA Dependencies: analisis de dependencias con OWASP Dependency-Check - EXITOSA
17 5. Security Test - DAST ZAP: escaneo dinamico OWASP ZAP contra la app desplegada - EXITOSA
18 6. Trazabilidad y Documentacion: generacion de este registro - EXITOSA
19
20 ## Artefactos Generados
21 - dependency-check-report.html / .xml (resultado del escaneo SCA)
22 - zap_report.html / .json (resultado del escaneo DAST)
23 - results.xml (resultado de pruebas unitarias)
24 - trazabilidad_build_20.md (este documento)
25
26 ## Vulnerabilidades Corregidas (Codigo Fuente)
27 - Inyeccion SQL en login (CVE-89): consulta parametrizada en vez de f-string concatenado
28 - Flask en modo debug activo (CVE-94): cambiado a debug=False
29 - IDOR en eliminacion de tareas: validacion de propietario (WHERE id = ? AND user_id = ?)
30 - Cabeceras de seguridad HTTP ausentes (CVE-693): CSP, X-Frame-Options, X-Content-Type-Options, Permissions-Policy, Cross-Origin-*Policy agregadas via @app.after_request
31
32 ## Gestion de Dependencias
33 - Dependabot alerts y security updates activados en GitHub
34 - Archivo .github/dependabot.yml configurado (revision semanal, ecosistema pip)
35 - Snapshot de requirements.txt vigente en este build:
36   - flask==3.1.3
37   - requests==2.34.2
38 - Ver detalle completo en DEPENDENCY_MANAGEMENT.md y SDLC_SECURITY_DOCUMENTATION.md
39
40 ## Monitorizacion
41 - Metricas expuestas via /metrics (prometheus_client en la app Flask)
42 - Prometheus recolectando metricas del contenedor ev3-app (scrape cada 15s)
43 - Dashboard Grafana con paneles de estado de servicios y tasa de peticiones HTTP por endpoint
44
```

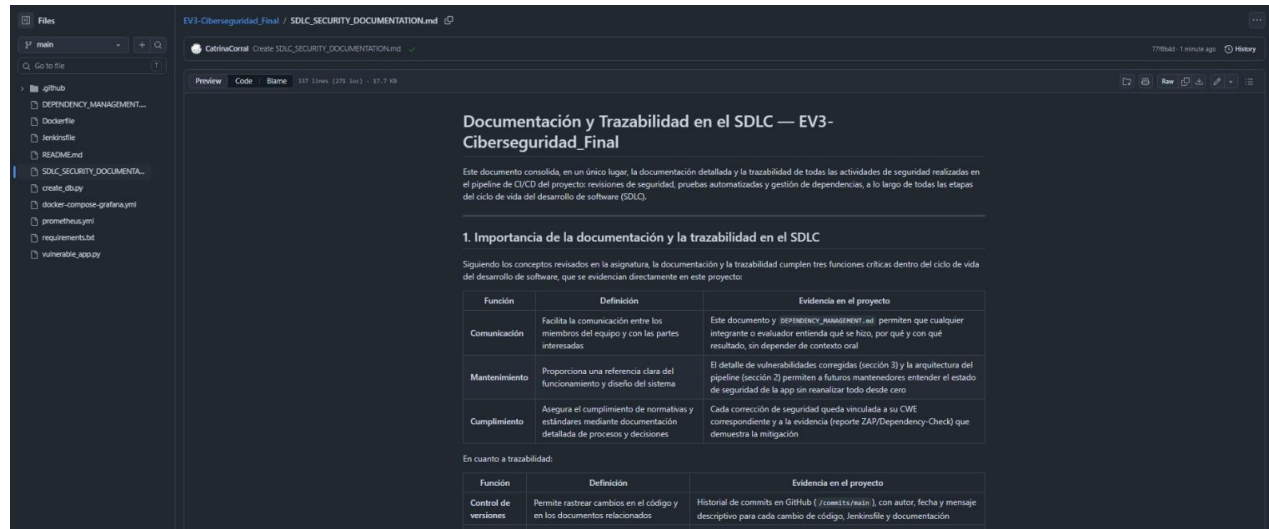
Descripción: Se visualiza el contenido del archivo trazabilidad_build_20.md, generado automáticamente por la etapa "Trazabilidad y Documentacion" del pipeline. El documento incluye la información del build (número, fecha de ejecución y job), los datos de control de versiones (commit y repositorio en GitHub), y el detalle de las seis etapas ejecutadas (Build, Test - Unit, Deploy - Staging, Security Test - SCA Dependencies, Security Test - DAST ZAP y Trazabilidad y Documentacion), todas con estado EXITOSA. Además, se listan los artefactos generados (dependency-check-report, zap_report, results.xml), las vulnerabilidades corregidas en el código fuente (inyección SQL, modo debug activo, IDOR y cabeceras HTTP ausentes), la gestión de dependencias vigente (flask==3.1.3, requests==2.34.2) y los datos de monitorización expuestos vía /metrics y consumidos por Prometheus y Grafana. Esta evidencia demuestra la generación automatizada de trazabilidad detallada por cada ejecución del pipeline.

Captura 85: Historial de commits del repositorio con verificación de firma.



Descripción: Se visualiza el historial de commits del repositorio "EV3-Ciberseguridad_Final" en GitHub, mostrando la secuencia cronológica de cambios realizados por Catrina Corral, incluyendo "Update Jenkinsfile" (en sus distintas iteraciones), "Add dependency management documentation with Dependabot" y "Update requirements.txt", todos marcados como Verified. Esta evidencia respalda la trazabilidad del proyecto al permitir identificar quién realizó cada cambio, cuándo se realizó y con qué mensaje descriptivo, cumpliendo con el requerimiento de mantener un control de versiones auditable a lo largo de todo el ciclo de vida del desarrollo.

Captura 86: Documento SDLC_SECURITY_DOCUMENTATION.md con la documentación consolidada del proyecto.

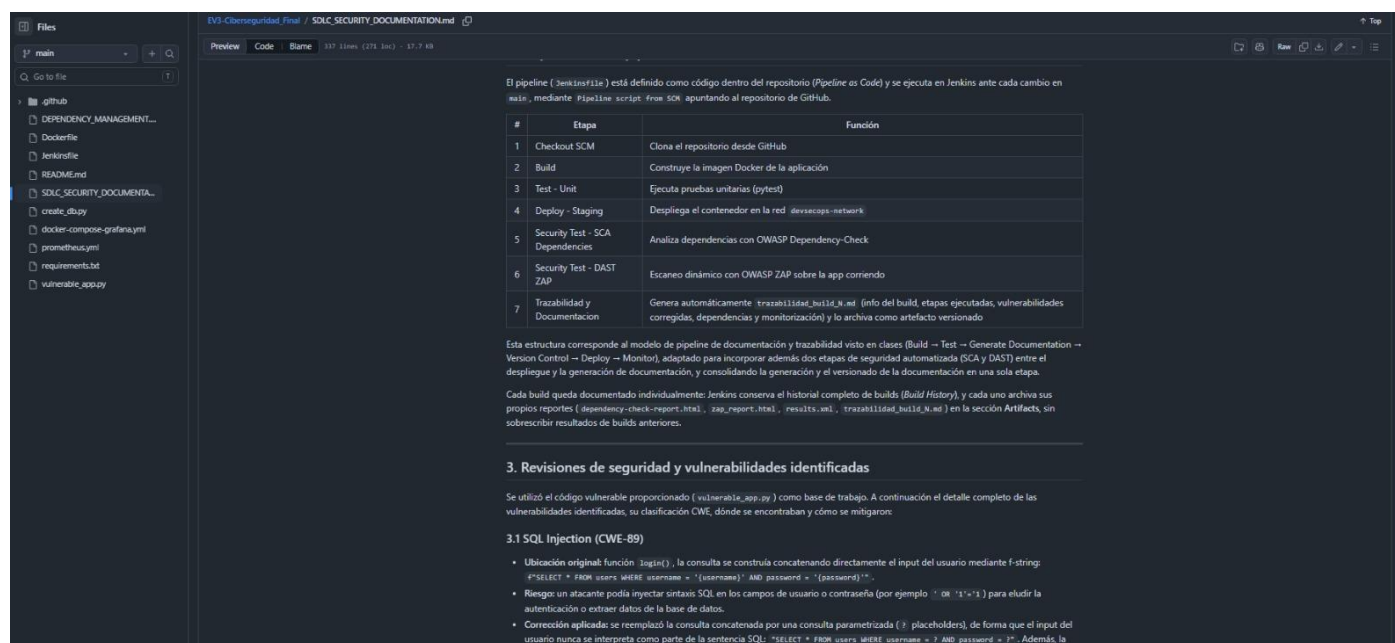


Descripción: Se visualiza el archivo `SDLC_SECURITY_DOCUMENTATION.md` creado en el repositorio mediante el commit "Create `SDLC_SECURITY_DOCUMENTATION.md`". El documento consolida en un único lugar la documentación detallada y la trazabilidad de todas las actividades de seguridad realizadas en el pipeline de CI/CD, cubriendo revisiones de seguridad, pruebas automatizadas y gestión de dependencias a lo largo de todas las etapas del SDLC. En la sección "1. Importancia de la documentación y la trazabilidad en el SDLC" se relacionan los conceptos de Comunicación, Mantenimiento y Cumplimiento con su evidencia concreta dentro del proyecto (`DEPENDENCY_MANAGEMENT.md`, detalle de vulnerabilidades corregidas y vínculo con CWE correspondiente), además de iniciar el desarrollo de los conceptos de trazabilidad, incluyendo el control de versiones mediante el historial de commits en GitHub. Esta evidencia demuestra la preparación formal de documentación detallada que respalda la implementación de prácticas de seguridad a lo largo de todo el ciclo de vida del desarrollo de software.

- Asegúrate de que toda la documentación refleje claramente la implementación de prácticas de seguridad a lo largo del ciclo de vida

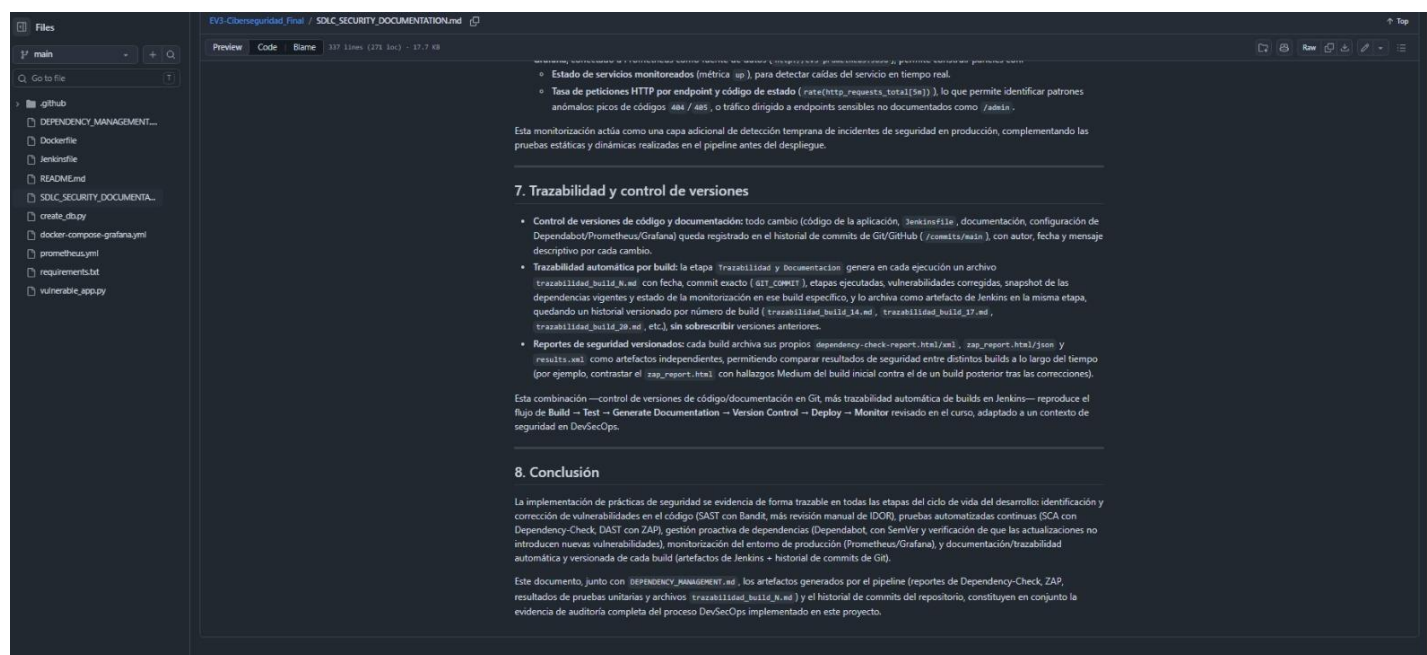
del desarrollo de software.

Captura 87: Estructura del pipeline y detalle de vulnerabilidades corregidas en SDLC_SECURITY_DOCUMENTATION.md.



Descripción: Se visualiza la sección del documento que describe el pipeline (Jenkinsfile) como código dentro del repositorio (Pipeline as Code), ejecutándose en Jenkins ante cada cambio en main mediante Pipeline script from SCM. Se presenta una tabla con las siete etapas del pipeline: Checkout SCM, Build, Test - Unit, Deploy - Staging, Security Test - SCA Dependencies, Security Test - DAST ZAP y Trazabilidad y Documentación, detallando la función de cada una. El documento explica que esta estructura corresponde al modelo de pipeline de documentación y trazabilidad visto en clases (Build → Test → Generate Documentation → Version Control → Deploy → Monitor), adaptado para incorporar etapas de seguridad automatizada (SCA y DAST). Además, se indica que Jenkins conserva el historial completo de builds y archiva individualmente sus reportes sin sobrescribir resultados anteriores. En la sección "3. Revisiones de seguridad y vulnerabilidades identificadas" se detalla la vulnerabilidad SQL Injection (CWE-89), explicando su ubicación original en la función login(), el riesgo asociado a la inyección de sintaxis SQL para eludir la autenticación, y la corrección aplicada mediante consulta parametrizada, evidenciando de forma clara la implementación de prácticas de seguridad en la etapa de desarrollo del código.

Captura 88: Monitorización, trazabilidad y conclusión final de la documentación de seguridad del SDLC.



Descripción: Se visualiza el cierre del documento SDLC_SECURITY_DOCUMENTATION.md. Se describe que Grafana, conectado a Prometheus como fuente de datos, permite construir paneles con el estado de servicios monitoreados (métrica up) para detectar caídas del servicio en tiempo real, y la tasa de peticiones HTTP por endpoint y código de estado, lo que permite identificar patrones anómalos como códigos 404/405 o tráfico dirigido a endpoints sensibles no documentados. En la sección "7. Trazabilidad y control de versiones" se explica que todo cambio de código y documentación queda registrado en el historial de commits de Git/GitHub, que cada build genera automáticamente un archivo de trazabilidad versionado por número de build, y que los reportes de seguridad (Dependency-Check, ZAP, results.xml) se archivan de forma independiente por build, permitiendo comparar resultados a lo largo del tiempo. Finalmente, la sección "8. Conclusión" resume que la implementación de prácticas de seguridad se evidencia de forma trazable en todas las etapas del ciclo de vida del desarrollo: identificación y corrección de vulnerabilidades (SAST con Bandit), pruebas automatizadas continuas (SCA con Dependency-Check, DAST con ZAP), gestión proactiva de dependencias (Dependabot), monitorización del entorno de producción (Prometheus/Grafana), y documentación/trazabilidad automática y versionada de cada build, constituyendo en conjunto la evidencia de auditoría completa del proceso DevSecOps implementado en el proyecto.